



1

Programming Fundamentals
605116

ISRA University
Faculty of IT

Textbook
BARBARA DOYLE
“C# Programming:
From Problem Analysis to
Program Design 4th
Edition

Chapter Objectives

- Introduction
- Learn to differentiate between system and application software
- Learn the steps of software development
- Explore different programming methodologies
- Learn why C# is being used today for software development

Chapter Objectives (continued)

- Distinguish between the different types of applications
- Explore a program written in C#
- Examine the basic elements of a C# program
- Compile, run, build, and debug an application
- Create an application that displays output
- Work through a programming example that illustrates the chapter's concepts

Introduction

- A computer is made up of hardware and software
- *hardware* – the physical, tangible pieces that support the computing effort (e.g. chips, wires, disks, keyboards...). Hardware need instructions to tell it what to do.
- *program* – a series of instructions that the hardware executes one after another
- Programs are sometimes called *applications*

System and Application Software

- Software consists of programs
 - Sets of instructions telling the computer exactly what to do
- Two types of software
 - System
 - Application
- Power of what the computer does lies with what types of software are available

System Software

- System software is more than operating systems
- Operating System
 - Loaded when you power on the computer
 - Examples include Windows 8, Linux, Android, iOS
- Includes compilers, interpreters, and assemblers

Application Software

- Application software performs a specific task
 - Word processors, spreadsheets, payroll, inventory
- Writes instructions using a high-level programming language
 - C#, Java, Visual Basic
- Compiler
 - Translates instructions into machine-readable form
 - First checks for rule violations
 - Syntax rules – how to write statements

High Level

User Level: Application Programs

High Level Languages

Assembly Language / Machine Code

Microprogrammed / Hardwired Control

Functional Units (Memory, ALU, *etc.*)

Logic Gates

Low Level

Transistors and Wires

Software Development Process

- Programming is a process of problem solving
- How do you start?
- Number of different approaches, or methodologies
- Successful problem solvers follow a methodical approach

Programming Language

- A *programming language* specifies the words and symbols that we can use to write a program (to express the problem solution)
- A programming language employs a set of rules that dictate how the words and symbols can be put together to form valid *program statements* (these statements are the instructions that are carried out when the program is executed).

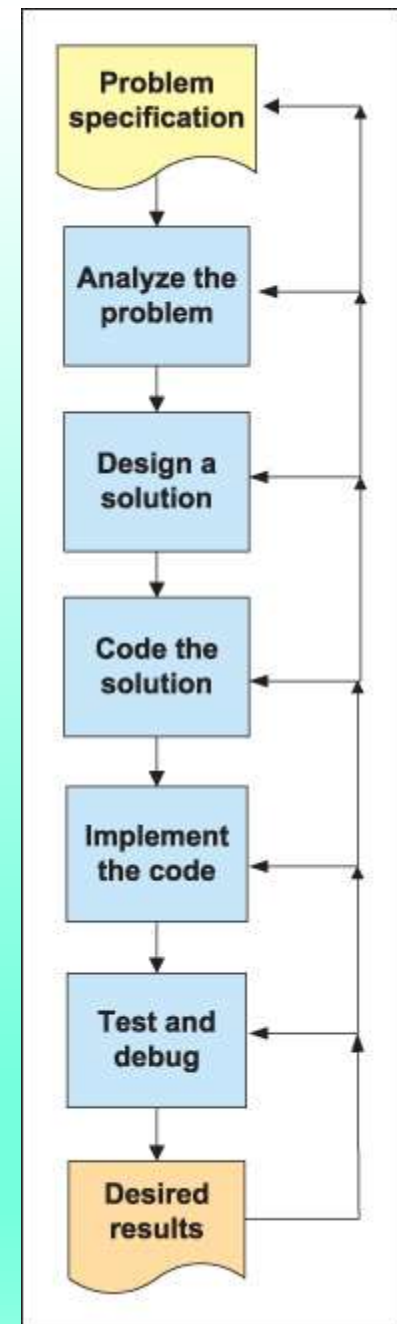
Steps in the Program Development Process

1. Analyze the problem
2. Design a solution
3. Code the solution
4. Implement the code
5. Test and debug

Steps in the Program Development Process (continued)

- Software development process is iterative
- As errors are discovered, it is often necessary to cycle back to a previous phase or step

Figure 1.1 Steps in software development process



Step 1: Analyze the Problem

- Precisely what is software supposed to accomplish?
- Understand the problem definition
- Review the problem specifications

Analyze the Problem (continued)

- What kind of data will be available for input?
- What types of values (i.e., whole numbers, alphabetic characters, and numbers with decimal points) will be in each of the identified data items?
- What is the domain (range of the values) for each input item?
- Will the user of the program be inputting values?
- If the problem solution is to be used with multiple data sets, are there any data items that stay the same, or remain constant, with each set?

Step 2: Design a Solution

- Divide and Conquer approach
 - Break the problem into smaller subtasks
 - Top-down design, stepwise refinement
- Object-oriented approach
 - Focus is on determining data and methods (behaviors)
- Tools used
 - Flowcharts
 - Pseudocode, structured English

Pseudocode

- **Pseudocode** is a plain language description of the steps in an algorithm or another system.
- Pseudocode often uses structural conventions of a normal programming language but is intended for human reading rather than machine reading. It typically omits details that are essential for machine understanding of the algorithm,

Pseudocode example

C style pseudocode:

```
void function fizzbuzz {  
    for (i = 1; i <= 100; i++) {  
        set print_number to true;  
        If i is divisible by 3 {  
            print "Fizz";  
            set print_number to  
false; }  
        If i is divisible by 5 {  
            print "Buzz";  
            set print_number to  
false; }  
        If print_number, print i;  
        print a newline;  
    }  
}
```

What is a Flowchart used for ?

Building a step-by-step picture of the process for analysis, discussion, or communication purposes

Is it possible to shorten the length of time it takes for a student to complete the engineering program?

Defining, standardizing, or finding areas for improvement in a process

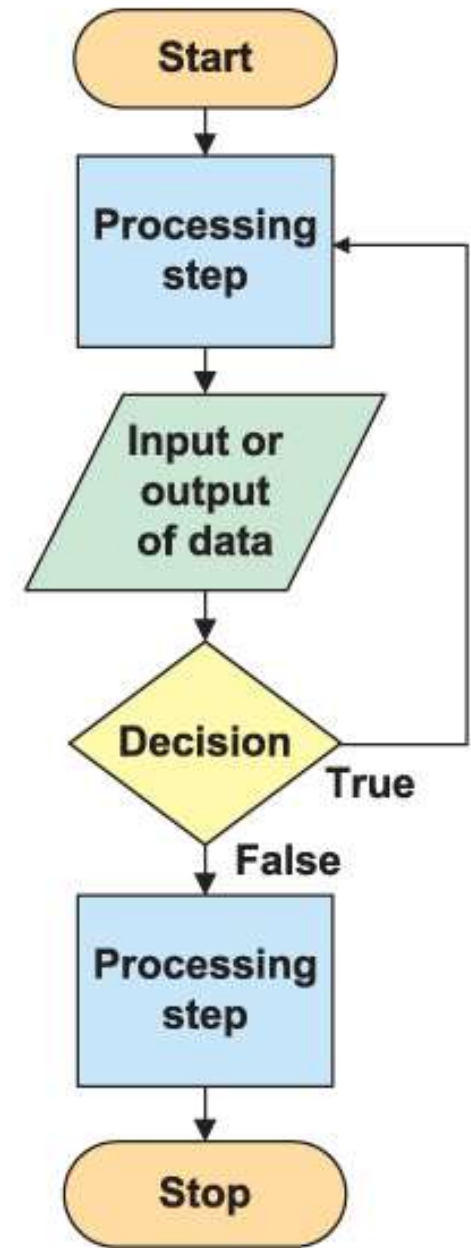
Crystal Clear Communication

- Flowcharts are recognized as one of the most important ways to effectively communicate ideas between people who may differ in knowledge, education, and position
- Without flowcharts, you can spend a ton of energy explaining how the process is to be done!

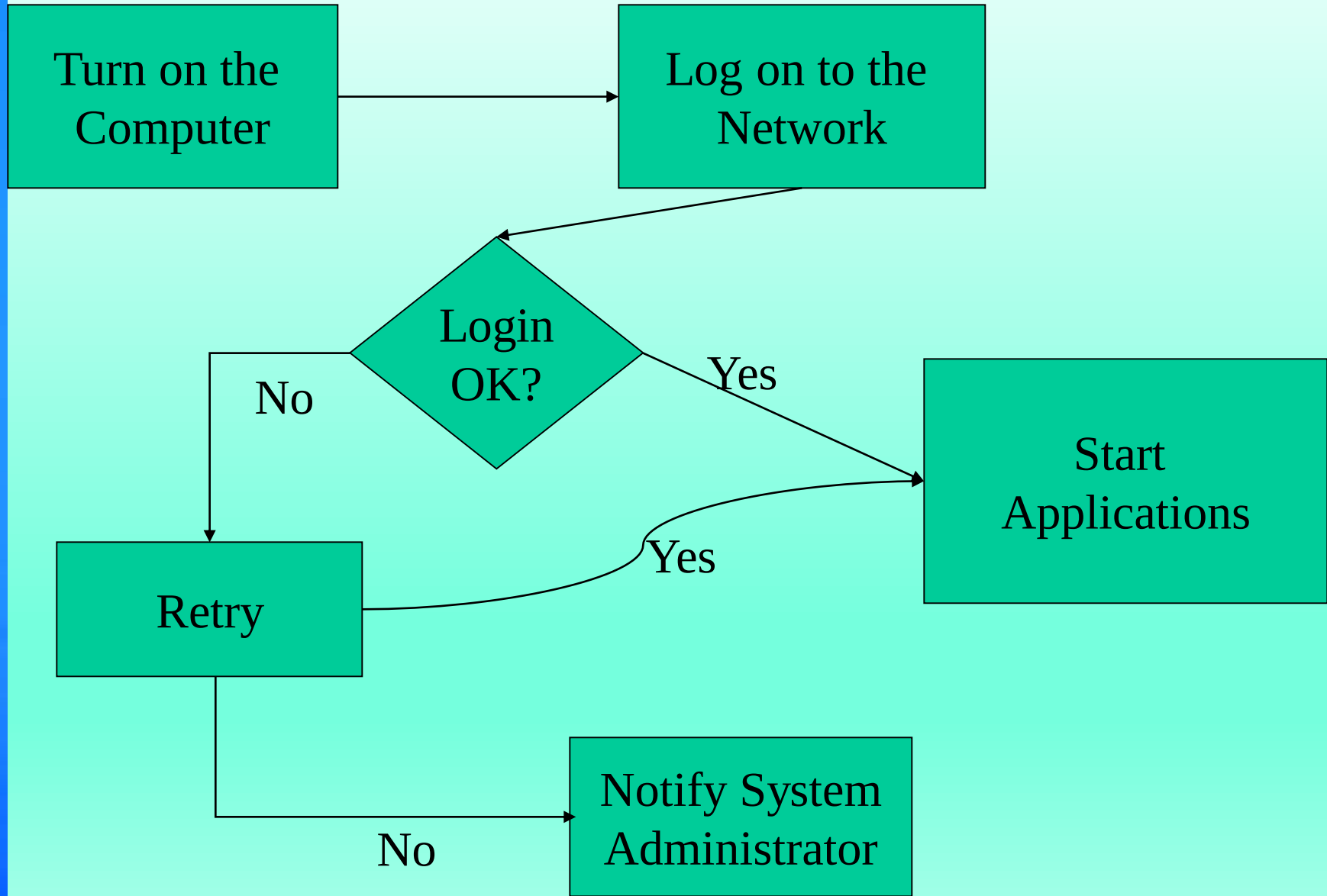
Flowchart

- Oval – beginning and end
- Rectangular – processes
- Diamond – decision to be made
- Parallelogram – inputs and output
- Flow line

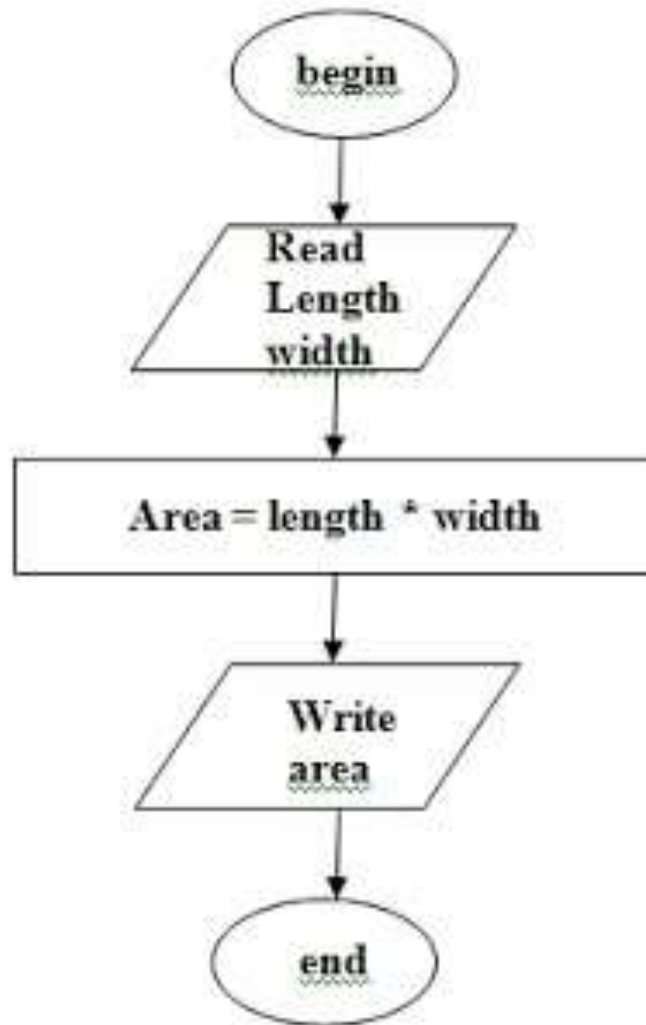
Figure 1-7 Flowchart symbols and their interpretation



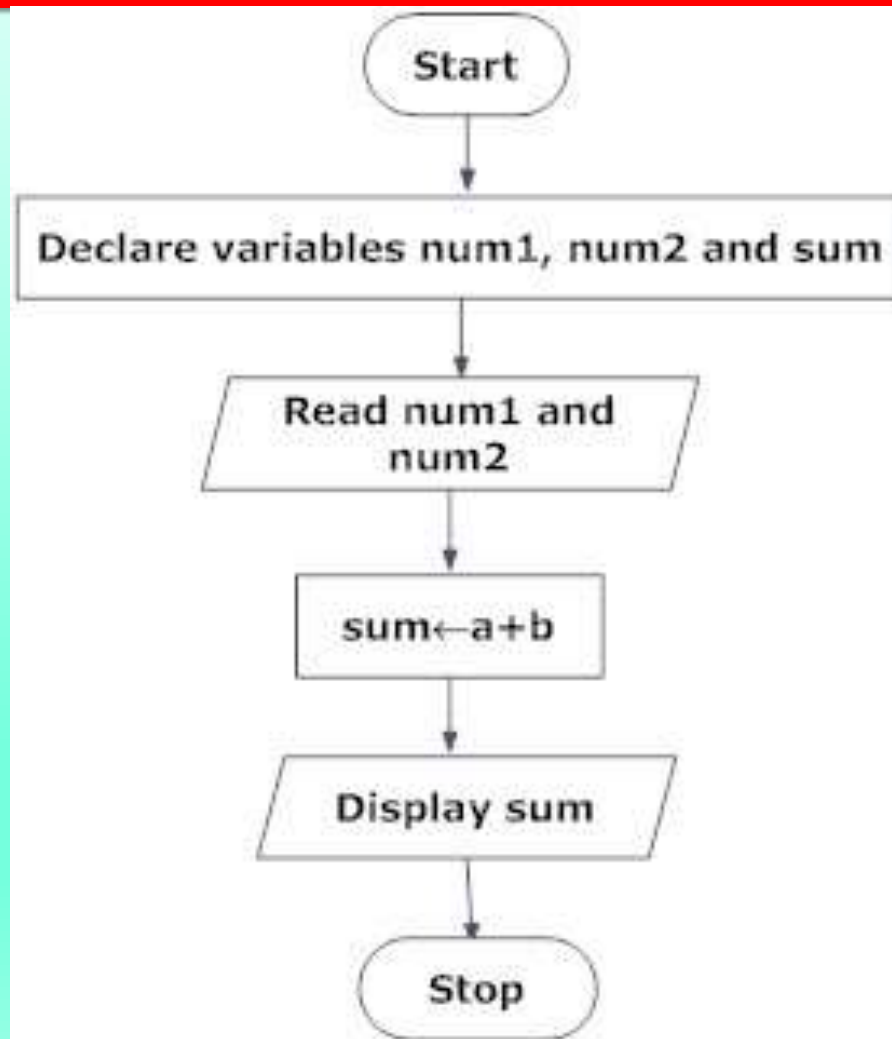
Examples of a Flow Chart



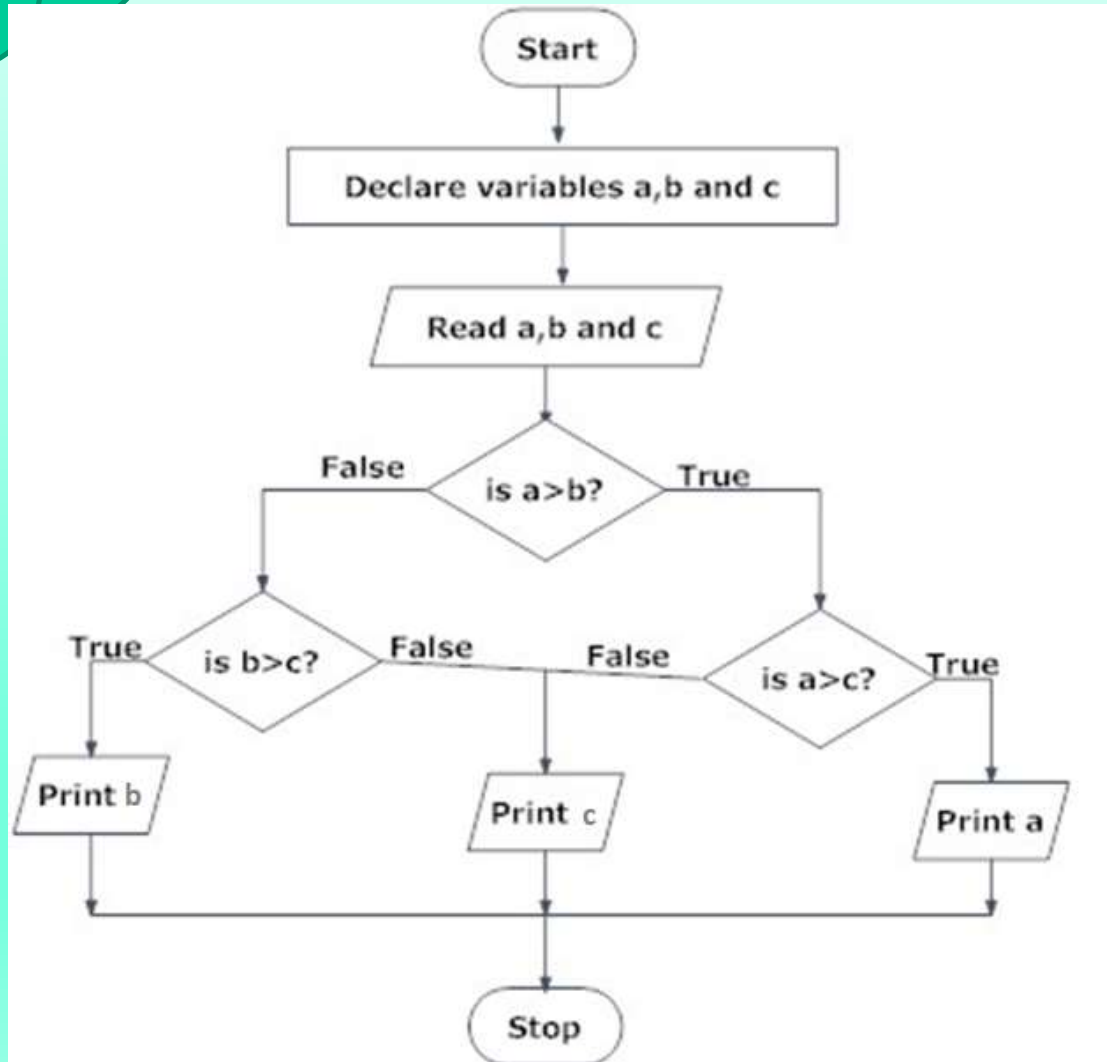
Finding the area of a given rectangle



Draw a flowchart to add two numbers entered by user.



Think, what
does this
flowchart
tends to do?



Step 3: Code the Solution

- Coding means converting the design into *source code*
 - Follow the rules of the language
- Integrated Development Environment (IDE)
 - Visual Studio
 - Tools for typing program statements, compiling, executing, and debugging applications

Step 4: Implement the Code

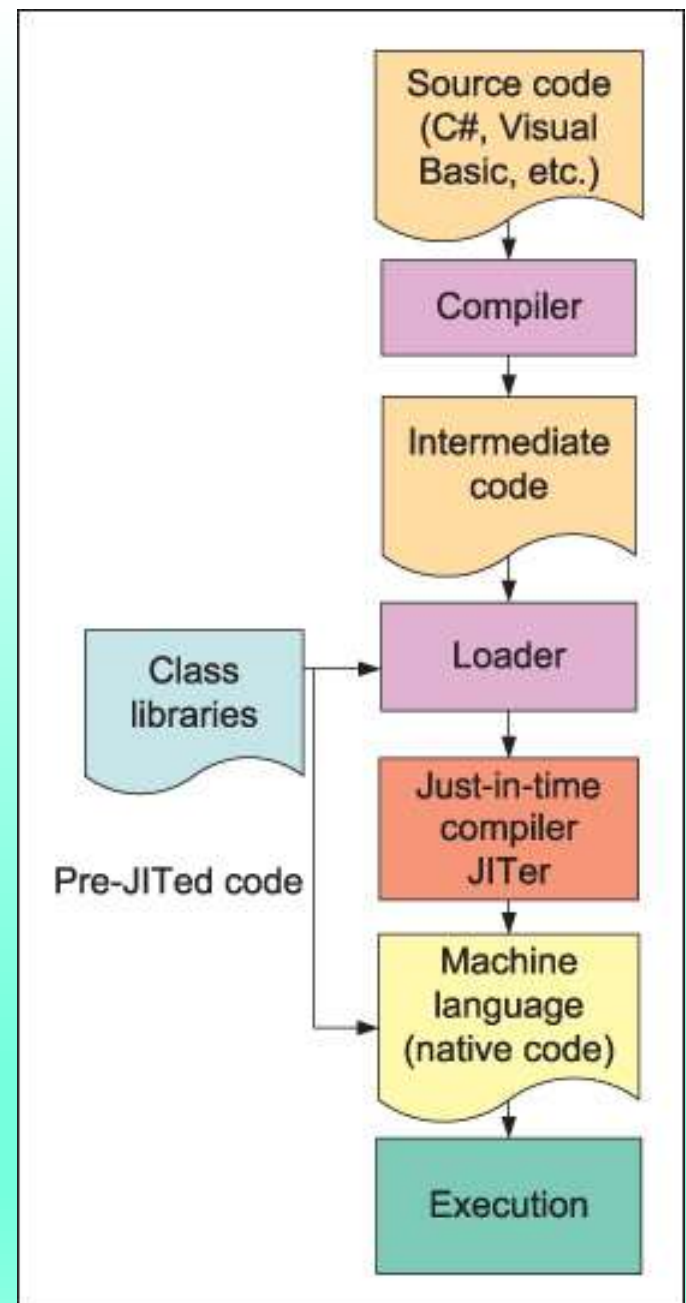
- Source code is compiled to check for rule violations
- C# → Source code is converted into Microsoft Intermediate Language (IL)
 - IL is between high-level source code and native code
 - IL code not directly executable on any computer
 - IL code not tied to any specific CPU platform
- Second step, managed by .NET's Common Language Runtime (CLR), is required

Coding	Programming
Coding means creating codes from one language to another.	Programming means to program a machine to perform using a set of instructions.
It's the primary method to facilitate communication between humans and machines.	Programming is the formal act of writing code but on a much higher level.
It's part of a computer programming procedure which involves translating requirements into a machine-readable language.	Programming is the bigger picture which involves everything from compiling and debugging to testing and implementation.
A coder compartmentalizes lines of code without worrying about the details.	The job of a programmer is to conceptualize and analyze solutions of the problems before they occur.
Coders are programmers but with less expertise.	Programmers are skilled professionals who work on substantial logic to solve complex problems.

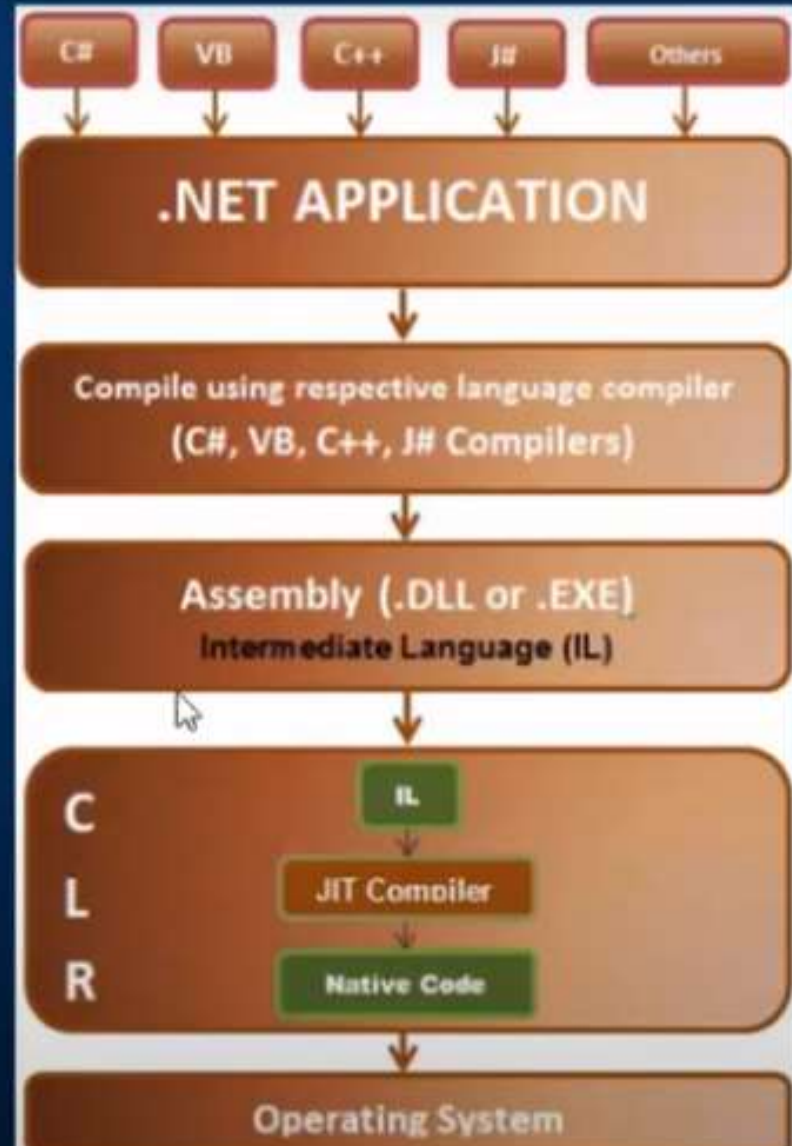
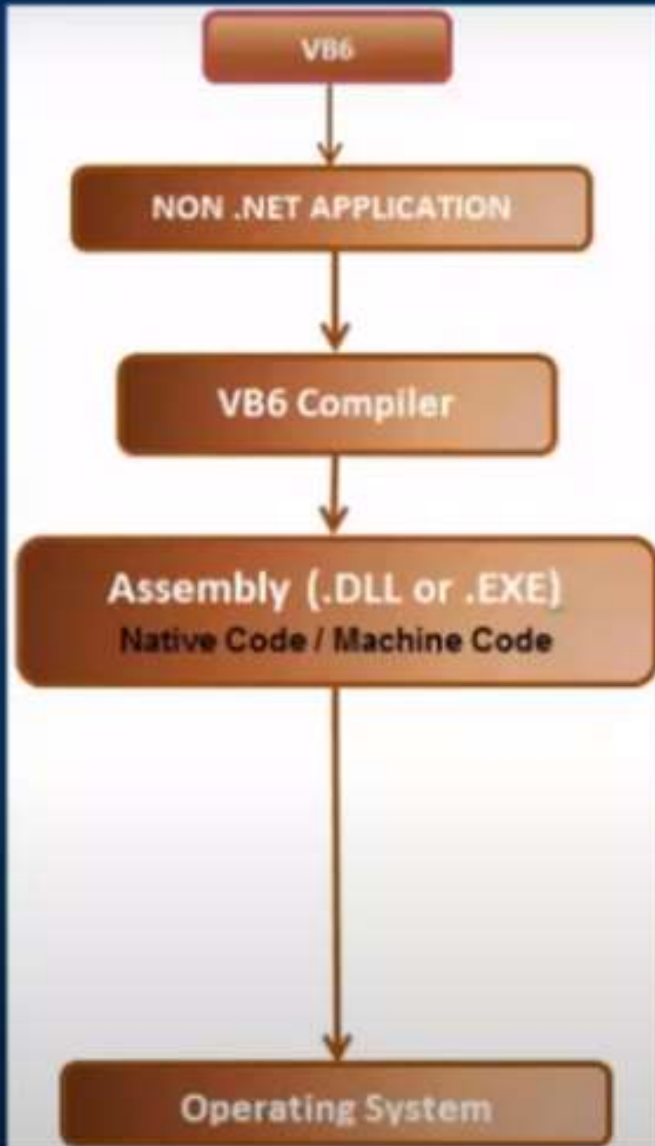
Implement the Code (continued)

- CLR loads .NET classes
- A second compilation, called a just-in-time (JIT) compilation, is performed
 - IL code is converted to the platform's native code

Figure 1-6 Execution steps for .NET



Application Execution



Step 5: Test and Debug

- Test the program to ensure consistent results
- Syntax errors: violations to language rules?
- Logic errors
 - Might cause abnormal termination or incorrect results to be produced
 - Run-time error is one form of logic error

.NET

- Not an operating system
- An environment in which programs run
- Resides at a layer between operating system and other applications
- Offers multi-language independence
 - One application can be written in more than one language
- Includes over 2,500 reusable types (classes)
- Enables creation of dynamic Web pages and Web services

C# Relationship to .NET

- Many compilers targeting the .NET platform are available
- C# was used most heavily for development of the .NET Framework class libraries
- C#, in conjunction with the .NET Framework classes, offers an exciting vehicle to incorporate and use emerging Web standards

Console Applications

- Normally send requests to the operating system
- Display text on the command console
- Easiest to create
 - Simplest approach to learning software development
 - Minimal overhead for input and output of data

Output from the First C# Program

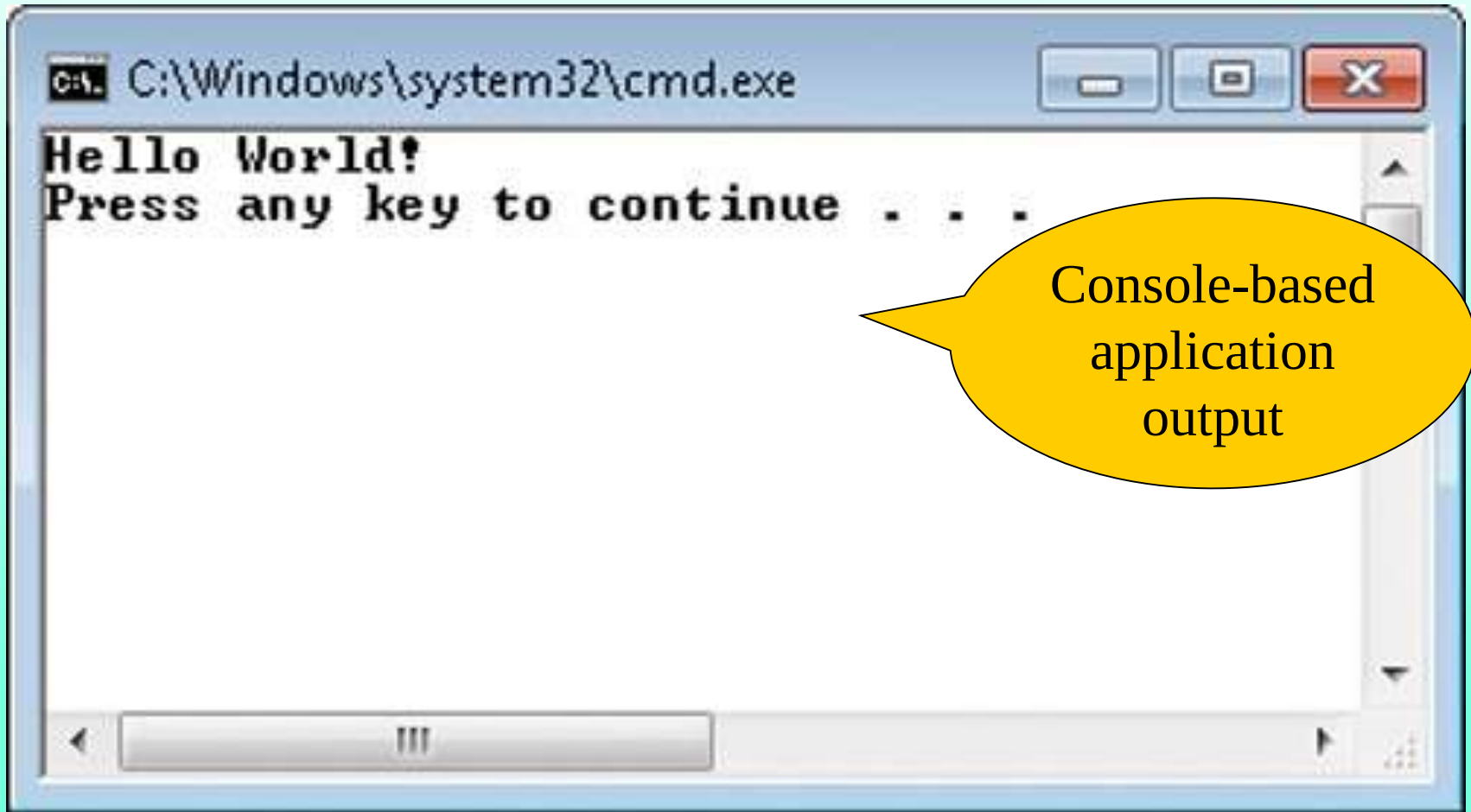


Figure 1-14 Output from Example 1-1 console application

Exploring the First C# Program

From Example 1-1

```
line 1  // This is traditionally the first program written.
line 2  using System;
line 3  namespace HelloWorldProgram
line 4  {
line 5      class Program
line 6      {
line 7          static void Main(string[] args )
line 8          {
line 9              Console.WriteLine("Hello World!");
line 10         }
line 11     }
line 12 }
```

Comments
in green

Keywords
in blue

Classes in
Cyan

Elements of a C# Program

- Comments
 - line 1 // This is traditionally the first program written.
 - Like making a note to yourself or readers of your program
 - Not considered instructions to the computer
 - Not checked for rule violations
 - Document what the program statements are doing

Comments

- Make the code more readable
- Three types of commenting syntax
 - Inline comments //
 - Multiline comments /*
 */

Inline Comments

- Indicated by two forward slashes (//)
- Considered a one-line comment
- Everything to the right of the slashes ignored by the compiler
- Carriage return (Enter) ends the comment

// This is traditionally the first program written.

Multiline Comment

- Forward slash followed by an asterisk (/*) marks the beginning
- Opposite pattern (*/) marks the end
- Also called block comments

/* This is the beginning of a block multiline comment. It can go on for several lines or just be on a single line. No additional symbols are needed after the beginning two characters. Notice there is no space placed between the two characters. To end the comment, use the following symbols.

*/

using Directive

- Permits use of classes found in specific namespaces without having to qualify them
- Framework class library
 - Over 2,000 classes included
- Syntax
 - `using` namespace Identifier;

Example;

```
using System;
```

Namespace

- Namespaces provide scope for the names defined within the group
- Groups semantically related types under a single umbrella
- System: most important and frequently used namespace
- Can define your own namespace
 - Each **namespace** enclosed in curly braces: { }

Namespace (continued)

From Example 1-1

```
line 1    // This is traditionally the first program written.  
line 2    using System;  
line 3    namespace HelloWorldProgram  
line 4    {  
  
line 12   }
```

Predefined namespace
(System)

User-defined
namespace

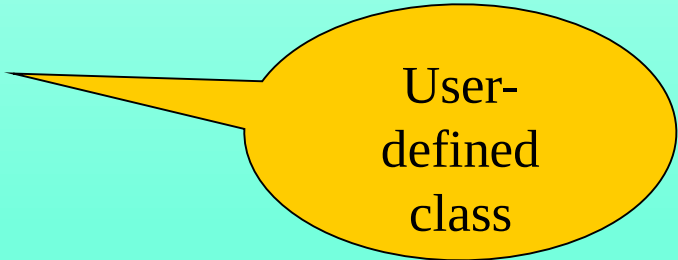
Body of user-
defined
namespace

Class Definition

- Building block of object-oriented program
- Everything in C# is designed around a class
- Every program must have at least one class
- Classes define a category, or type, of object
- Every class is named

Class Definition (continued)

```
line 1    // This is traditionally the first program written.  
line 2    using System;  
line 3    namespace HelloWorldProgram  
line 4    {  
line 5        class HelloWorld  
line 6        {  
  
line 11    }  
line 12    }
```



User-
defined
class

Class Definition (continued)

- Define class members within curly braces
 - Include data members
 - Stores values associated with the state of the class
 - Include method members
 - Performs some behavior of the class
- Can call predefined classes' methods
 - Main()

Main() Method

- “Entry point” for all applications
 - Where the program begins execution
 - Execution ends after last statement in Main()
- Can be placed anywhere inside the class definition
- Applications must have *one* Main() method
- Begins with uppercase character

Main() Method Heading

line 7 static void Main()

- Begins with the keyword static
- Second keyword → return type
 - void signifies no value returned
- Name of the method
 - Main is the name of Main() method
- Parentheses “()” used for arguments
 - No arguments for Main() – empty parentheses

Method Body – Statements

- Enclosed in curly braces
 - Example Main() method body

```
line 7      static void Main( )  
line 8      {  
line 9          Console.WriteLine("Hello World!");  
line 10     }
```

- Includes program statements
 - Calls to other method
- Here Main() calling WriteLine() method

Method Calls

line 9 Console.WriteLine("Hello World!");

- Program statements
- WriteLine() → member of the Console class
- Main() invoking WriteLine() method
- Member of Console class
- Method call ends in semicolon

Program Statements

- Write () → Member of Console **class**
 - Argument(s) enclosed in double quotes inside ()
 - "Hello World!" is the method's argument
 - "Hello World!" is **string** argument
 - String of characters
- May be called with or without arguments
 - Console.WriteLine();
 - Console.WriteLine("Hello .");
 - Console.Write("Hello.");

Program Statements (continued)

- Read() and ReadKey() accept one character from the input device
- ReadLine() accepts string of characters
 - Until the enter key is pressed
- Write() does not automatically advance to next line
- Write("An example\n");
 - Same as WriteLine("An example");
 - Includes special escape sequences

Escape Sequence Characters

Escape sequence character	Description
\n	Cursor advances to the next line; similar to pressing the Enter key
\t	Cursor advances to the next horizontal tab stop
\"	Double quote is printed
\'	Single quote is printed
\\	Backslash is printed
\r	Cursor advances to the beginning of the current line
\a	Alert signal (short beep) is sounded

Table 1-1 Escape sequences

C# Elements

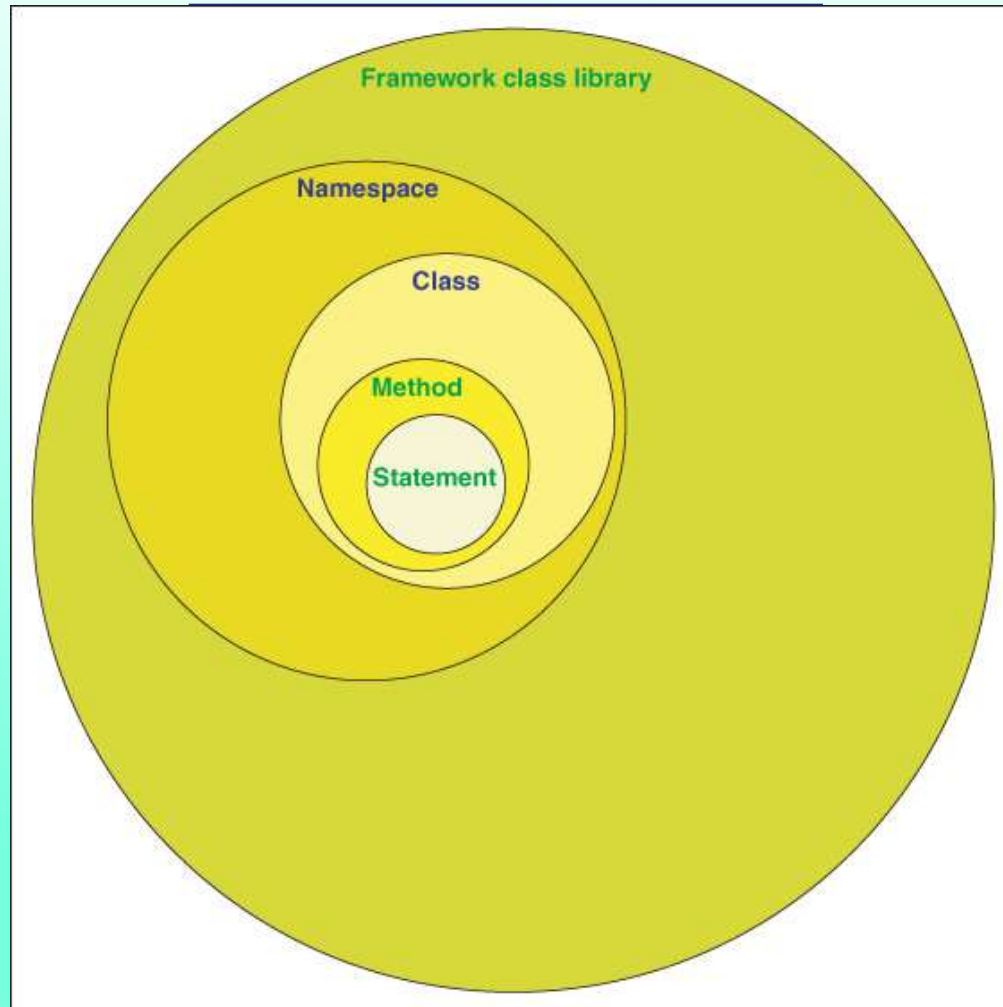


Figure 1-15 Relationship among C# elements

Create Console Application

- Begin by opening Visual Studio
- Create new project
 - Select **New Project** on the Start page
 - OR use **File** → **New Project** option

Create New Project

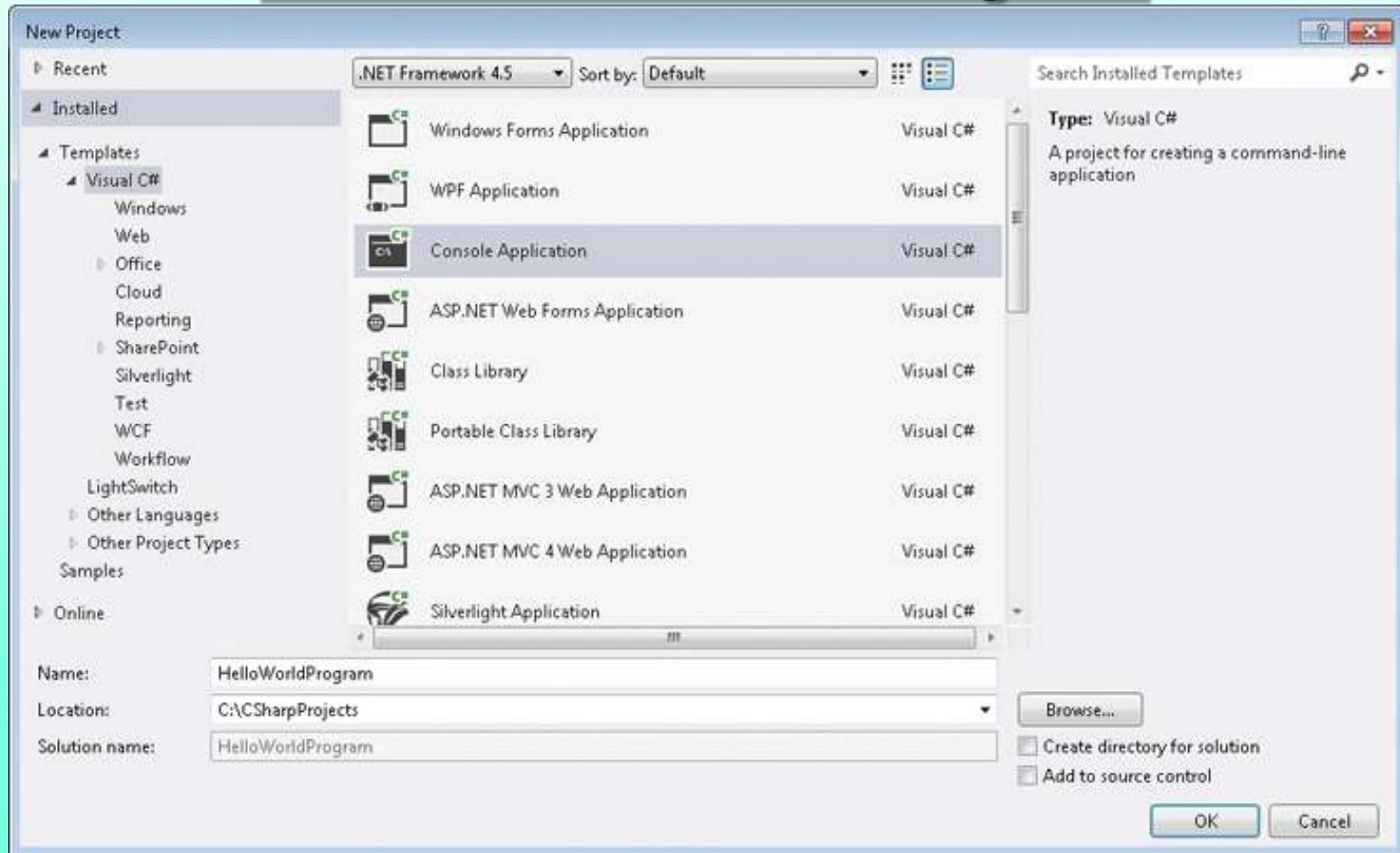


Figure 1-16 Creating a console application

Code Automatically Generated

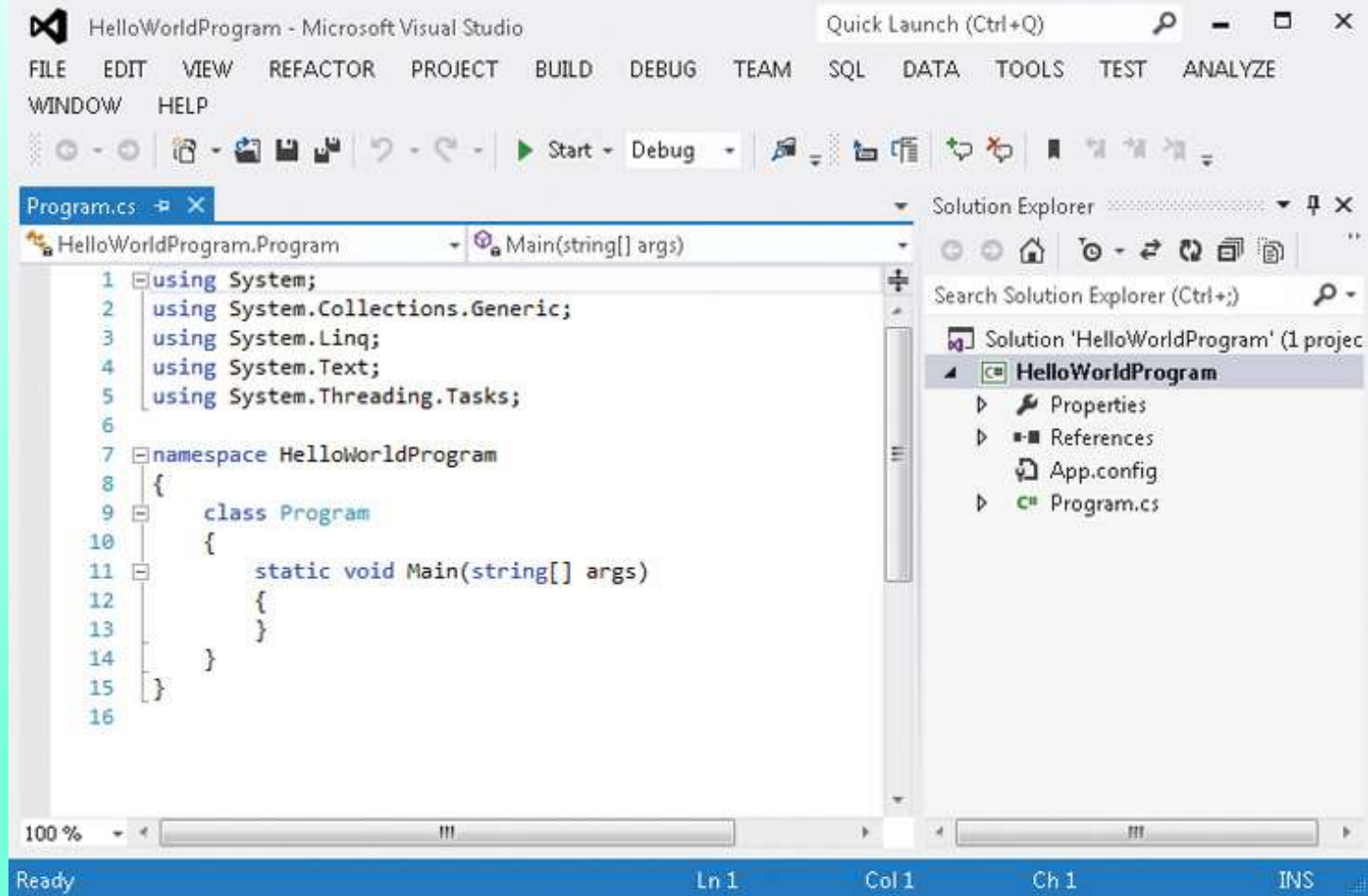


Figure 1-17 Code automatically generated by Visual Studio

Rename Source Code Name

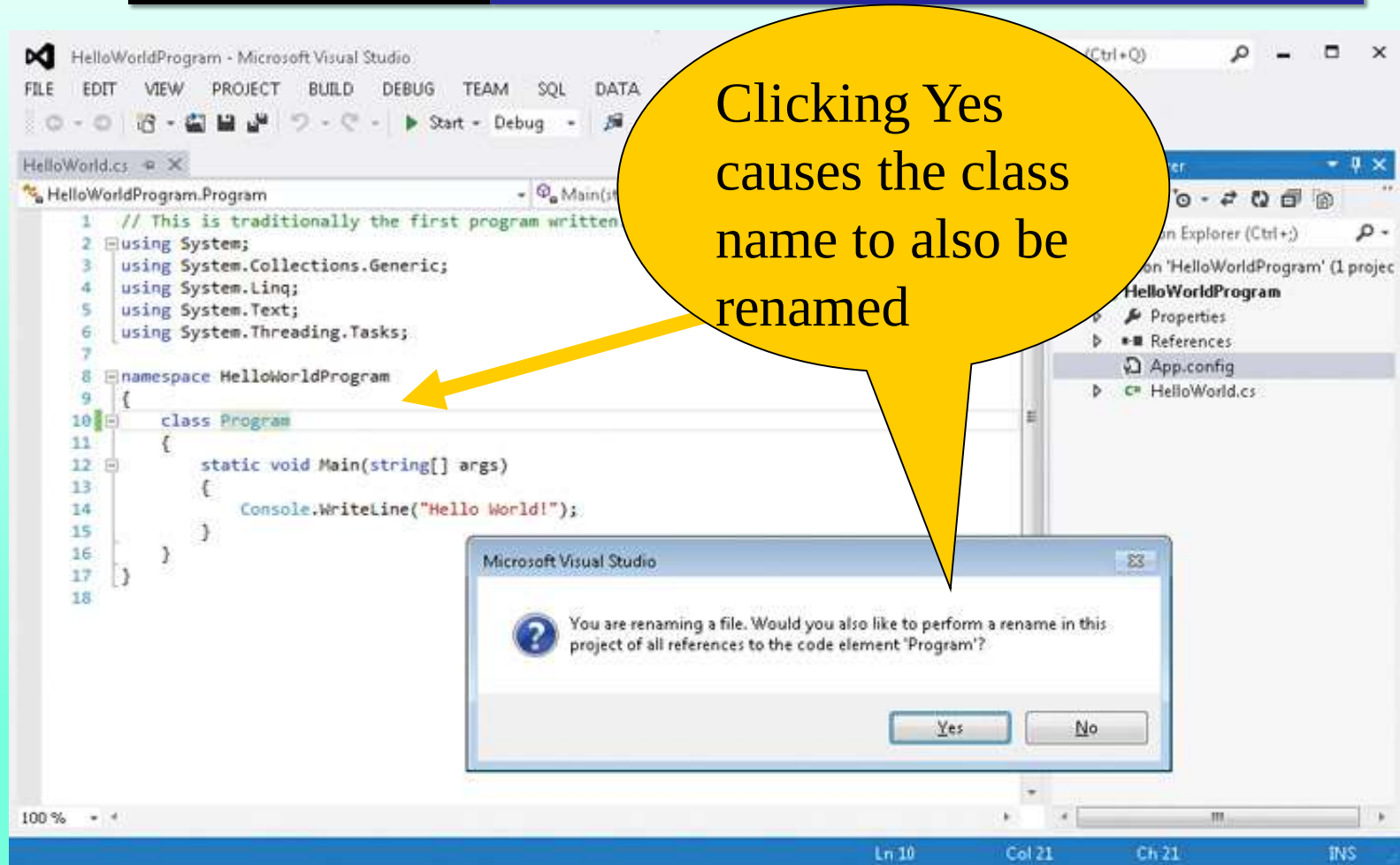


Figure 1-18 Changing the source code name from Program

Compile and Run Application

- To Compile – click Build on the Build menu
- To run or execute application – click Start or Start Without Debugging on the Debug menu
 - Shortcut – if executing code that has not been compiled, automatically compiles first
- Start option does not hold output screen → output flashes quickly
 - Last statement in Main(), could add Console.Read(); or ReadKey();

Build Visual Studio Project

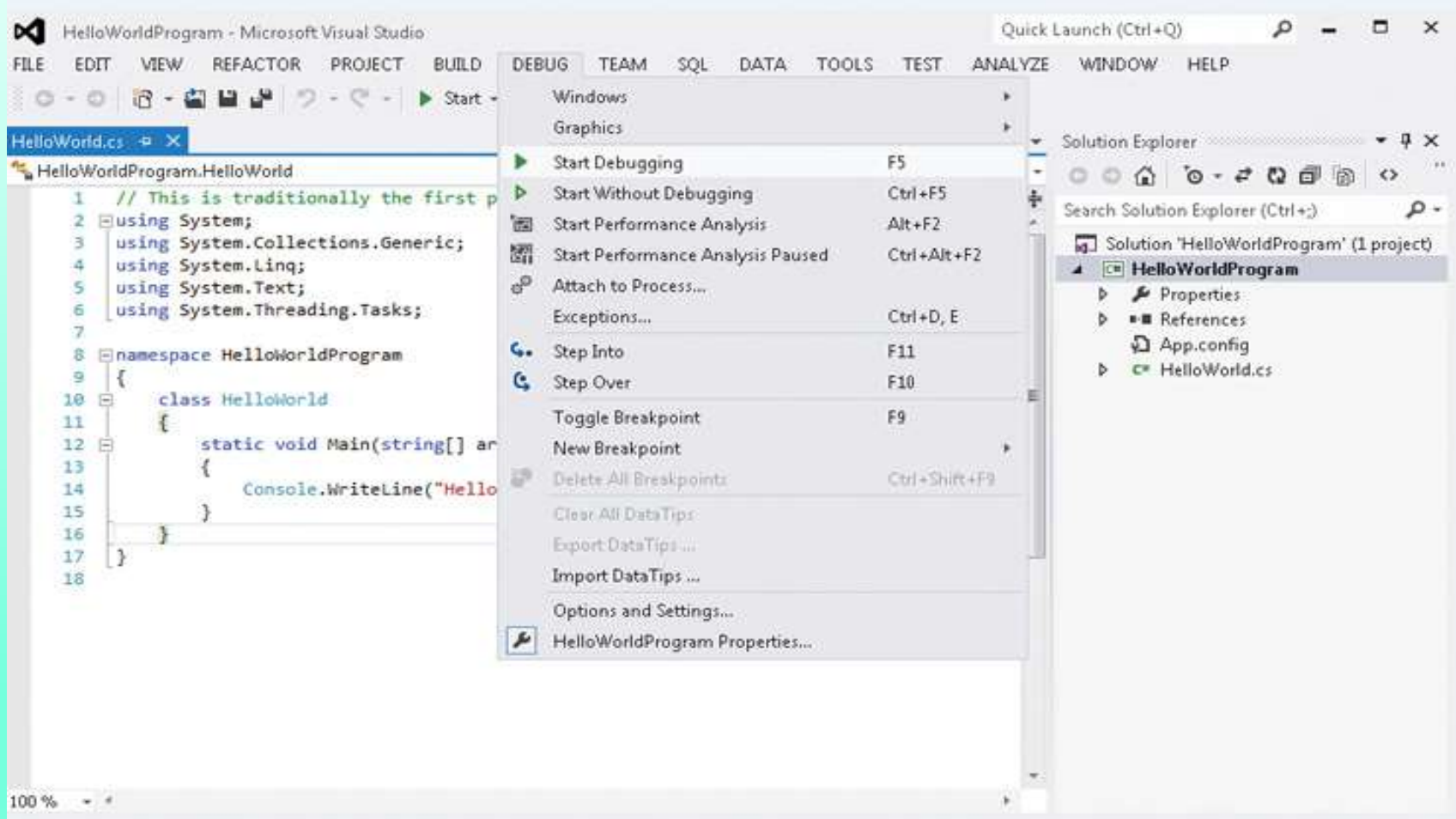


Figure 1-19 Execution of an application using Visual Studio

Debugging an Application

- Types of errors
 - Syntax errors
 - Typing error
 - Misspelled name
 - Forget to end a statement with a semicolon
 - Run-time errors
 - Failing to fully understand the problem
 - More difficult to detect

Error Listing

100 %

1 // This is traditionally the first program written.
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace HelloWorldProgram
9 {
10 class HelloWorld
11 {
12 static void Main(string[] args)
13 {
14 Console.WriteLine("Hello world!");
15 }
16 }
17 }

Missing ending double quote

Pushpin

Errors reported

Error List

3 Errors 0 Warnings 0 Messages

Search Error List

	Description	File	Line	Colu...	Project
1	Newline in constant	HelloWorld.cs	14	31	HelloWorldProgram
2	; expected	HelloWorld.cs	14	46	HelloWorldProgram
3	) expected	HelloWorld.cs	14	46	HelloWorldProgram

Ready Ln 14

Figure 1-20 Syntax error message listing

Creating an Application – ProgrammingMessage Example

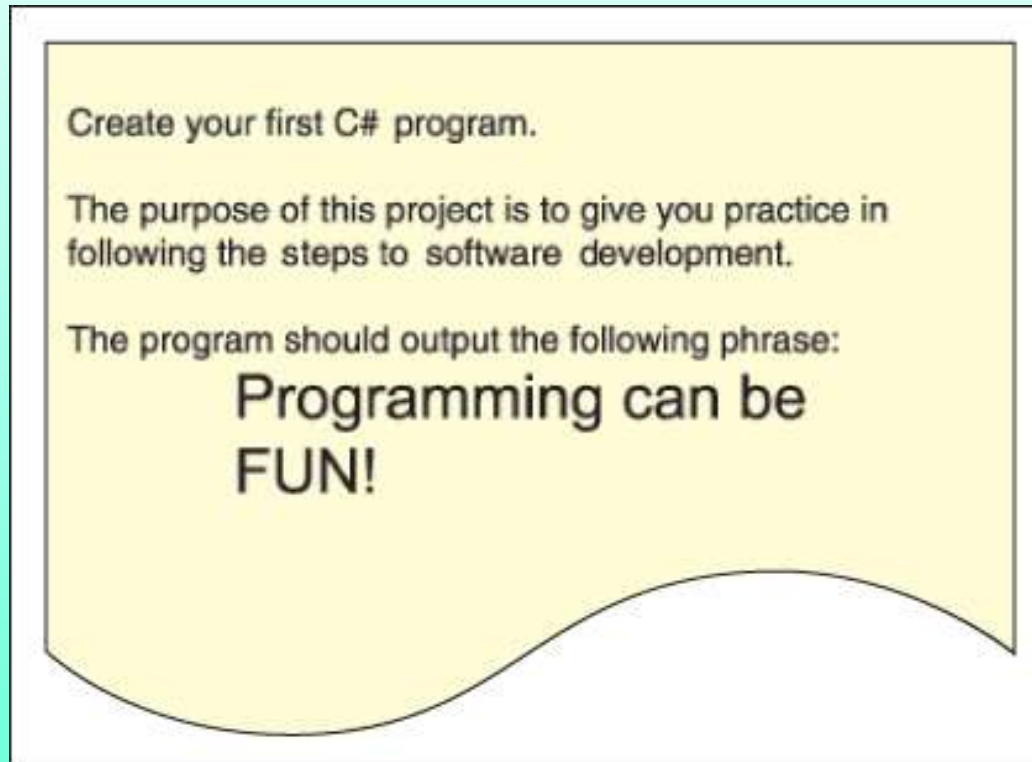


Figure 1-21 Problem specification sheet for the ProgrammingMessage example

ProgrammingMessage Example

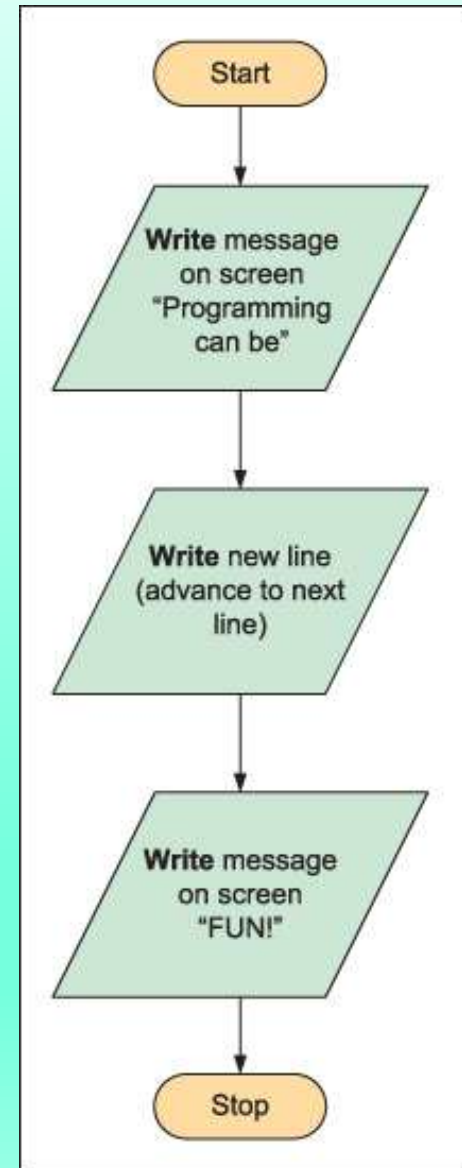


Figure 1-22 Prototype for the ProgrammingMessage example

ProgrammingMessage Example

- Pseudocode would include a single line to display the message "Programming can be FUN!" on the output screen

Figure 1-23 Algorithm for ProgrammingMessage example



ProgrammingMessage

Example

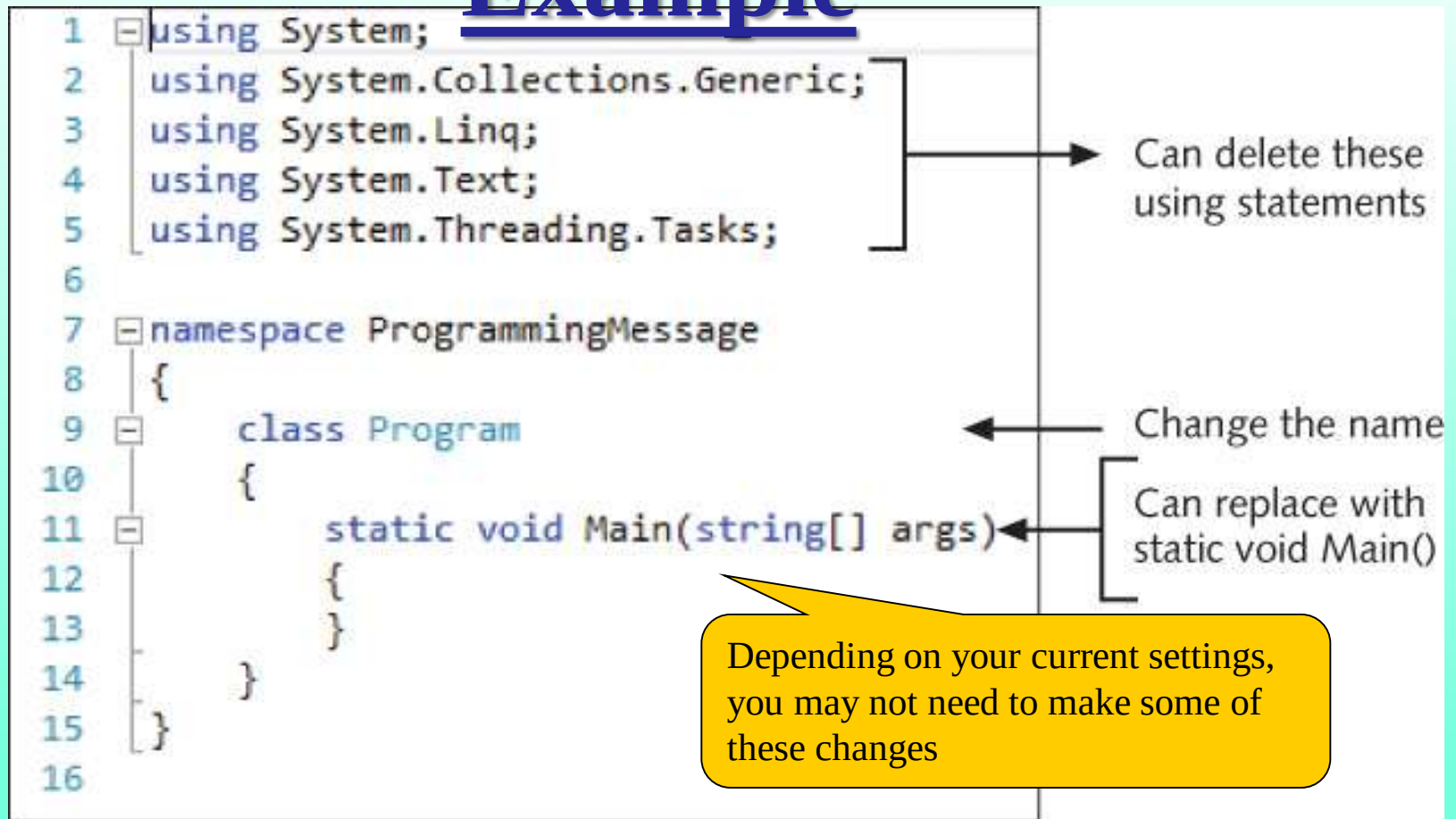


Figure 1-24 Recommended deletions

ProgrammingMessage Example

```
/* Programmer: [supply your name]
```

```
*/
```

```
using System;
```

```
namespace ProgrammingMessage
```

```
{
```

```
class ProgrammingMessage
```

```
{
```

```
static void Main( )
```

```
{
```

```
    Console.WriteLine("Programming can be");
```

```
    Console.WriteLine("FUN! ");
```

```
    Console.ReadKey( );
```

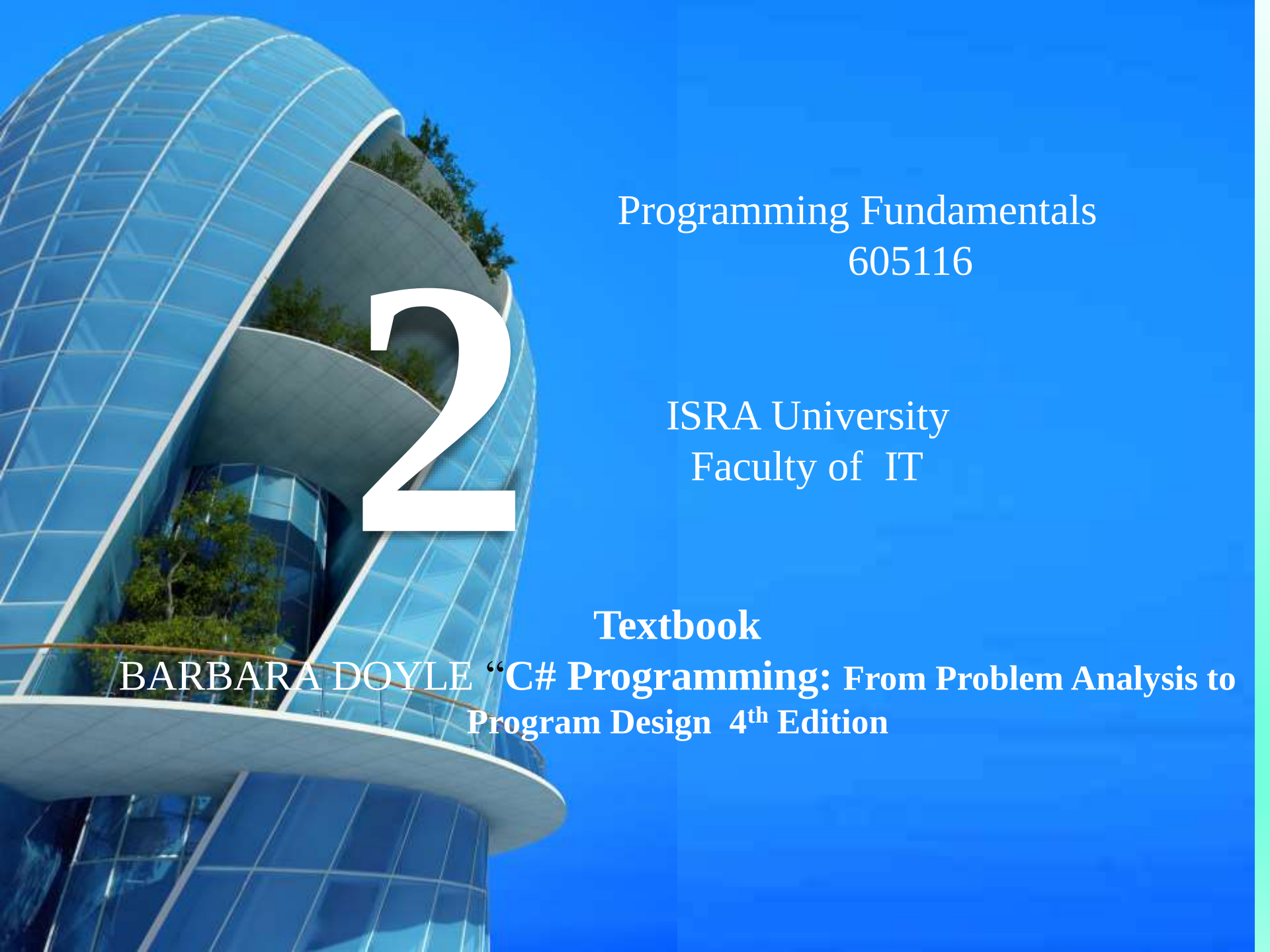
```
}
```

```
}
```

```
}
```



Complete
program
listing



Programming Fundamentals
605116

2

ISRA University
Faculty of IT

Textbook

BARBARA DOYLE **“C# Programming: From Problem Analysis to
Program Design 4th Edition**



2

Data Types and Expressions

C# Programming: From Problem Analysis to Program Design
4th Edition

Chapter Objectives

- Examine how computers represent data
- Declare memory locations for data
- Explore the relationship between classes, objects, and types
- Use predefined data types
- Use integral data types

Chapter Objectives (continued)

- Use floating-point types
- Learn about the decimal data type
- Declare Boolean variables
- Declare and manipulate strings
- Work with constants

Chapter Objectives (continued)

- Write assignment statements using arithmetic operators
- Learn about the order of operations
- Learn special formatting rules for currency
- Work through a programming example that illustrates the chapter's concepts

Data Representation

- Bits
 - Bit – "Binary digIT"
 - Binary digit can hold 0 or 1
 - 1 and 0 correspond to on and off, respectively
- Bytes
 - Combination of 8 bits
 - Represent one character, such as the letter A
 - To represent data, computers use the base-2 number system, or binary number system

Decimal Number System

10^3 (1000) 1	10^2 (100) 3	10^1 (10) 2	10^0 (1) 6
1 times	10^3 (1000)	=	1000
+	3 times	10^2 (100)	= 300
+	2 times	10^1 (10)	= 20
+	6 times	10^0 (1)	= 6
			1326

Figure 2-1 Base-10 positional notation of 1326

Decimal Numbering System uses 10 symbols ranging from 0 to 9 to represent a value.

Binary Numbering System

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
(128)	(64)	(32)	(16)	(8)	(4)	(2)	(1)
0	1	1	0	1	0	0	1
	0 times 2^7 (128)	=	0				
+	1 times 2^6 (64)	=	64				
+	1 times 2^5 (32)	=	32				
+	0 times 2^4 (16)	=	0				
+	1 times 2^3 (8)	=	8				
+	0 times 2^2 (4)	=	0				
+	0 times 2^1 (2)	=	0				
+	1 times 2^0 (1)	=	1				
				105			

- Binary numbering systems uses base 2, and has only two symbols, the values 0 and 1. Therefore a binary value might look like 01110100

Data Representation (continued)

Storage capacity	Size in bytes	Abbreviation
Kilobyte	2^{10} (1,024)	KB
Megabyte	2^{20} (1,048,576)	MB
Gigabyte	2^{30} (1,073,741,824)	GB
Terabyte	2^{40} (1,099,511,627,776)	TB
Petabyte	2^{50} (1,125,899,906,842,624)	PB
Exabyte	2^{60} (1,152,921,504,606,846,976)	EB
Zettabyte	2^{70} (1,180,591,620,717,411,303,424)	ZB
Yottabyte	2^{80} (1,208,925,819,614,629,174,706,176)	YB

Table 2-2 Common abbreviations for data representations

Memory Locations for Data

- Identifier
 - Name
 - Rules for creating an identifier
 - Combination of alphabetic characters (a-z and A-Z), numeric digits (0-9), and the underscore
 - First character in the name may not be numeric
 - No embedded spaces – concatenate (append) words together
 - Keywords cannot be used (See next slide for reserved keywords).
 - Use the case of the character to your advantage
 - Be descriptive with meaningful names

Reserved Words in C#

Keywords					
<code>abstract</code>	<code>as</code>	<code>base</code>	<code>bool</code>	<code>break</code>	<code>byte</code>
<code>case</code>	<code>catch</code>	<code>char</code>	<code>checked</code>	<code>class</code>	<code>const</code>
<code>continue</code>	<code>decimal</code>	<code>default</code>	<code>delegate</code>	<code>do</code>	<code>double</code>
<code>else</code>	<code>enum</code>	<code>event</code>	<code>explicit</code>	<code>extern</code>	<code>false</code>
<code>finally</code>	<code>fixed</code>	<code>float</code>	<code>for</code>	<code>foreach</code>	<code>goto</code>
<code>if</code>	<code>implicit</code>	<code>in</code>	<code>int</code>	<code>interface</code>	<code>internal</code>
<code>is</code>	<code>lock</code>	<code>long</code>	<code>namespace</code>	<code>new</code>	<code>null</code>
<code>object</code>	<code>operator</code>	<code>out</code>	<code>override</code>	<code>params</code>	<code>private</code>
<code>protected</code>	<code>public</code>	<code>readonly</code>	<code>ref</code>	<code>return</code>	<code>sbyte</code>
<code>sealed</code>	<code>short</code>	<code>sizeof</code>	<code>stackalloc</code>	<code>static</code>	<code>string</code>
<code>struct</code>	<code>switch</code>	<code>this</code>	<code>throw</code>	<code>true</code>	<code>try</code>
<code>typeof</code>	<code>uint</code>	<code>ulong</code>	<code>unchecked</code>	<code>unsafe</code>	<code>ushort</code>
<code>using</code>	<code>virtual</code>	<code>volatile</code>	<code>void</code>	<code>while</code>	

Table 2-3 C# keywords/ reserved words

Reserved Words in C# (continued)

- Contextual keywords
 - As powerful as regular keywords
 - Contextual keywords have special meaning only when used in a specific context; other times they can be used as identifiers. For example, **yield** only has a special meaning when it is used immediately before a return statement. Other times, it could be used as an identifier

Contextual keywords					
<code>add</code>	<code>alias</code>	<code>ascending</code>	<code>async</code>	<code>await</code>	<code>descending</code>
<code>dynamic</code>	<code>equals</code>	<code>from</code>	<code>get</code>	<code>global</code>	<code>group</code>
<code>into</code>	<code>join</code>	<code>let</code>	<code>orderby</code>	<code>partial</code>	<code>remove</code>
<code>select</code>	<code>set</code>	<code>value</code>	<code>var</code>	<code>where</code>	<code>yield</code>

Table 2-4 C# contextual keywords

Examples of Valid Names (Identifiers)

Valid identifiers		
studentName	age	numberOfCourses
soc_sec_number	departureTime	course1
AmountOwed	count_of_Values	taxAmount
n	streetAddress	zipCode
roomSize	courseName	x3
moreData	bookTitle	homeRuns
pointsScored	CLUB_NAME	exam4

Table 2-5 Valid identifiers

Examples of Invalid Names (Identifiers)

Invalid identifiers	Description of violation
soc sec number	Embedded space
<code>int</code>	Reserved keyword
3n	Begins with a digit
room #	Special symbol other than underscore and an embedded space
first-name	Special symbol other than underscore
A number	Embedded space
<code>class</code>	Reserved keyword

Table 2-6 Invalid identifier

Variables

- Area in computer memory where a value of a particular data type can be stored
 - Declare a variable
 - Allocate memory
- Syntax
 - type identifier;
- Compile-time initialization
 - Initialize a variable when it is declared
- Syntax
 - type identifier = expression;

Type, Class, and Object Examples

Description	Identifier	Data type	Data
Grade point average	gradePointAverage	double	3.99
Current age	age	int	19
Full name	studentName	string	Elizabeth Hill
Final grade in a course	courseGrade	char	A

Table 2-7 Sample data types

Value Types (continued)

Table 2-8 C# value data types with .NET alias

C# type	Numeric type	.NET alias type
<code>byte</code>	Integral	<code>System.Byte</code>
<code>sbyte</code>	Integral	<code>System.SByte</code>
<code>char</code>	Integral	<code>System.Char</code>
<code>decimal</code>	Decimal	<code>System.Decimal</code>
<code>double</code>	Floating-point	<code>System.Double</code>
<code>float</code>	Floating-point	<code>System.Single</code>
<code>int</code>	Integral	<code>System.Int32</code>
<code>uint</code>	Integral	<code>System.UInt32</code>
<code>long</code>	Integral	<code>System.Int64</code>
<code>ulong</code>	Integral	<code>System.UInt64</code>
<code>short</code>	Integral	<code>System.Int16</code>
<code>ushort</code>	Integral	<code>System.UInt16</code>

Integral Data Types

- Primary difference
 - How much storage is needed
 - Whether a negative value can be stored
- Includes number of types
 - byte & sbyte
 - char
 - int & uint
 - long & ulong
 - short & ushort

Data Types

C# type	Numeric range	Width in bits
<code>byte</code>	0 to 255	8-bit
<code>sbyte</code>	-128 to 127	8-bit
<code>char</code>	U+0000 to U+ffff	16-bit
<code>int</code>	-2,147,483,648 to 2,147,483,647	32-bit
<code>uint</code>	0 to 4,294,967,295	32-bit
<code>long</code>	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	64-bit
<code>ulong</code>	0 to 18,446,744,073,709,551,615	64-bit
<code>short</code>	-32,768 to 32,767	16-bit
<code>ushort</code>	0 to 65,535	16-bit

Table 2-9 Values and sizes for integral types

Examples of Integral Variable Declarations

```
int studentCount;           // number of students in the class
int ageOfStudent = 20;      // age - originally initialized to 20
int numberOfExams;          // number of exams
int coursesEnrolled;        // number of courses enrolled
```

Floating-Point Types

- May be in scientific notation with an exponent
- $n.ne \pm P$
 - 3.2e+5 is equivalent to 320,000
 - 1.76e-3 is equivalent to .00176
- OR in standard decimal notation
- Default type is double

Type	Numeric range	Precision	Size
<code>float</code>	$\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$	7 digits	32-bit
<code>double</code>	$\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$	15–16 digits	64-bit

Table 2-10 Values and sizes for floating-point types

Examples of Floating-Point Declarations

```
double extraPerson = 3.50;    // extraPerson originally set
                               // to 3.50
double averageScore = 70.0;  // averageScore originally set
                               // to 70.0
double priceOfTicket;        // cost of a movie ticket
double gradePointAverage;    // grade point average
float totalAmount = 23.57f;   // note the f must be placed after
                               // the value for float types
```

Decimal Types

- Monetary data items
- As with the float, must attach the suffix 'm' or 'M' onto the end of a number to indicate decimal
 - Float attach 'f' or 'F'

Type	Numeric range	Precision	Size
<code>decimal</code>	1.0×10^{-28} to 7.9×10^{28}	28–29 significant digits	128 bits

Table 2-11 Value and size for decimal data type

- Examples
 - `decimal` endowmentAmount = 33897698.26M;
 - `decimal` deficit;

Boolean Variables

- Based on true/false, on/off logic
- Boolean type in C# → bool
- Does not accept integer values such as 0, 1, or -1

```
bool undergraduateStudent;  
bool moreData = true;
```

Strings

- Reference type
- Represents a string of Unicode characters

```
string studentName;  
string courseName = "Programming  
I";  
  
string twoLines = "Line1\nLine2";
```

Making Data Constant

- Add the keyword `const` to a declaration
- Value cannot be changed
- Standard naming convention
- Syntax
 - `const type identifier = expression;`

```
const double TAX_RATE = 0.0675;
```

```
const int SPEED = 70;
```

```
const char HIGHEST_GRADE = 'A';
```


Assignment Statements

- Used to change the value of the variable
 - Assignment operator (=)
- Syntax
 - variable = expression;
- Expression can be:
 - Another variable
 - Compatible literal value
 - Mathematical equation
 - Call to a method that returns a compatible value
 - Combination of one or more items in this list

Examples of Assignment Statements

```
int    numberOfMinutes,  
        count,  
        minIntValue;
```

```
numberOfMinutes = 45;  
count = 0;  
minIntValue = -2147483648;
```

Examples of Assignment Statements

```
char    firstInitial,  
        yearInSchool,  
        punctuation,  
        enterKey,  
        lastChar;
```

```
firstInitial = 'B';  
yearInSchool = '1';  
punctuation = ';';  
enterKey = '\n';           // newline escape character  
lastChar = '\u005A';       // Unicode character 'Z'
```

Examples of Assignment Statements (continued)

```
double accountBalance, weight;
```

```
bool isFinished;
```

```
accountBalance = 4783.68;
```

```
weight = 1.7E-3;           //scientific notation may be used
```

```
isFinished = false;        //declared previously as a bool
```

```
//Notice – no quotes used
```

Examples of Assignment Statements (continued)

```
decimal    amountOwed,  
           deficitValue;
```

```
amountOwed = 3000.50m; // m or M must be suffixed to  
                        // decimal data types
```

```
deficitValue = -322888672.50M;
```

```
string result;
```

```
string fullName;
```

Examples of Assignment Statements (continued)

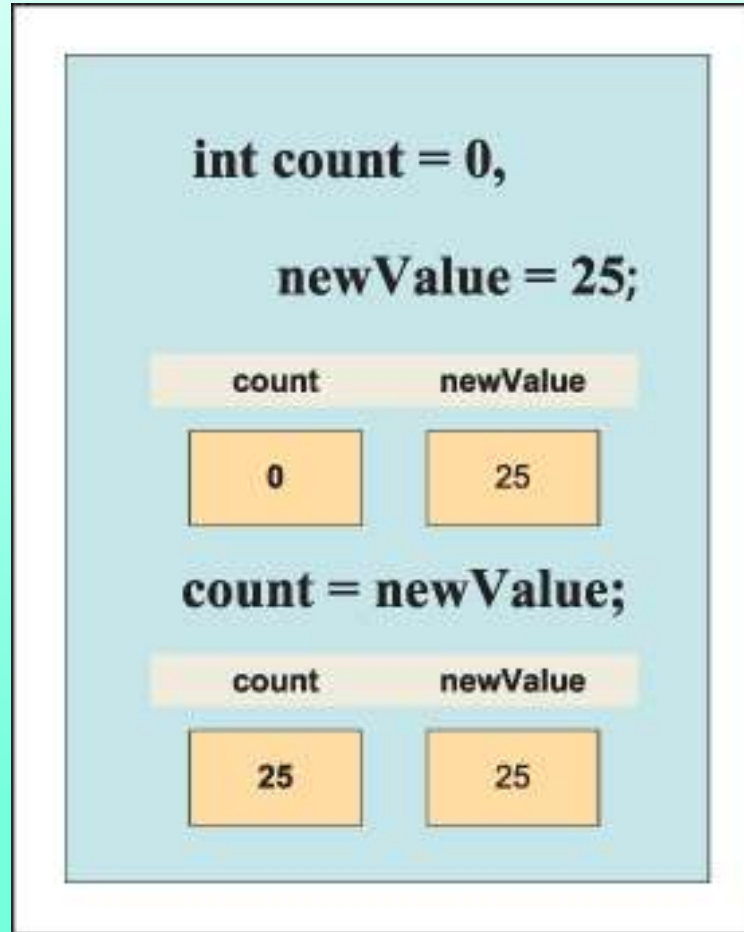


Figure 2-7 Impact of assignment statement

Arithmetic Operations

- Simplest form of an assignment statement
resultVariable = operand1 operator operand2;
- Readability
 - Space before and after every operator

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Table 2-12 Basic arithmetic operators

Basic Arithmetic Operations

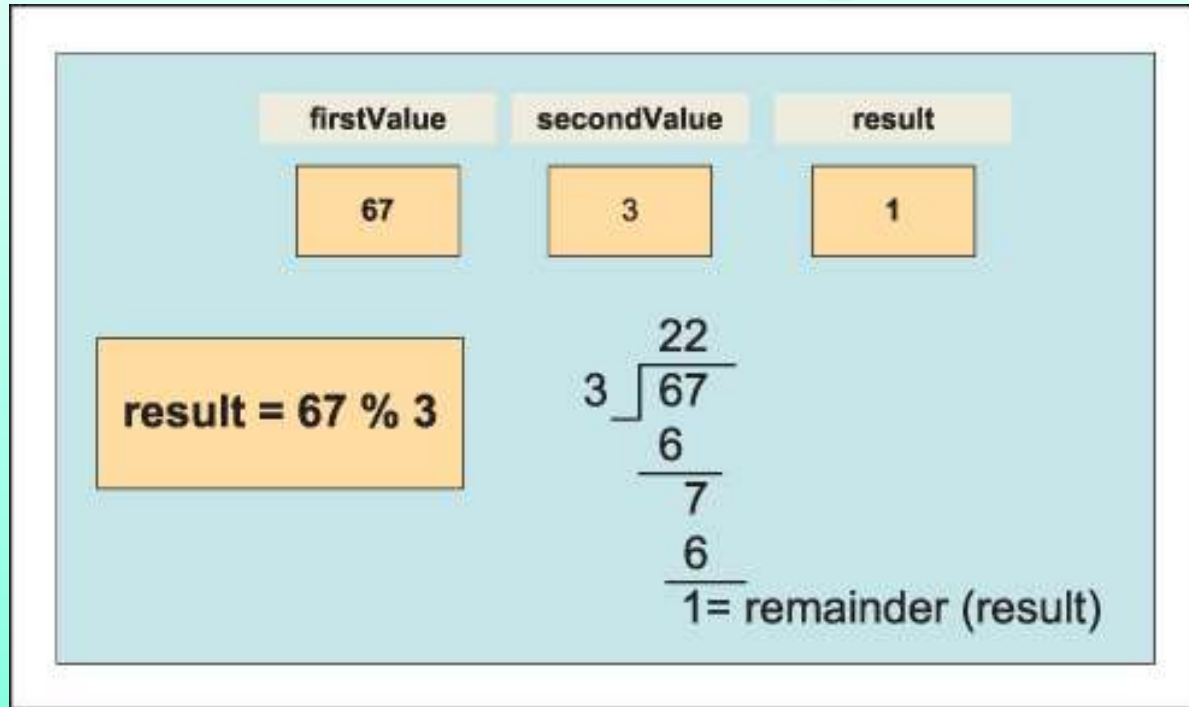


Figure 2-8 Result of $67 \% 3$

- Modulus operator with negative values
 - Sign of the dividend determines the result
 - $-3 \% 5 = -3$; $5 \% -3 = 2$; $-5 \% -3 = -2$;

Division and Remainder

- If both operands to the division operator (/) are integers, the result is an integer (the fractional part is discarded) $14 / 3$ equals 4

$$8 / 12 \text{ equals } 0$$

- The remainder operator (%) returns the remainder after dividing the second operand into the first

$$14 \% 3 \text{ equals } 2$$

$$8 \% 12 \text{ equals } 8$$

– Division (/) operator produces

- If both operands are integer, the / operator performs integer division meaning that any fractional part of the result is discarded
- If one or the other or both operands are floating point values, the / operator performs floating point division and the fractional part of result is kept.

• Ex.

$$\begin{array}{lcl} 10/4 & \longrightarrow & 2 \\ 10.0/4 & & \\ 10/4.0 & \left. \vphantom{\begin{array}{l} 10.0/4 \\ 10/4.0 \end{array}} \right\} & \\ 10.0/4.0 & & 2.5 \end{array}$$

Basic Arithmetic Operations

(continued)

- Plus (+) with string Identifiers
 - Concatenates operand2 onto end of operand1

```
string result;
```

```
string fullName;
```

```
string firstName = "Rochelle";
```

```
string lastName = "Howard";
```

```
fullName = firstName + " " + lastName;
```

Concatenation

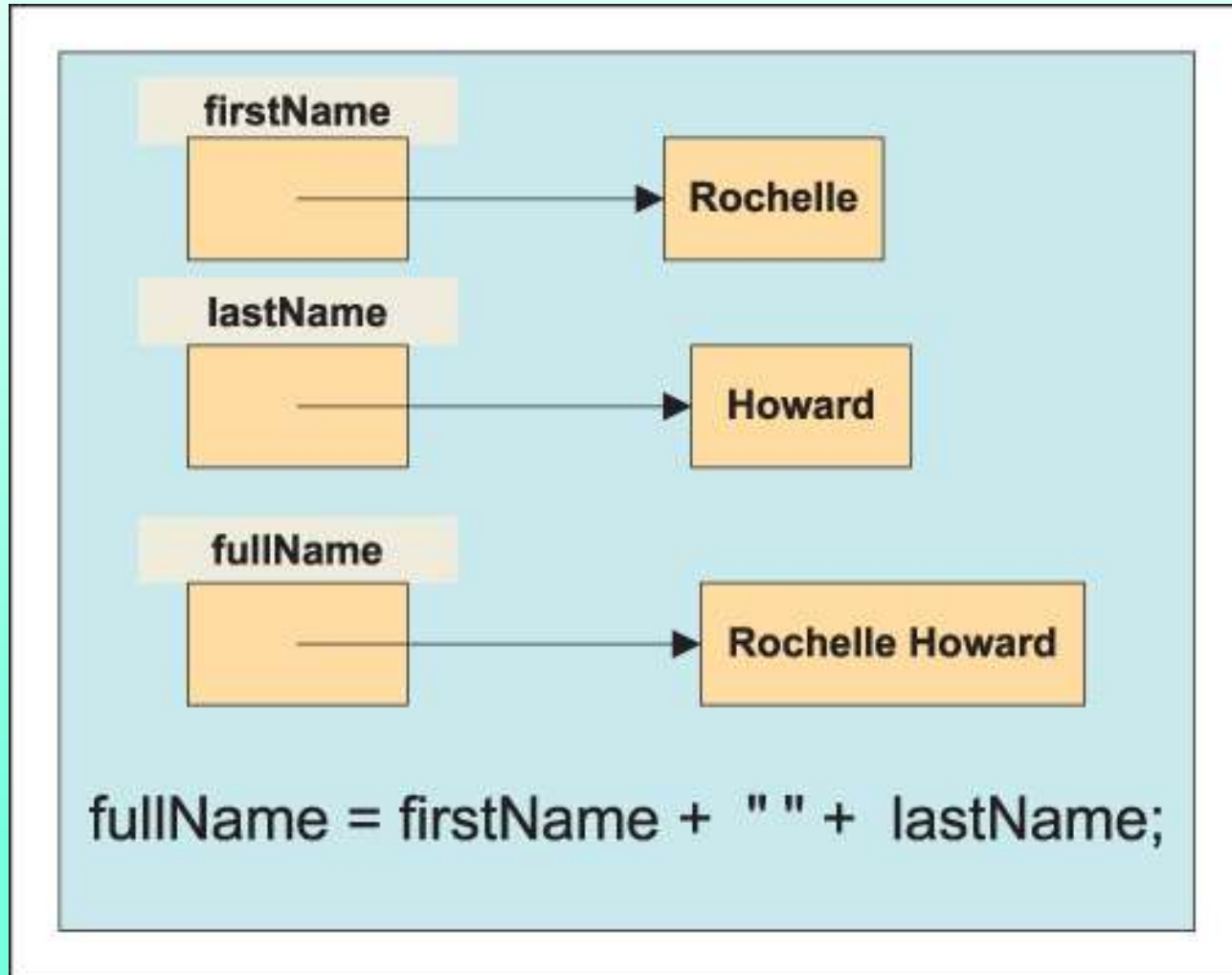


Figure 2-9 String concatenation

Addition or String Concatenation?

```
Console.WriteLine ("24 and 45 concatenated: " + 24 + 45);
```

```
Console.WriteLine ("24 and 45 added: " + (24 + 45));
```

- **Basic Rules:**

1. The + operator is evaluated left to right by default.
2. The function that it performs depends on the data type of the information on which it operates.

- **Usage:**

- If both operands are Strings, or if one is a String and one is a number, it performs String concatenation as in:

- ```
Console.WriteLine ("1 plus 2 is " + 1 + 2);
```

- Output: 1 plus 2 is 12 **<-Note: Big Problem here...**

- Parentheses can be used to override the default order:

- ```
Console.WriteLine ("1 plus 2 is " + (1 + 2));
```

- » Output: 1 plus 2 is 3 **<-Note: Problem Fixed!**

- If both operands are numeric, it performs addition:

- ```
Console.WriteLine (1 + 2);
```

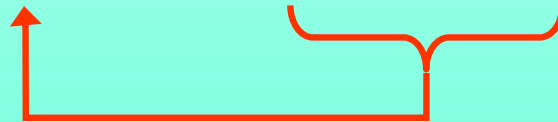
- Output: 3

# Assignment Revisited

- The right and left hand sides of an assignment statement can contain the same variable

First, one is added to the  
original value of count

```
count = count + 1;
```



Then the result is stored back into count  
(overwriting the original value)

# Increment and Decrement

- The increment and decrement operators use only one operand
- The *increment operator* (++) adds one to its operand
- The *decrement operator* (--) subtracts one from its operand
- They are both unary operators because they operate on only on operand
- The statement

```
count++;
```

is functionally equivalent to

```
count = count + 1;
```

# Increment and Decrement

- The increment and decrement operators can be applied in *postfix form*

`count++`

- or *prefix form*

`++count`

- When used alone, the prefix and postfix forms are functionally equivalent.
- When used as part of a larger expression, the two forms can have different effects
- Because of their subtleties, the increment and decrement operators should be used with care



# examples

What are the outputs of the following expressions:

```
int c=1;
```

```
Console.WriteLine(++c);
```

```
Console.WriteLine(c);
```

```
int c=1;
```

```
Console.WriteLine(c++);
```

```
Console.WriteLine(c);
```

```
int num = 100;
```

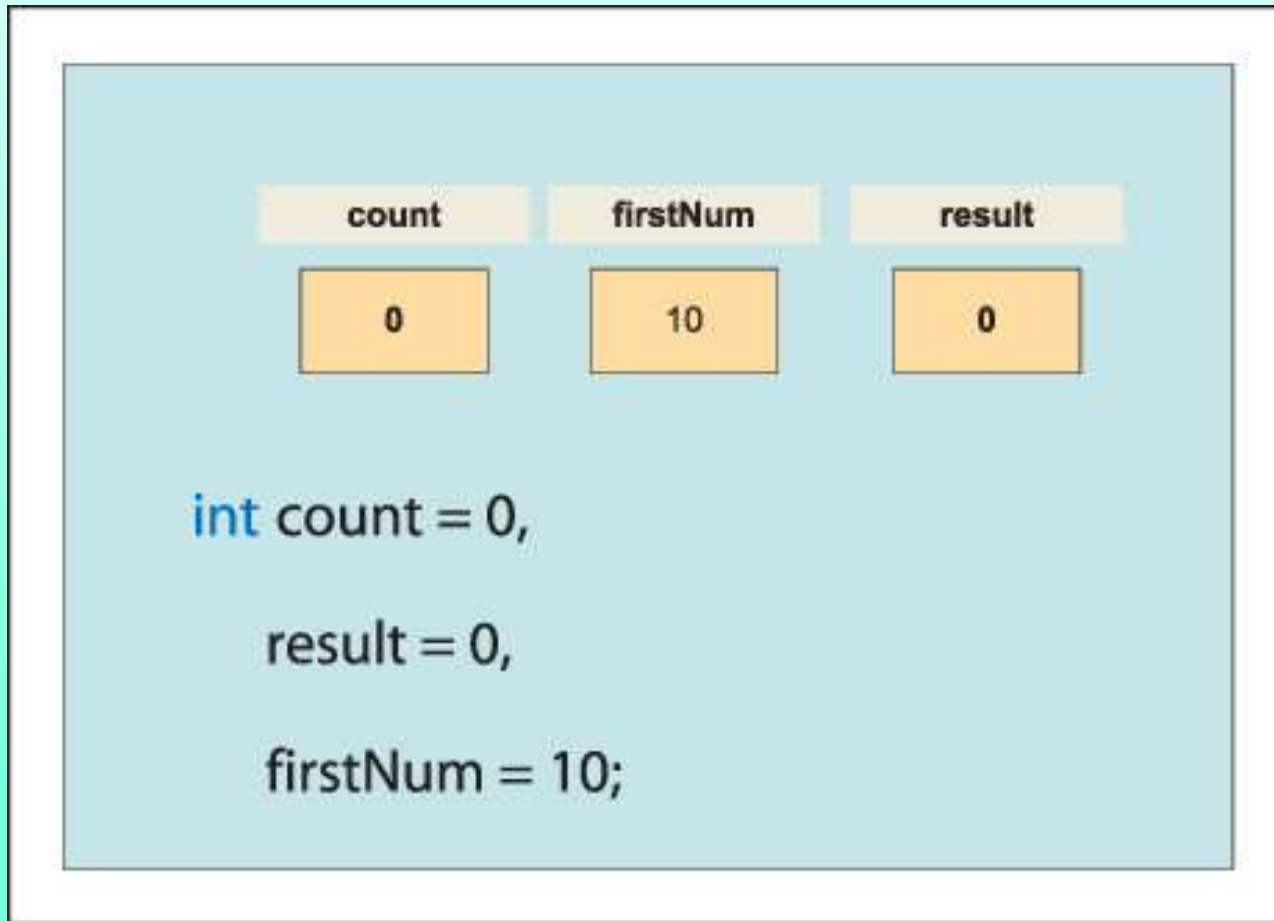
```
Console.WriteLine(num++); // Displays 100
```

```
Console.WriteLine(num); // Display 101
```

```
Console.WriteLine(++num); // Displays 102
```

# Basic Arithmetic Operations

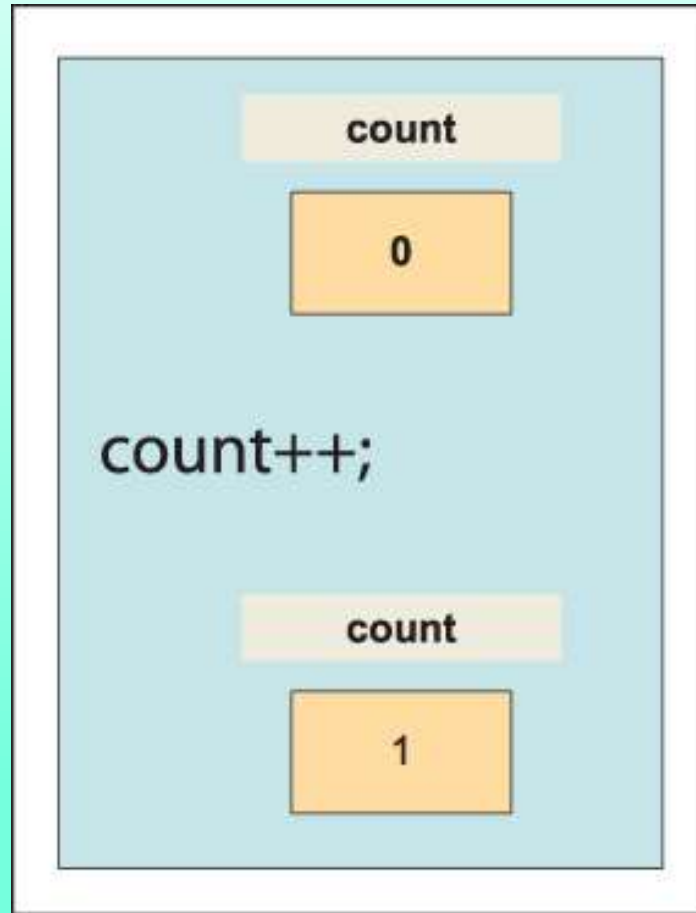
## (continued)



**Figure 2-10** Declaration of value type variables

# Basic Arithmetic Operations

## (continued)



**Figure 2-11** Change in memory after `count++;` statement executed

# Basic Arithmetic Operations

## (continued)

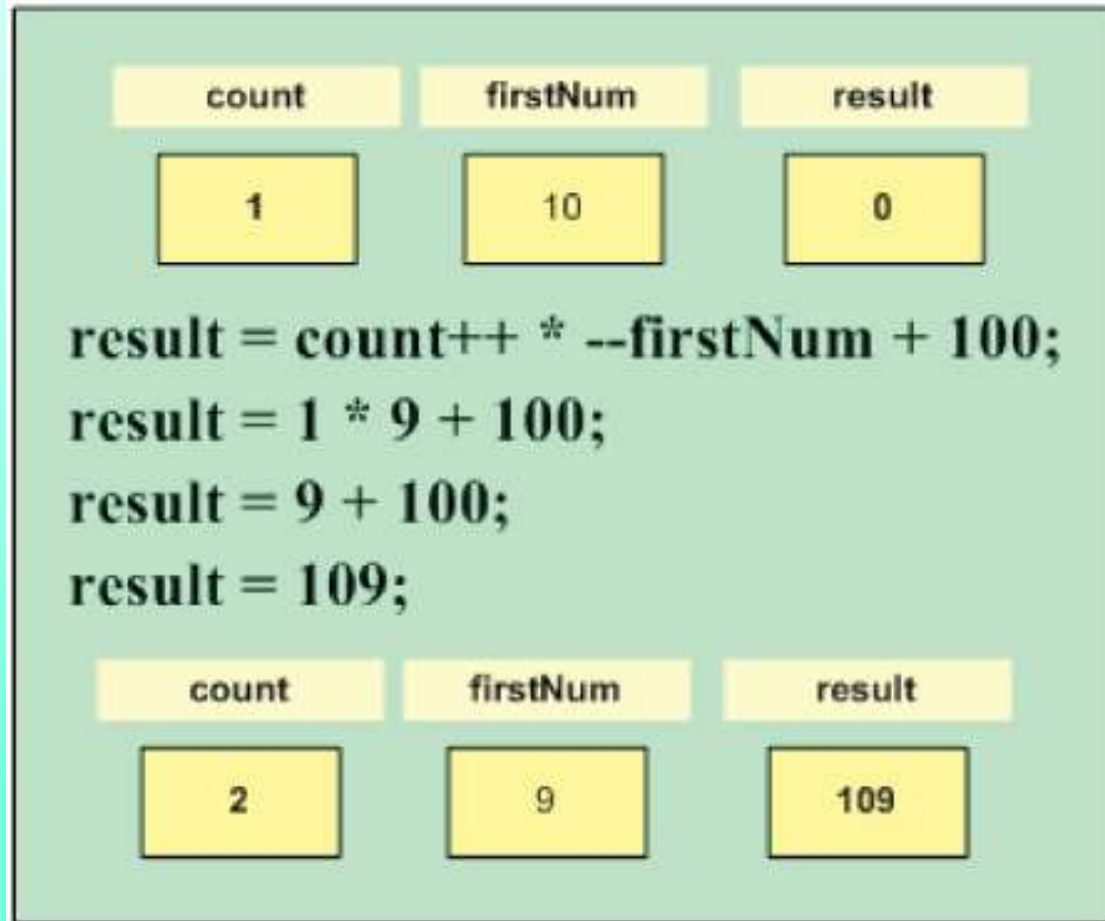
```
int num = 100;
```

```
Console.WriteLine(x++ + " " + ++x);
```

```
// Displays 100 102
```

# Basic Arithmetic Operations

## (continued)



**Figure 2-12** Results after statement is executed

# Assignment Operators

- Often we perform an operation on a variable, and then store the result back into that variable
- C# provides *assignment operators* to simplify that process
- For example, the statement

```
num += count;
```

is equivalent to

```
num = num + count;
```

# Assignment Operators

- There are many assignment operators in C#, including the following

| <u>Operator</u> | <u>Example</u> | <u>Equivalent To</u> |
|-----------------|----------------|----------------------|
| <b>+=</b>       | <b>x += y</b>  | <b>x = x + y</b>     |
| <b>-=</b>       | <b>x -= y</b>  | <b>x = x - y</b>     |
| <b>*=</b>       | <b>x *= y</b>  | <b>x = x * y</b>     |
| <b>/=</b>       | <b>x /= y</b>  | <b>x = x / y</b>     |
| <b>%=</b>       | <b>x %= y</b>  | <b>x = x % y</b>     |

# Assignment Operators

- The right hand side of an assignment operator can be a complex expression
- The entire right-hand expression is evaluated first, then the result is combined with the original variable
- **Therefore**

```
result /= (total-MIN) % num;
```

**is equivalent to**

```
result = result / ((total-MIN) % num);
```

**likewise:**

```
total*=count1+count2;
```

**is equivalent to**

```
total= total *(count1+count2);
```



# Assignment Operators

The behavior of some assignment operators depends on the types of the operands

- If the operands to the += operator are strings, the assignment operator performs string concatenation
- The behavior of an assignment operator (+=) is always consistent with the behavior of the corresponding operator (+)

# Basic Arithmetic Operations

## (continued)

- Order of operations
  - Order in which the calculations are performed
- Example
  - `answer = 100;`
  - `answer += 50 * 3 / 25 - 4;`
    - $50 * 3 = 150$
    - $150 / 25 = 6$
    - $6 - 4 = 2$
    - $100 + 2 = 102$

# Order of Operations

| C# category    | Operators |    |    |    |    |    | Associativity |
|----------------|-----------|----|----|----|----|----|---------------|
| Unary          | +         | -  | ++ | -- |    |    | Right         |
| Multiplicative | *         | /  | %  |    |    |    | Left          |
| Additive       | +         | -  |    |    |    |    | Left          |
| Assignment     | =         | *= | /= | %= | += | -= | Right         |

**Table 2-14** Operator precedence

- Associativity of operators
  - Left
  - Right

# Order of Operations (continued)

answer  $\ast =$  400 + 10 / 2 - (25 + 2  $\ast$  4)  $\ast$  3;  
7 5 3 6 2 1 4

(1.)  $2 \ast 4 = 8$

(2.)  $25 + 8 = 33$

(3.)  $10 / 2 = 5$

(4.)  $33 \ast 3 = 99$

(5.)  $400 + 5 = 405$

(6.)  $405 - 99 = 306$

(7.)  $10 \ast 306 = 3060$

Order  
of  
operations

**Figure 2-13** Order of execution of the operators

# Operator Precedence

- What is the order of evaluation in the following expressions?

$$a + b + c + d + e$$

1 2 3 4

$$a + b * c - d / e$$

3 1 4 2

$$a / (b + c) - d \% e$$

2 1 4 3

$$a / (b * (c + (d - e)))$$

4 3 2 1

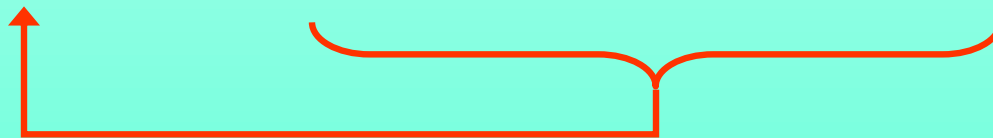
## 2.4 – Assignment Revisited

- The assignment operator has a lower precedence than the arithmetic operators

First the expression on the right hand side of the = operator is evaluated

```
answer = sum / 4 + MAX * lowest;
```

          4       1    3       2



Then the result is stored in the variable on the left hand side

# Data Conversion

- Sometimes it is convenient to convert data from one type to another
- For example, in a particular situation we may want to treat an integer as a floating point value
- These conversions do not change the type of a variable or the value that's stored in it – they only convert a value as part of a computation

# Conversion techniques

- In C#, data conversions can occur in three ways
  - assignment conversion
  - promotion
  - casting



# Assignment Conversion

- *Assignment conversion* occurs when a value of one type is assigned to a variable of another
- If `money` is a `float` variable and `dollars` is an `int` variable, the following assignment converts the value in `dollars` to a `float`

```
money = dollars;
```

- If we attempt to assign `money` to `dollars`, the compiler will issue an error message alerting us (narrowing conversion could lose info.)
- Only widening conversions can happen via assignment
- Note that the value or type of `dollars` did not change

# Data Conversion

- *Promotion* happens automatically when operators in expressions convert their operands
- For example, if `sum` is a `float` and `count` is an `int`, the value of `count` is converted to a floating point value to perform the following calculation

```
result = sum / count;
```

e.g. when number is concatenated with a string, first the number converted (promoted) to string, then the two strings are concatenated.

# Casting

- *Casting* is the most powerful, and dangerous, technique for conversion
- Both widening and narrowing conversions can be accomplished by explicitly casting a value
- To cast, the type is put in parentheses in front of the value being converted
- For example: If `money` is a `float` variable and `dollars` is an `int` variable and we want to convert `money` to an integer value and then store it in the `dollars` variable:

```
dollars=(int) money;
```

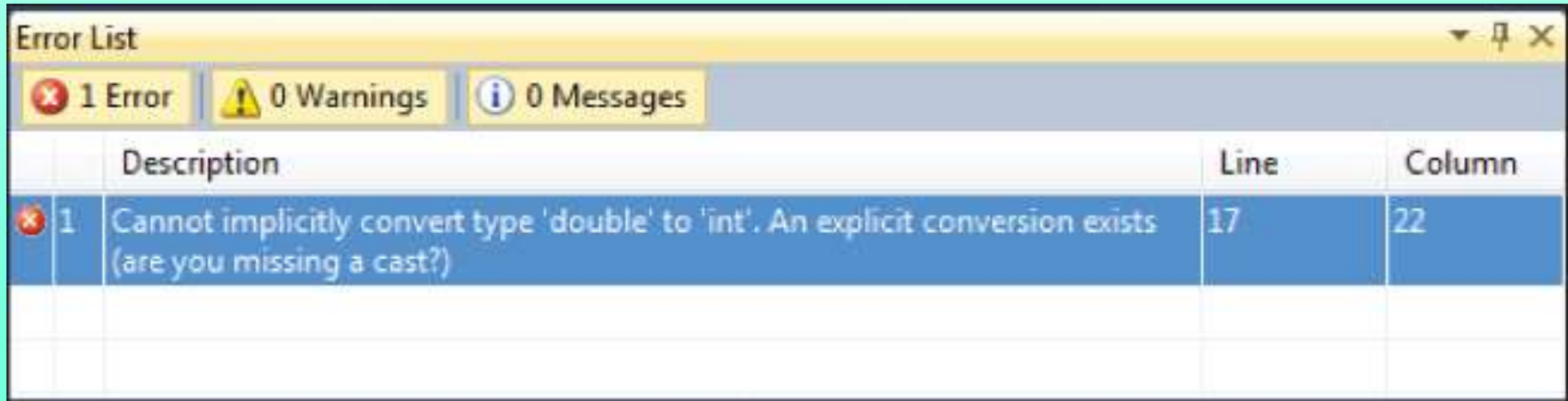
- For example, if `total` and `count` are integers, but we want a floating point result when dividing them, we can cast `total`

```
result = (float) total / count;
```

# Mixed Expressions

```
int value1 = 440;
double value2 = 100.60;
```

```
value1 = (int) value2; // syntax error as shown in Figure
2-14
```



**Figure 2-14** Syntax error generated for assigning a double to an int

# Mixed Expressions (continued)

- **Explicit** type coercion
  - Cast
  - (type) expression
  - examAverage = (exam1 + exam2 + exam3) / (**double**) count;

```
int value1 = 0,
 anotherNumber = 75;
double value2 = 100.99,
 anotherDouble = 100;
value1 = (int) value2; // value1 = 100
value2 = (double) anotherNumber; // value2 = 75.0
```

# Formatting Output

- You can format data by adding dollar signs, percent symbols, and/or commas to separate digits
- You can suppress leading zeros
- You can pad a value with special characters
  - Place characters to the left or right of the significant digits
- Use format specifiers

# Numeric Format Specifiers

| Character | Name                  | Description                                                                                                                                                                                                                    | Example                                                                  |
|-----------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| C or c    | Currency              | Number converted to a string that represents a currency amount with dollar symbols, decimal, and commas. If present, the precision modifier indicates the number of decimal places.                                            | (*{0:C3}*, 4238.8) – produces \$4,238.800                                |
| D or d    | Decimal               | Number converted to a string of integer digits (0–9), prefixed by a minus sign if the number is negative. If required, the number is padded with zeros to its left to produce the number of digits requested in the specifier. | (*{0:D5}*, 42) – produces 00042                                          |
| E or e    | Scientific (exponent) | Number is converted to a string of the form *-d.ddd...E+ddd* or *-d.ddd...e+ddd*, where each 'd' indicates a digit (0–9). The string starts with a minus sign if the number is negative.                                       | (*{0:E2}*, 4238.8) produces 4.24E+003                                    |
| F or f    | Fixed point           | Number is converted to a string of the form *-ddd.ddd...* where each d indicates a digit (0–9).                                                                                                                                | (*{0:F2}*, 4238.8) produces 4238.80                                      |
| G or g    | General               | Number is converted to the most compact of either fixed-point or scientific notation, depending on the type of the number and whether a precision specifier is present.                                                        | (*{0:G}*, 4238.8) produces 4238.8<br>(*{0:G2}*, 4238.8) produces 4.2E+03 |

**Table 2-16** Standard numeric format specifiers

# Numeric Format Specifiers (continued)

| Character | Name    | Description                                                                                                                                                                                                                          | Example                                                                                                  |
|-----------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| N or n    | Number  | Number is converted to a string of the form <code>"-d,ddd,ddd.ddd..."</code> . Thousand separators and minus symbol (if the value is negative) are inserted. The precision specifier indicates the desired number of decimal places. | <code>("{0:N2}", 4238.8)</code><br>produces 4,238.80                                                     |
| P or p    | Percent | Number is converted to a string that represents a percent. The converted number is multiplied by 100 to be presented as a percentage. The precision specifier indicates the desired number of decimal places.                        | <code>("{0:P}", 0.123)</code><br>produces 12.30 %<br><code>("{0:P1}", 0.12783)</code><br>produces 12.8 % |

**Table 2-16** Standard numeric format specifiers (continued)



# Formatting Output (continued)

| Character | Description | Examples                                          | Output      |
|-----------|-------------|---------------------------------------------------|-------------|
| C or c    | Currency    | <code>Console.Write("{0:c}", 26666.7888);</code>  | \$26,666.79 |
| C or c    | Currency    | <code>Console.Write("{0:c}", -2);</code>          | (\$2.00)    |
| C or c    | Currency    | <code>Console.Write("{0:c}", 38.8);</code>        | \$38.80     |
| F or f    | Fixed point | <code>Console.Write("{0:F4}", 50);</code>         | 50.0000     |
| F or f    | Fixed point | <code>Console.Write("{0:F0} {1}", 50, 22);</code> | 50 22       |

**Table 2-15** Examples using format specifiers

# Width Specifier

- Useful when you want to control the alignment of items on multiple lines
- Alignment component goes after the index ordinal followed by a comma (before the colon)
  - If alignment number is less than actual size, it is ignored
  - If alignment number is greater, pads with white space
    - Negative alignment component places spaces on right

```
Console.WriteLine("{0,10:F0}{1,8:C}", 9, 14);
```

```
9 $14.00 //Right justifies values
```

# Other Operators (2)

- Member access operator `.` is used to access object members
- Square brackets `[ ]` are used with arrays indexers and attributes
- they are the so-called **indexer**. Indexers are also used for accessing characters in a string.

```
string str = "Hello";
Console.WriteLine(str[1]);
```

# Other Operators (3)

- Conditional operator `?:` has the form **`b ? x : y`**(if `b` is true then the result is `x` else the result is `y`)

Example shows the usage of conditional operator to find the larger of two numbers:

```
int a = 6;
int b = 4;
Console.WriteLine(a > b ? "a>b" : "b<=a"); // a>b
```



# 4

Programming Fundamentals  
605116

ISRA University  
Faculty of IT

Textbook  
BARBARA DOYLE “C# Programming: From Problem Analysis to Program Design 4<sup>th</sup>  
Edition



# Console Input and Output

# Table of Contents

- Printing to the Console
  - Printing Strings and Numbers
- Reading from the Console
  - Reading Characters
  - Reading Strings
  - Parsing Strings to Numeral Types
  - Reading Numeral Types
- Various Examples





# Printing to the Console

- Console is used to display information in a text window
- Can display different values:
  - Strings
  - Numeral types
  - All primitive data types
- To print to the console use the class `Console` (`System.Console`)





# The Console Class

- Provides methods for console input and output
  - Input
    - `Read(...)` – reads a single character
    - `ReadKey(...)` – reads a combination of keys
    - `ReadLine(...)` – reads a single line of characters
  - Output
    - `Write(...)` – prints the specified argument on the console
    - `WriteLine(...)` – prints specified data to the console and moves to the next line

# Console.Write(...)

## ◆ Printing an integer variable

```
int a = 15;
...
Console.Write(a); // 15
```

- Printing more than one variable using a formatting string

```
double a = 15.5;
int b = 14;
...
Console.Write("{0} + {1} = {2}", a, b, a + b);
// 15.5 + 14 = 29.5
```

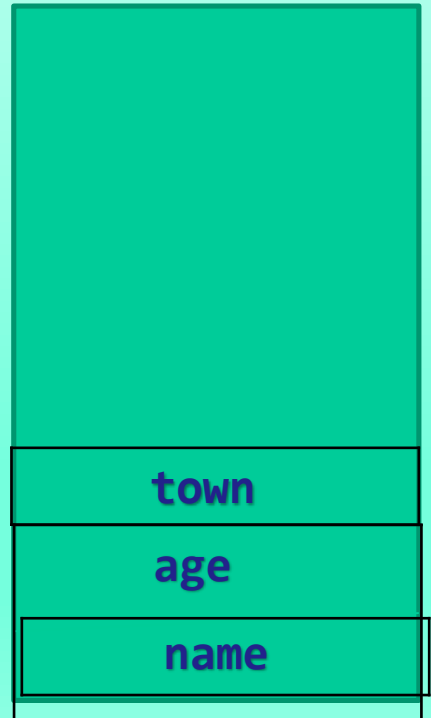
## ◆ Next print operation will start from the same line

# Printing to the Console – Example

```
static void Main()
{
 string name = "Peter";
 int age = 18;
 string town = "Sofia";

 Console.Write("{0} is {1} years old from {2}.",
 name, age, town);
 // Result: Peter is 18 years old from Sofia.
 Console.Write("This is on the same line!");
 Console.WriteLine("Next sentence will be" +
 " on a new line.");

 Console.WriteLine("Bye, bye, {0} from {1}.",
 name, town);
}
```



# Using Parameters – Example

```
static void Main()
{
 int a=2, b=3;
 Console.Write("{0} + {1} =", a, b);
 Console.WriteLine(" {0}", a+b);
 // 2 + 3 = 5

 Console.WriteLine("{0} * {1} = {2}", a, b, a*b);
 // 2 * 3 = 6

 float pi = 3.14159206f;
 Console.WriteLine("{0:F2}", pi); // 3,14

 Console.WriteLine("Bye - Bye!");
}
```

# Reading from the Console

- We use the console to read information from the command line
- We can read:
  - Characters
  - Strings
  - Numeral types (after conversion)
- To read from the console we use the methods `Console.Read()` and `Console.ReadLine()`



# Console.Read()

- Gets a single character from the console (after [Enter] is pressed)
  - Returns a result of type `int`
  - Returns `-1` if there aren't more symbols
- To get the actually read character we need to cast it to `char`

```
int i = Console.Read();
char ch = (char) i; // Cast the int to char

// Gets the code of the entered symbol
Console.WriteLine("The code of '{0}' is {1}.", ch, i);
```

# Console.ReadKey()

- Waits until a combination of keys is pressed
  - Reads a single character from console or a combination of keys
- Returns a result of type **ConsoleKeyInfo**
  - **KeyChar** – holds the entered character
  - **Modifiers** – holds the state of [Ctrl], [Alt], ...

```
ConsoleKeyInfo key = Console.ReadKey();
Console.WriteLine();
Console.WriteLine("Character entered: " + key.KeyChar);
Console.WriteLine("Special keys: " + key.Modifiers);
```

# Console.ReadLine()

- Gets a line of characters
- Returns a `string` value
- Returns `null` if the end of the input is reached

```
Console.Write("Please enter your first name: ");
string firstName = Console.ReadLine();
```

```
Console.Write("Please enter your last name: ");
string lastName = Console.ReadLine();
```

```
Console.WriteLine("Hello, {0} {1}!",
 firstName, lastName);
```



# Reading Numeral Types

- Numeral types can not be read directly from the console
- To read a numeral type do the following:
  1. Read a string value
  2. Convert (parse) it to the required numeral type
- `int.Parse(string)` – parses a `string` to `int`

```
string str = Console.ReadLine()
int number = int.Parse(str);

Console.WriteLine("You entered: {0}", number);
```

# Converting Strings to Numbers

- Numeral types have a method `Parse(...)` for extracting the numeral value from a string
  - `int.Parse(string)` – `string` → `int`
  - `long.Parse(string)` – `string` → `long`
  - `float.Parse(string)` – `string` → `float`
  - Causes `FormatException` in case of error

```
string s = "123";
int i = int.Parse(s); // i = 123
long l = long.Parse(s); // l = 123L
```

```
string invalid = "xxx1845";
int value = int.Parse(invalid); // FormatException
```

# Reading Numbers from the Console – Example

```
static void Main()
{
 int a = int.Parse(Console.ReadLine());
 int b = int.Parse(Console.ReadLine());

 Console.WriteLine("{0} + {1} = {2}", a, b, a+b);
 Console.WriteLine("{0} * {1} = {2}", a, b, a*b);

 float f = float.Parse(Console.ReadLine());
 Console.WriteLine("{0} * {1} / {2} = {3}", a, b, f, a*b/f);
}
```

# Converting Strings to Numbers (2)

- Converting can also be done using the methods of the `Convert` class
  - `Convert.ToInt32(string)` – string → int
  - `Convert.ToSingle(string)` – string → float
  - `Convert.ToInt64(string)` – string → long
  - Internally uses the parse methods of the numeral types

```
string s = "123";
int i = Convert.ToInt32(s); // i = 123
long l = Convert.ToInt64(s); // l = 123L

string invalid = "xxx1845";
int value = Convert.ToInt32(invalid); // FormatException
```

# Printing a Letter – Example

```
Console.Write("Enter person name: ");
string person = Console.ReadLine();
```

```
Console.Write("Enter company name: ");
string company = Console.ReadLine();
```

```
Console.WriteLine(" Dear {0},", person);
Console.WriteLine("We are pleased to tell you that {0} has chosen you to take
part in the \"Introduction To Programming\" course. {0} wishes you good
luck!", company);
```

```
Console.WriteLine(" Yours,");
Console.WriteLine(" {0}", company);
```

# Calculating Area – Example

```
Console.WriteLine("This program calculates the area of a rectangle or a triangle");

Console.Write("Enter a and b (for rectangle) or a and h (for triangle): ");
int a = int.Parse(Console.ReadLine());
int b = int.Parse(Console.ReadLine());

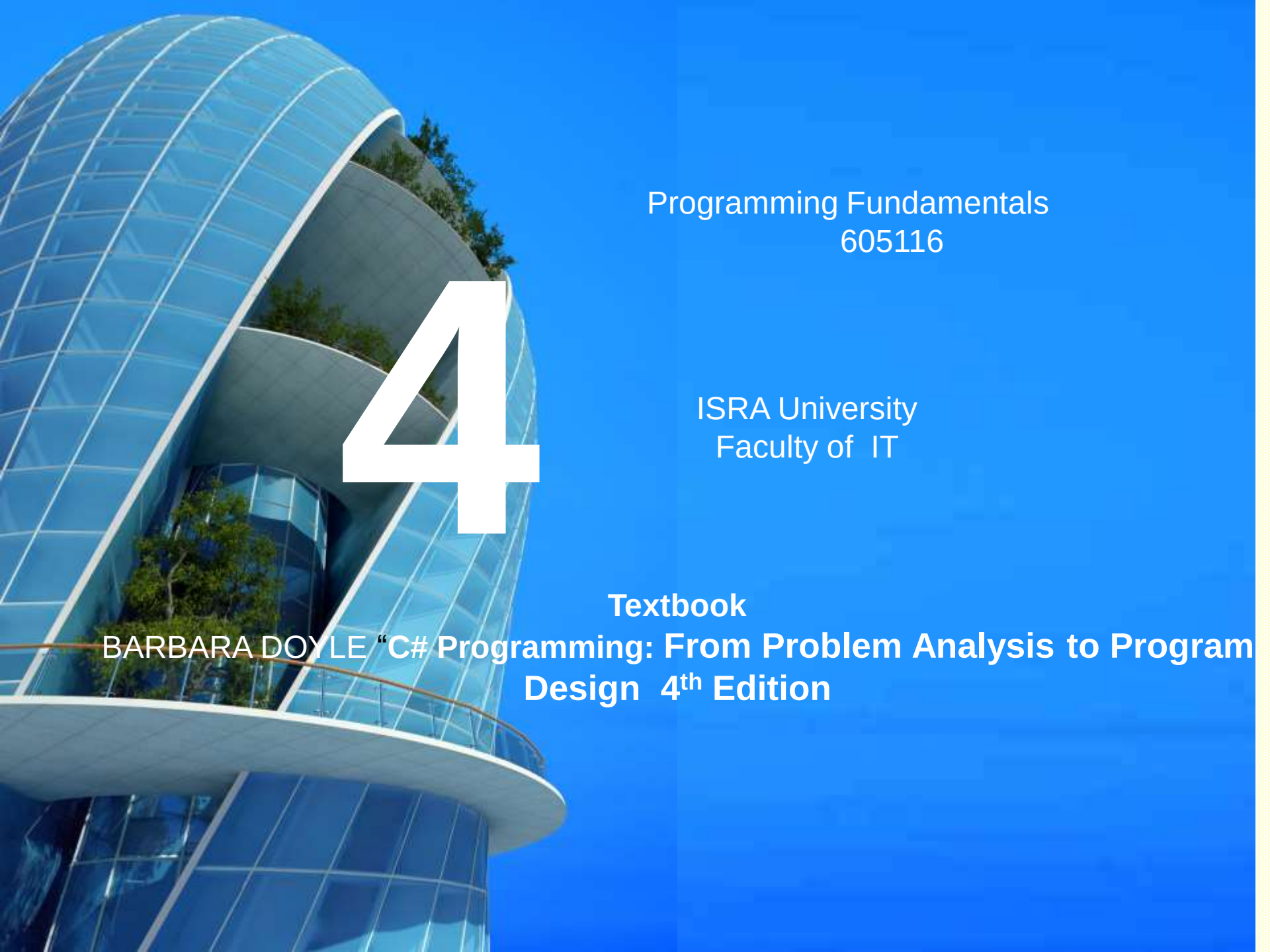
Console.Write("Enter 1 for a rectangle or 2 for a triangle: ");

int choice = int.Parse(Console.ReadLine());
double area = (double) (a*b) / choice;

Console.WriteLine("The area of your figure is {0}", area);
```

# Exercises

1. Write a program that reads 3 integer numbers from the console and prints their sum.
2. Write a program that reads the radius  $r$  of a circle and prints its perimeter and area.



Programming Fundamentals  
605116

4

ISRA University  
Faculty of IT

**Textbook**

BARBARA DOYLE **"C# Programming: From Problem Analysis to Program  
Design 4<sup>th</sup> Edition**





# **Making Decisions**

# Chapter Objectives

- Learn about conditional expressions that return Boolean results and those that use the bool data type
- Examine equality, relational, and logical operators used with conditional expressions
- Write if selection type statements to include one-way, two-way, and nested forms
- Learn about and write switch statements

# Chapter Objectives (continued)

- Learn how to use the ternary operator to write selection statements
- Revisit operator precedence and explore the order of operations
- Work through a programming example that illustrates the chapter's concepts

# A program consists of...

*[both PP programs & OOP programs]*

## 0) **Header** code for

- packages, classes
- methods & their arguments / parameters
- classes to be imported from external library  
(of commonly-used, pre-written  
methods / procedures / functions)

# 1) Declarations

- request memory locations for data storage
- give a name & data type  
to each of those locations
- possibly give an initial value to be stored

(i.e., declare variables & constants & objects)

## 2) **Action** statements to be executed:

### – operate on data

- Arithmetic (on numeric types)    +    –    \*    /    %
- Concatenate (string types)    +

### – move data around

- in memory                      (assignment)                      =
- memory to/from console/keyboard  
    (or a file, or window) (i.e., I/O)

### 3) **Control** statements

decide the order that actions execute

- next statement?
- skip over some?
- repeat some?
- transfer control to somewhere else?
  - possibly return control back to “here”?

NOTE:

control STARTS at 1<sup>st</sup> statement in `main` method

# Transfer control

1. **next** statement (in physical sequence) (*default*)
2. **skip** 1+ actions **if** some condition is FALSE
  - or choose which action(s) to do next from SET of several choices, based on the value of some variable
3. **loop** & do 1+ actions again (& again & again ...)
  - either a fixed number of times
  - or until some event happens  
(based on checking if some condition is True or False)
4. **call a method** & return “here” when it’s done
  - method itself does some actions  
& it may itself transfer control further
5. **go to** some statement & do NOT return “here” when done



# Control Structures

Traditionally:

1. sequence – the default
2. selection (decision, conditional execution)
3. repetition (looping)

Also:

4. call method (& go back)
5. *goto / break / return (& not go back)*

# Combine the control structures

Stack them

or

Nest them

# Selection options (in C#)

*[“selection”, “decision”, “conditional”]*

Single selection                      `if`

Double selection                    `if ... else`

Multiple selection

`switch`

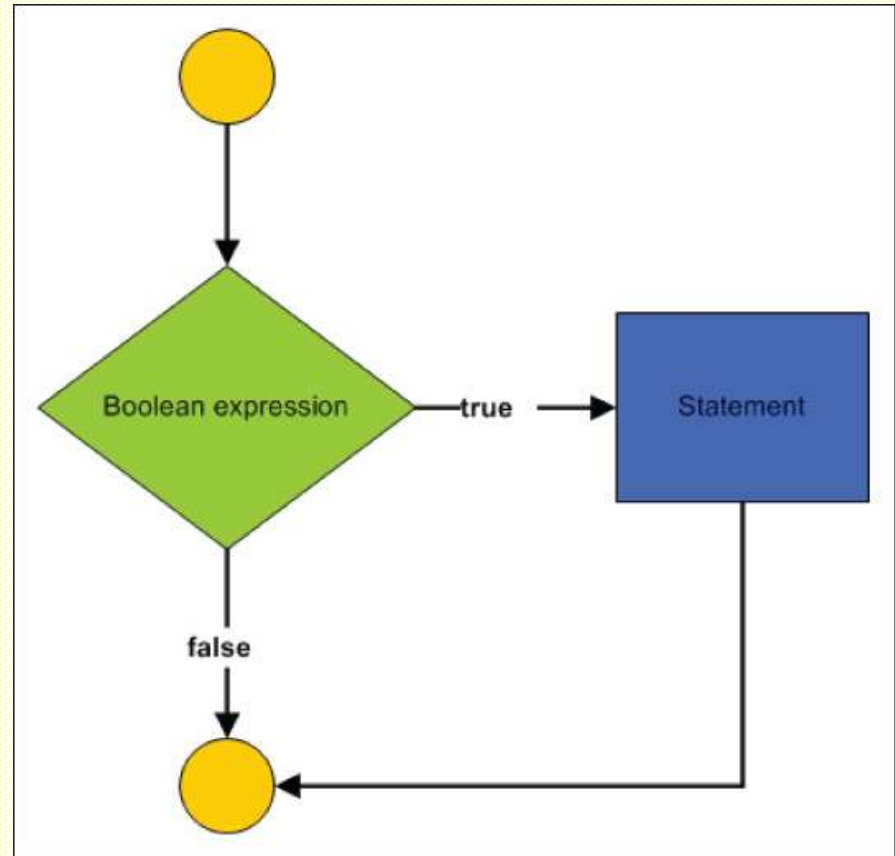
`or`

`if ... else if ... else if ... else ...`

# One-Way Selection Statement

```
if (expression)
{
 statement;
}
```

- No semicolon placed at end of expression
  - Null statement
- Curly braces required with multiple statements



**Figure 5-1**  
One-way if statement

# Plain if's (3 variations)

```
if (condition true)
 action;
```

-----

```
if (condition true)
{
 action;
}
```

```
| if (condition true)
| {
```

```
| action1;
```

```
| . . .
```

```
| actionN;
```

```
| }
```

```
|
```

```
|
```

```
|
```

```
|
```

# if ... else (3 variations)

```
if (condition true)
 action1;
```

```
else
 action2;
```

---

```
if (condition true)
{
 action1;
```

```
}
else
{ action2;
}
```

```
| if (condition true)
| {
```

```
| action1A;
| . . .
| action7A;
```

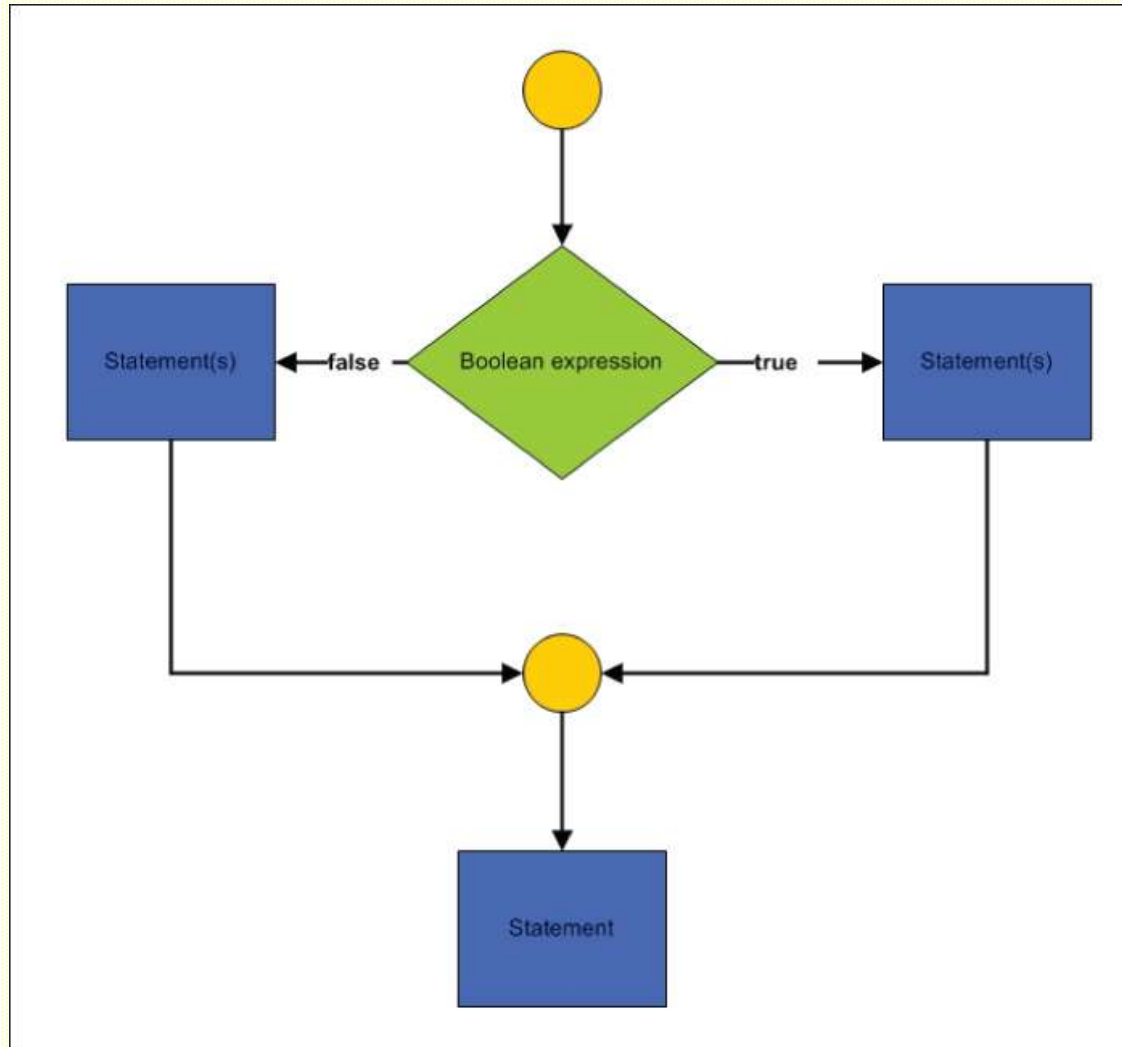
```
| }
| else
```

```
| {
| action1B;
| . . .
| action5B;
| }
```

# One-Way if Statement

```
if (examScore > 89)
{
 grade = 'A';
 Console.WriteLine("Congratulations - Great job!");
}
Console.WriteLine("I am displayed, whether the expression
evaluates true or false");
```

# Selection



DECISION:

determine which of  
2 paths to follow

(1+ statements  
in each path)

(ELSE path optional)



# if-else Statement – Example

- Checking a number if it is odd or even

```
Console.WriteLine("Please Enter the number");

int number = int.Parse(Console.ReadLine());
if (number % 2 == 0)
{
 Console.WriteLine("This number is even.");
}
else
{
 Console.WriteLine("This number is odd.");
}
```

# Each action could be any of:

- A simple action:
  - Assignment statement  
with arithmetic expression or method call
  - I/O from keyboard / file / window
  - Call to another method
- Another selection statement:  
`if or if...else or switch`
- A `while or do...while or for` loop
- or, do absolutely NOTHING ;

# The Condition

Comparison (equality & relational) operators:

==      !=      <      >      <=      >=

IMPORTANT:    ==    *compare for equality*  
                  =    *the assignment operator*

Compare 2 operands - operands can be:

variables, constants, arithmetic expressions,  
returned values from a method call, . . .

**But NOT Strings** (different methods to compare them)

Strings can be compared using == and !=

# A Condition is true or false

```
(ageOfStudent < MI_DRINKING_AGE)
```

```
(age != 21)
```

```
(michiganResident) // a boolean variable
```

```
((a + 2 * 3) <= ((b - 4) % 3))
```

```
((Math.PI * r * r) < maxSize)
```

*NOTE: need ( ) around whole condition*

# Relational Tests

| Expression                                                       | Result                      | Explanation                                                                                                                                                                                                                      |
|------------------------------------------------------------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>(money == 100.00)</code>                                   | Syntax error                | Type mismatch. money is <b>decimal</b> , 100.00 is <b>double</b> . Receive an error message that says "Operator '==' cannot be applied to <b>double</b> and <b>decimal</b> ".                                                    |
| <code>(money == 50.22m)</code><br><code>(money != dValue)</code> | <b>true</b><br>Syntax error | Must suffix the <b>decimal</b> number with M or m. Type mismatch. money is <b>decimal</b> , dValue is <b>double</b> . Receive an error message that says "Operator '==' cannot be applied to <b>double</b> and <b>decimal</b> ". |
| <code>(aValue &gt; bValue)</code>                                | <b>false</b>                | 100 is not greater than 1000.                                                                                                                                                                                                    |
| <code>(sValue &lt; "CS")</code>                                  | Syntax error                | < cannot be used with <b>string</b> operand.                                                                                                                                                                                     |
| <code>(aValue &gt; (bValue - 999))</code>                        | <b>true</b>                 | 100 is greater than 1.                                                                                                                                                                                                           |

```

int aValue = 100,
 bValue = 1000;
decimal money =
 50.22m;
double dValue =
 50.22;
string sValue =
 "CS158";

```

**Table 5-3** Results of sample conditional expressions

# Relational Tests

| Expression        | Result       | Explanation                                                                                        |
|-------------------|--------------|----------------------------------------------------------------------------------------------------|
| (aValue > dValue) | true         | Integer aValue is converted to <b>double</b> and compared correctly.                               |
| (aValue < money)  | false        | Integer aValue is converted to <b>decimal</b> and compared correctly.                              |
| (cValue = 'F')    | Syntax error | Cannot use single equal symbol (=) for comparison. Single equal symbol (=) is used for assignment. |
| (cValue < 'f')    | true         | Unicode A has a value of 65; Unicode f has a value of 102.                                         |

```
int aValue = 100;
decimal money =
 50.22m;
double dValue =
 50.22;
char cValue = 'A';
```

**Table 5-3** Results of sample conditional expressions

# Logical Operators

- If we want to check for more than one condition then we need to use **logical operators**
- These **combine** logical expressions (i.e. expressions that have a true/false value)
- There are three logical operators
  - &&            and
  - ||            or
  - !            Not

# Examples of Logic operators

```
if((x > 7) && (x < 20))
```

```
if((temp > 30.0) && (humidity > 0.9))
```

```
if((salary < minSalary) || (dependents > 5))
```

```
(examScore > 69 < 91) Invalid if ((examScore>69) && (examScore<91))
```

```
(69 < examScore < 91) Invalid
```

```
((examScore > 69) && (examScore < 91)) //Correct way
```

```
(letterGrade == 'A' || 'B') //Invalid
```

```
((letterGrade == 'A') || (letterGrade == 'B')) //Correct way
```



# Logic operators in conditions

**&&** (*and*)

**||** (*or*)

**!** (*not*)

`( ! (age == 25) )`      [same as `(age != 25)`]

`( (a < b) && (c < d) )`

`( (firstName == 'W') || (gender == 'F') )`

*[Note: use truth tables to determine results]*

# Logical Operators

| Expression1 | Expression2 | Expression1 && Expression2 |
|-------------|-------------|----------------------------|
| true        | true        | true                       |
| true        | false       | false                      |
| false       | true        | false                      |
| false       | false       | false                      |

**Table 5-4** is sometimes referred to as a truth table

(examScore > 69 < 91)    *//Invalid*

(69 < examScore < 91)    *//Invalid*

((examScore > 69) && (examScore < 91))  
*//Correct way*

# Logical Operators

| Expression1 | Expression2 | Expression1    Expression2 |
|-------------|-------------|----------------------------|
| true        | true        | true                       |
| true        | false       | true                       |
| false       | true        | true                       |
| false       | false       | false                      |

**Table 5-5** Conditional logical OR ( || )

(letterGrade == 'A' || 'B')    *//Invalid*

((letterGrade == 'A') || (letterGrade == 'B'))    *//Correct way*

# Logical Operators

| Expression | ! Expression |
|------------|--------------|
| true       | false        |
| false      | true         |

**Table 5-6** Logical NOT ( ! )

- NOT operator (!) returns the logical complement, or negation, of its operand
- Easier to debug a program that includes only positive expressions

# Translation from English?

*“All juniors **and** seniors should get bonus points”*

```
if (status == 3 && status == 4)
{
 . . .
}
```

WRONG

NOTE: status variable holds ONE value

```
if (status == 3 || status == 4) {
 . . .
}
```

RIGHT

# Boolean Data Types

- `bool` type holds the value of `true` or `false`
- When a `bool` variable is used in a conditional expression, you do not have to add symbols to compare the variable against a value
- Boolean flags used as flags to signal when a condition exists or when a condition changes

`if (moreData)`

# Order of precedence of operators

|                         |     |    |       |    |
|-------------------------|-----|----|-------|----|
| unary operators         | -   | +  | !     |    |
| arithmetic operators    | *   | /  | %     |    |
|                         | +   | -  |       |    |
| relational operators    | <   | >  | <=    | >= |
| equality operators      | ==  | != |       |    |
| logic operators         | & & |    | [and] |    |
| NOTE: “and” before “or” |     |    | [or]  |    |
| assignment operator     | =   |    |       |    |

*but ( ) can over-ride these*

# Order of precedence ?

*NOTE: need ( ) around whole condition*

|   |                 |                 |                 |   |              |
|---|-----------------|-----------------|-----------------|---|--------------|
| ( | ( a == b )      | &&              | ( c > -14 )     | ) | typical      |
|   | ( a == b )      | &&              | ( c > -14 )     |   | <b>WRONG</b> |
| ( | a == b          | &&              | c > -14         | ) | OK           |
|   | 2 <sup>nd</sup> | 3 <sup>rd</sup> | 1 <sup>st</sup> |   |              |

*“All juniors and seniors with a gpa of at least a 3.0”*

```
(gpa >= 3.0 && classYear == 3 || classYear == 4)
```

**WRONG**

```
((gpa >= 3.0 && classYear == 3) || (gpa >= 3.0 && classYear == 4))
```



# Short-Circuit Evaluation

- Short-circuiting logical operators
  - `&&` and `||`
- OR (`||`) expressions – if the first evaluates as true, no need to evaluate the second operand
- AND (`&&`) expressions – if the first evaluates as false, no need to evaluate second operand
- C# also includes the `&` and `|` operators
  - Logical, do not perform short-circuit evaluation

# Short-Circuit Evaluation

```
int examScore = 75;
int homeWkGrade = 100;
double amountOwed = 0;
char status = 'I';
((examScore > 90) && (homeWkGrade > 80))
((amountOwed == 0) || (status == 'A'))
```

No need to evaluate  
the second  
expression

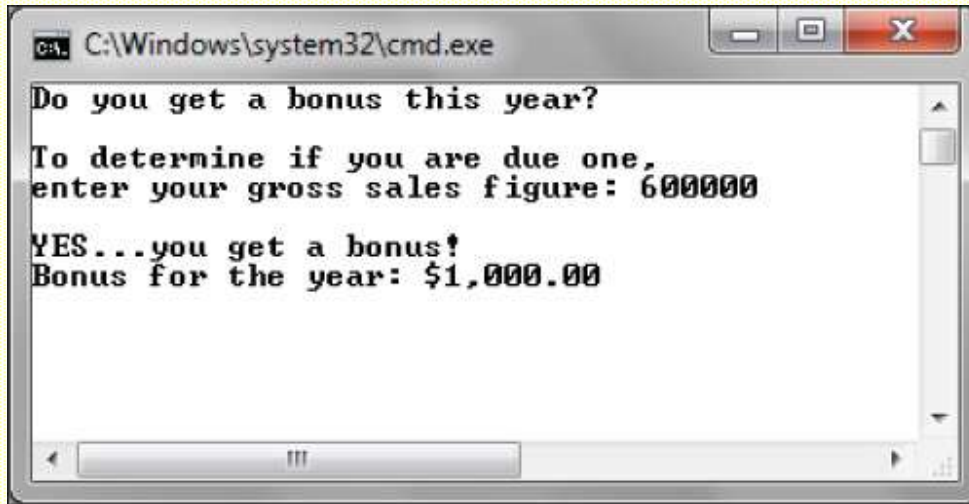
Again, no need to  
evaluate the second  
expression

# One-Way if Selection Statement Example

```
/* BonusCalculator.cs Author: Doyle */
using System;
namespace BonusApp
{
 class BonusCalculator
 {
 static void Main()
 {
 string inValue;
 decimal salesForYear, bonusAmount = 0M;
 Console.WriteLine("Do you get a bonus this year?");
 Console.WriteLine();
 Console.WriteLine("To determine if you are due one, ");
 }
 }
}
```

```
Console.Write("enter your gross sales figure: ");
inValue = Console.ReadLine();
salesForYear = Convert.ToDecimal(inValue);
if (salesForYear > 500000.00M)
{
 Console.WriteLine();
 Console.WriteLine("YES...you get a bonus!");
 bonusAmount = 1000.00M;
}
Console.WriteLine("Bonus for the year: {0:C}",
 bonusAmount);
Console.ReadLine();
} // end of Main() method
} // end of class BonusCalculator
} // end of BonusApp namespace
```

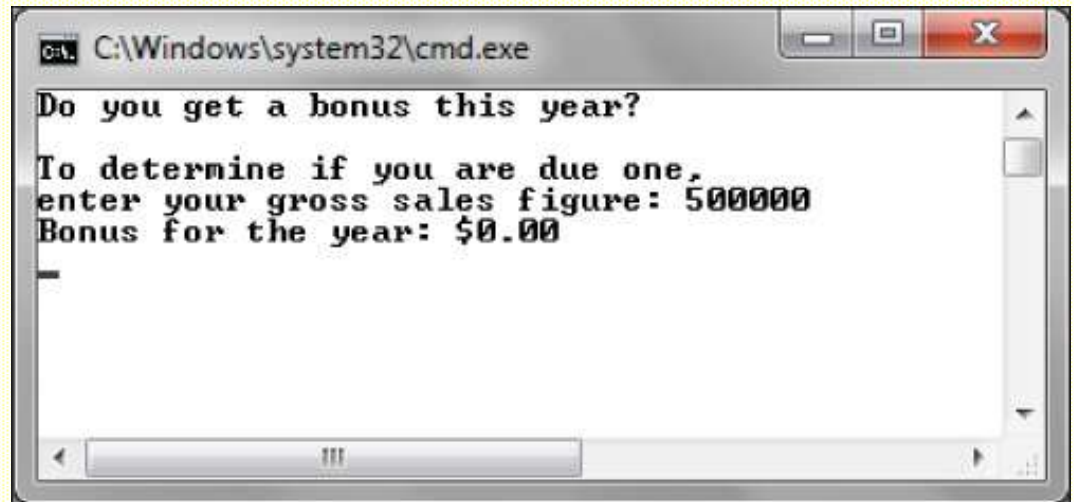
# Output from BonusCalculator



```
C:\Windows\system32\cmd.exe
Do you get a bonus this year?
To determine if you are due one,
enter your gross sales figure: 600000
YES...you get a bonus!
Bonus for the year: $1,000.00
```

**Figure 5-2** BonusApp  
with salesForYear  
equal  
to 600,000.00

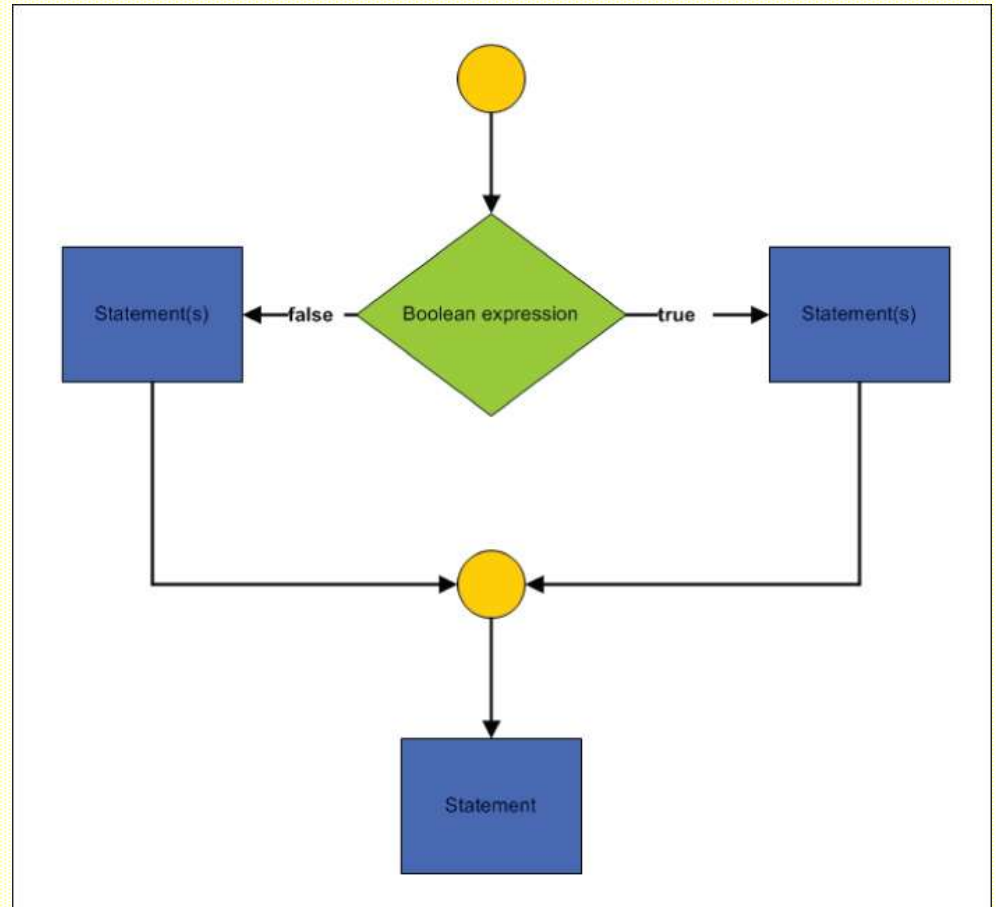
**Figure 5-3** BonusApp  
with salesForYear  
equal to 500,000.00



```
C:\Windows\system32\cmd.exe
Do you get a bonus this year?
To determine if you are due one,
enter your gross sales figure: 500000
Bonus for the year: $0.00
-
```

# Two-Way Selection Statement

- Either the true statement(s) executed or the false statement(s), but not both
- No need to repeat expression test in **else** portion



**Figure 5-5** Two-way if statement

# Two-Way Selection Statement

## (continued)

if (expression)

{

statement;

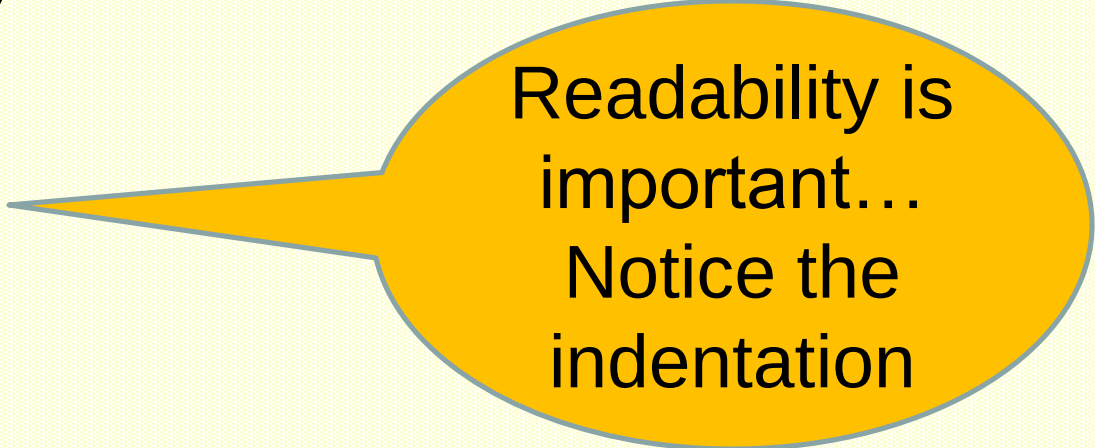
}

else

{

statement;

}



Readability is  
important...  
Notice the  
indentation

# Two-Way if...else Selection Statement Example

```
if (hoursWorked > 40)
{
 payAmount = (hoursWorked - 40) * payRate * 1.5 + payRate * 40;
 Console.WriteLine("You worked {0} hours overtime.",
 hoursWorked - 40);
}
else
 payAmount = hoursWorked * payRate;

Console.WriteLine("Displayed, whether the expression evaluates" +
 " true or false");
```



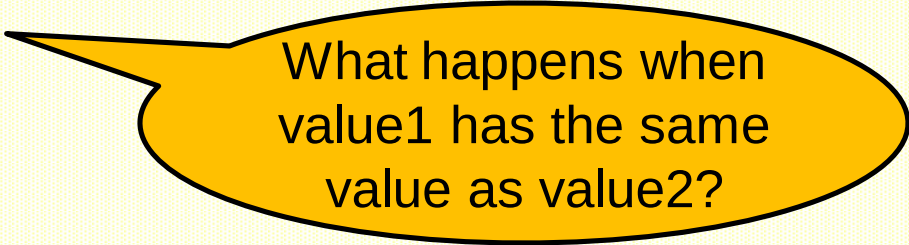
# Two-way if...else

- Try to avoid repeating code

```
if (value1 > value2)
{
 Console.WriteLine("The largest value entered was " + value1);
 Console.WriteLine("Its square root is {0:f2}", Math.Sqrt(value1));
}
else
{
 Console.WriteLine("The largest value entered was " + value2);
 Console.WriteLine("Its square root is {0:f2}", Math.Sqrt(value2));
}
```

# Alternative Solution

```
int largest;
if (value1 > value2)
{
 largest = value1;
}
else
{
 largest = value2;
}
```



What happens when  
value1 has the same  
value as value2?

```
Console.WriteLine("The largest value entered was " + largest);
Console.WriteLine("Its square root is {0:f2}", Math.Sqrt(largest));
```

# Example: Math Class Methods

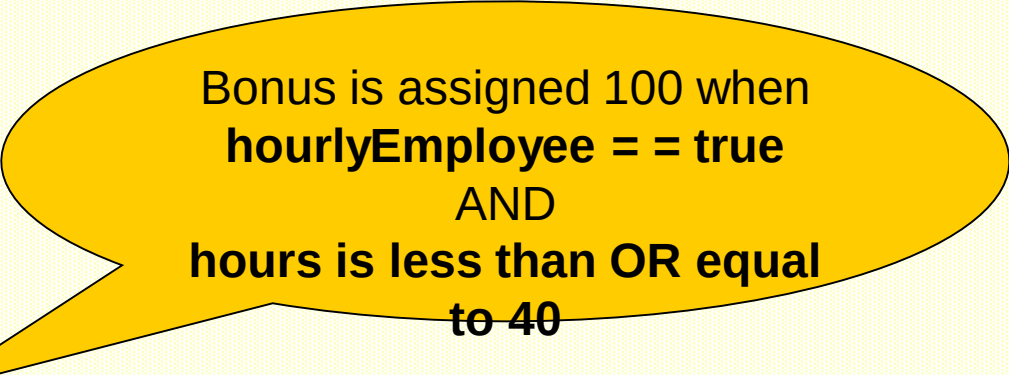
| Method                                   | Description                                                                                          | Example                                                                              |
|------------------------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Abs ( x )</b>                         | absolute value of x                                                                                  | <b>Abs ( 23.7 )</b> is 23.7<br><b>Abs ( 0 )</b> is 0<br><b>Abs ( -23.7 )</b> is 23.7 |
| <b>Ceiling( x )</b>                      | rounds x to the smallest integer not less than x                                                     | <b>Ceiling( 9.2 )</b> is 10.0<br><b>Ceiling( -9.8 )</b> is -9.0                      |
| <b>Floor( x )</b>                        | rounds x to the largest integer not greater than x                                                   | <b>Floor( 9.2 )</b> is 9.0<br><b>Floor( -9.8 )</b> is -10.0                          |
| <b>Max( x, y )</b>                       | larger value of x and y<br>(also has versions for <b>float</b> , <b>int</b> and <b>long</b> values)  | <b>Max( 2.3, 12.7 )</b> is 12.7<br><b>Max( -2.3, -12.7 )</b> is -2.3                 |
| <b>Min( x, y )</b>                       | smaller value of x and y<br>(also has versions for <b>float</b> , <b>int</b> and <b>long</b> values) | <b>Min( 2.3, 12.7 )</b> is 2.3<br><b>Min( -2.3, -12.7 )</b> is -12.7                 |
| <b>Pow( x, y )</b>                       | x raised to power y (xy)                                                                             | <b>Pow( 2.0, 7.0 )</b> is 128.0<br><b>Pow( 9.0, .5 )</b> is 3.0                      |
| <b>Sqrt( x )</b>                         | square root of x                                                                                     | <b>Sqrt( 900.0 )</b> is 30.0<br><b>Sqrt( 9.0 )</b> is 3.0                            |
| Commonly used <b>Math</b> class methods. |                                                                                                      |                                                                                      |

# Nested if...else Statement

- Acceptable to write an `if` within an `if`
- When block is completed, all remaining conditional expressions are skipped or bypassed
- Syntax for nested `if...else` follows that of two-way
  - Difference: With a nested `if...else`, the statement may be another if statement
- No restrictions on the depth of nesting
  - Limitation comes in the form of whether you and others can read and follow your code

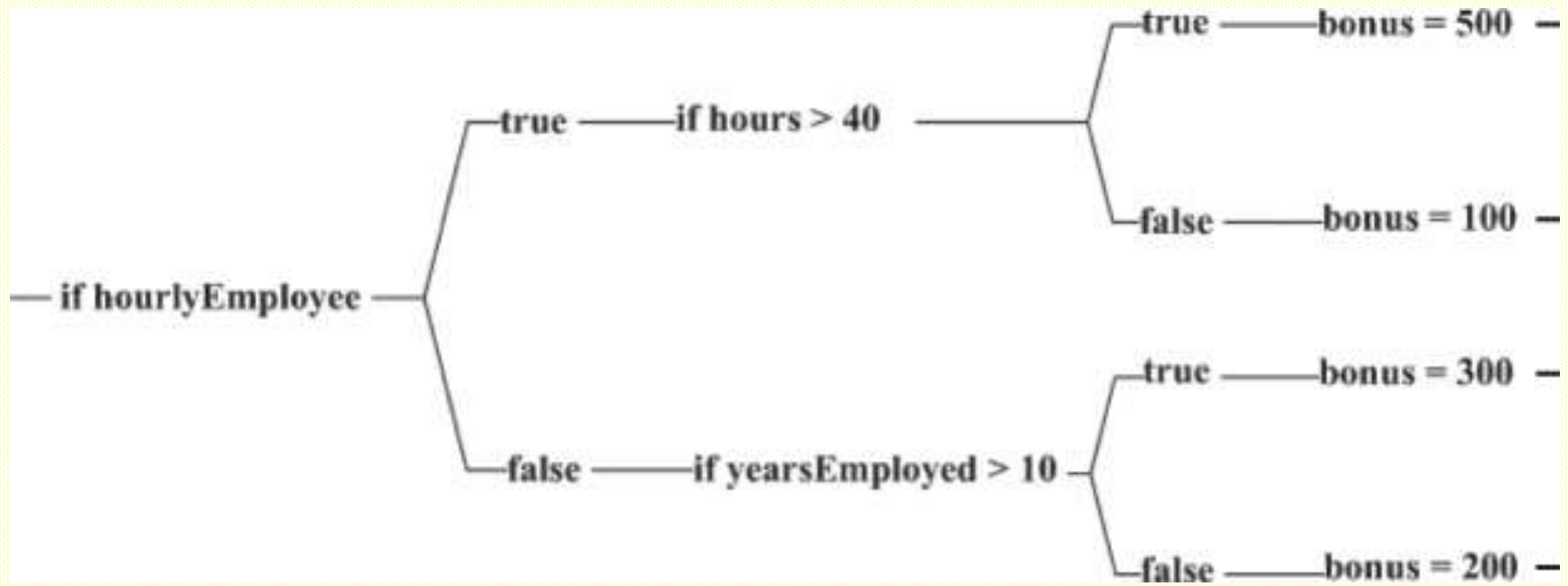
# Nested if...else Statement (continued)

```
bool hourlyEmployee;
double hours, bonus;
int yearsEmployed;
if (hourlyEmployee)
 if (hours > 40)
 bonus = 500;
 else
 bonus = 100;
else
 if (yearsEmployed > 10)
 bonus = 300;
 else bonus = 200;
```



Bonus is assigned 100 when  
**hourlyEmployee == true**  
AND  
**hours is less than OR equal**  
**to 40**

# Nested if...else Statement (continued)



**Figure 5-7** Bonus decision tree

# Matching up Else and If Clauses

```
if (aValue > 10) // Line 1
 if (bValue == 0) // Line 2
 amount = 5; // Line 3
 else // Line 4
 if (cValue > 100) // Line 5
 if (dValue > 100) // Line 6
 amount = 10; // Line 7
 else // Line 8
 amount = 15; // Line 9
 else // Line 10
 amount = 20; // Line 11
 else // Line 12
 if (eValue == 0) // Line 13
 amount = 25; // Line 14
```

else goes with the closest previous if that does not have its own else

# Matching up Else and If Clauses

- You can use braces to attach an **else** to an outer **if**

```
if (average > 59)
{
 if (average < 71)
 grade = 'D';
}
else
 grade = 'F';
```



# Nested if...else Statement

```
if (average > 89)
 grade = 'A';
else
 if (average > 79)
 grade = 'B';
 else
 if (average > 69)
 grade = 'C';
// More statements follow
```

Not necessary for second expression to be a compound expression using &&.

**You do not have to write**  
**if (average > 79 &&**  
**average <= 89)**

# Nested if...else Statement

```
if (average > 89)
 grade = 'A';
else if (average > 79)
 grade = 'B';
else if (average > 69)
 grade = 'C';
else if (average > 59)
 grade = 'D';
else
 grade = 'F';
```

- Could be written with a series of **if. . . else** statements
  - This prevents indentation problems

# Nested if...else Statement

```
if (weekDay == 1)
 Console.WriteLine("Monday");
else if (weekDay == 2)
 Console.WriteLine("Tuesday");
else if (weekDay == 3)
 Console.WriteLine("Wednesday");
else if (weekDay == 4)
 Console.WriteLine("Thursday");
else if (weekDay == 5)
 Console.WriteLine("Friday");
else
 Console.WriteLine("Not Monday through Friday");
```

- When you have a single variable being tested for equality against four or more values, a **switch** statement can be used

# Switch Selection Statements

- Multiple selection structure
- Also called case statement
- Works for tests of equality only
- Single variable or expression tested
  - Must evaluate to an integral or **string** value
- Requires the **break** for any case
  - No fall-through available

# Switch Statements General Form

`switch` (expression)

{

`case` value1: statement(s),

break,

...

`case` valueN: statement(s);

break;

[`default`: statement(s);

`break`;]

}

Selector

Value must be  
of the same  
type as selector

Optional

# Switch Statement

```
switch (weekDay)
{
 case 1: Console.WriteLine("Monday");
 break;
 case 2: Console.WriteLine("Tuesday");
 break;
 case 3: Console.WriteLine("Wednesday");
 break;
 : // Lines missing;
 default: Console.WriteLine("Not Monday through Friday");
 break;
}
```

# Switch Statement Example

```
/* StatePicker.cs Author: Doyle */
using System;
namespace StatePicker
{
 class StatePicker
 {
 static void Main()
 {
 string stateAbbrev;
 Console.WriteLine("Enter the state abbreviation. ");
 Console.WriteLine("Its full name will be displayed");
 Console.WriteLine();
 stateAbbrev = Console.ReadLine();
 }
 }
}
```

```
switch(stateAbbrev)
{
 case "AL": Console.WriteLine("Alabama");
 break;
 case "FL": Console.WriteLine("Florida");
 break;
 : // More states included
 case "TX": Console.WriteLine("Texas");
 break;
 default: Console.WriteLine("No match");
 break;
} // End switch
} // End Main()
} // End class
} // End namespace
```



# Switch Statements

- Associate same executable with more than one case
  - Example (creates a logical OR)

```
case "AL":
```

```
case "aL":
```

```
case "Al":
```

```
case "al": Console.WriteLine("Alabama");
 break;
```

- Cannot test for a range of values

# Switch Statement

```
switch (examScore / 10)
```

```
{
```

```
 case 1:
```

```
 case 2:
```

```
 case 3:
```

```
 case 4:
```

```
 case 5: Console.WriteLine("Failing Grade");
```

```
 break;
```

- **break** statement is required as soon as a case includes an executable statement

– No fall through

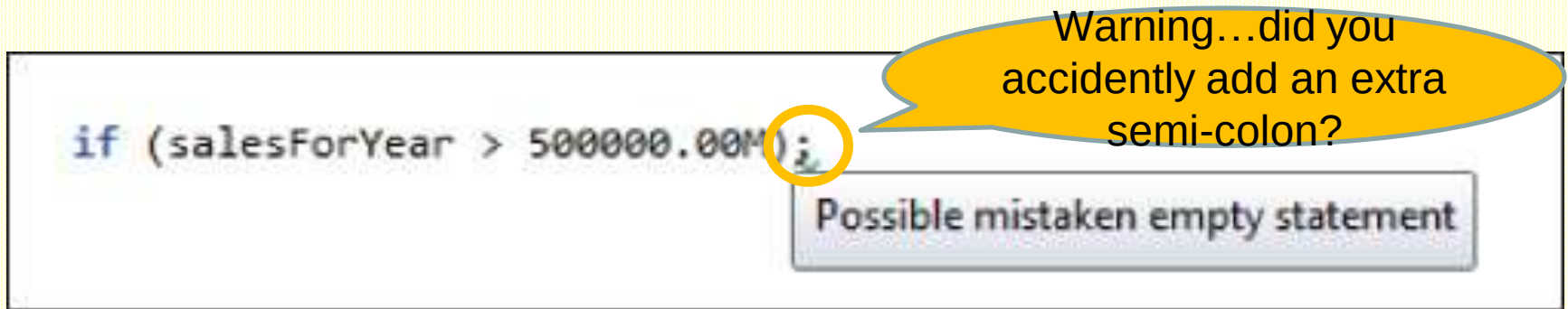
# Switch Statements (continued)

- Case value must be a constant literal
  - Cannot be a variable

```
int score,
 high = 90;
switch (score)
{
 case high : // Syntax error. Case value must be a
 constant
 // Can write "case 90:" but not "case high:"
```

- Value must be a compatible type
  - char value enclosed in single quote
  - string value enclosed in double quotes

# One-Way if Selection Statement



**Figure 5-4** Intellisense pop-up message

- One-way **if** statement does not provide a set of steps for situations where the expression evaluates to false

# Caution with ;

(it means “*Empty block of actions*”)

## WRONG

```
if (a < b) ;
```

```
 Console.WriteLine("a < b");
```

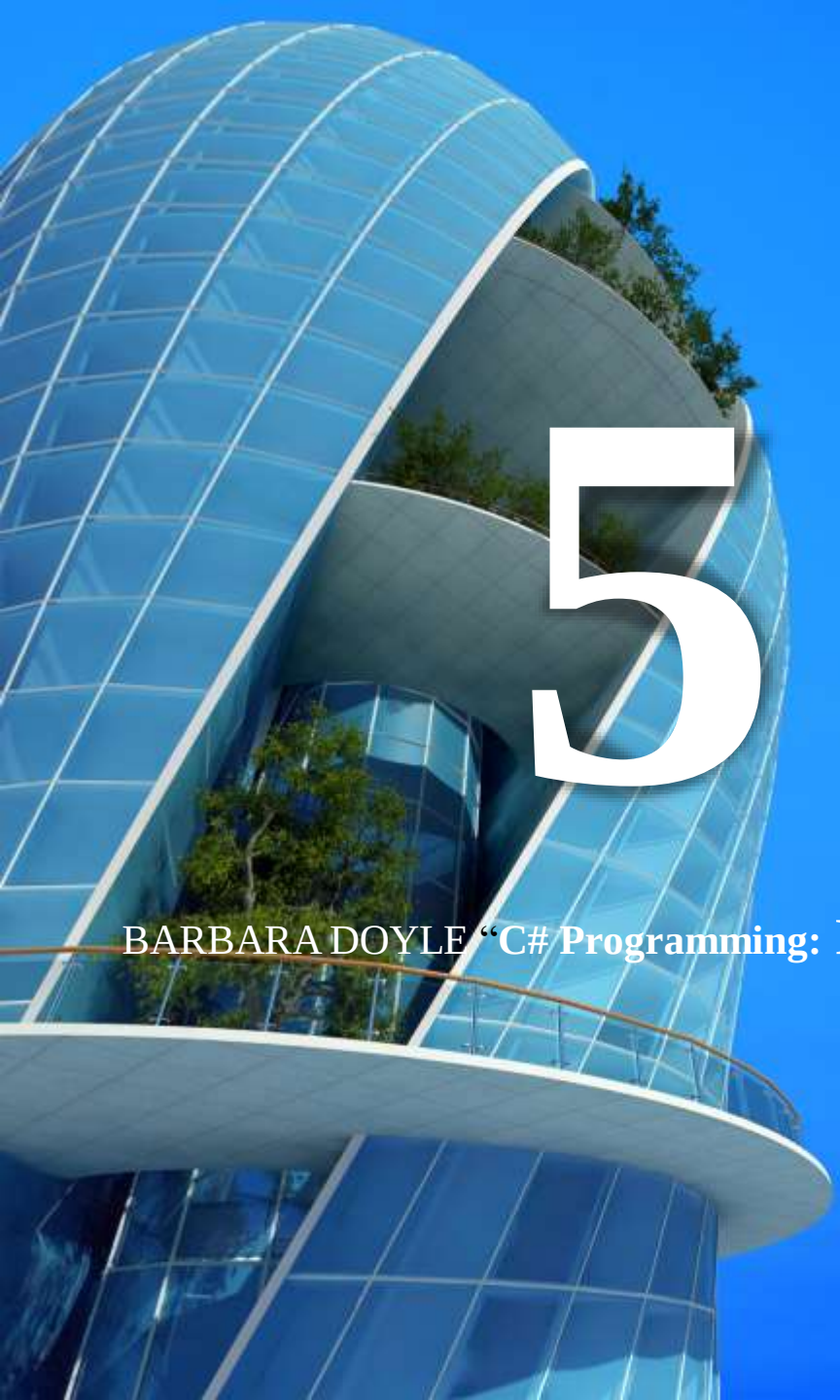
// Console.WriteLine will ALWAYS happen; not related to if

## RIGHT

```
if (a < b) // no ; here
```

```
 Console.WriteLine("a < b");
```

// Console.WriteLine MAY happen, depending on if condition



Programming Fundamentals  
605116

ISRA University  
Faculty of IT

Textbook

BARBARA DOYLE “C# Programming: From Problem Analysis to Program Design 4<sup>th</sup>  
Edition



# Loops



# Chapter Objectives

- Learn about conditional expressions that return Boolean results and those that use the bool data type
- Examine equality, relational, and logical operators used with conditional expressions
- Write if selection type statements to include one-way, two-way, and nested forms
- Learn about and write switch statements

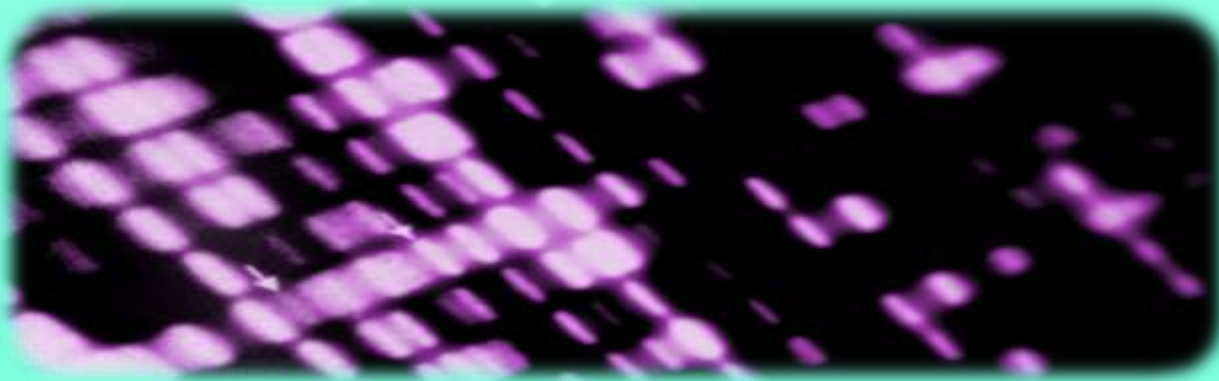


# What Is Loop?

- A loop is a control statement that allows repeating execution of a block of statements
  - May execute a code block fixed number of times
  - May execute a code block while given condition holds
  - May execute a code block for each member of a collection
- Loops that never end are called an infinite loops

# Using while(...) Loop

**Repeating a Statement While  
Given Condition Holds**



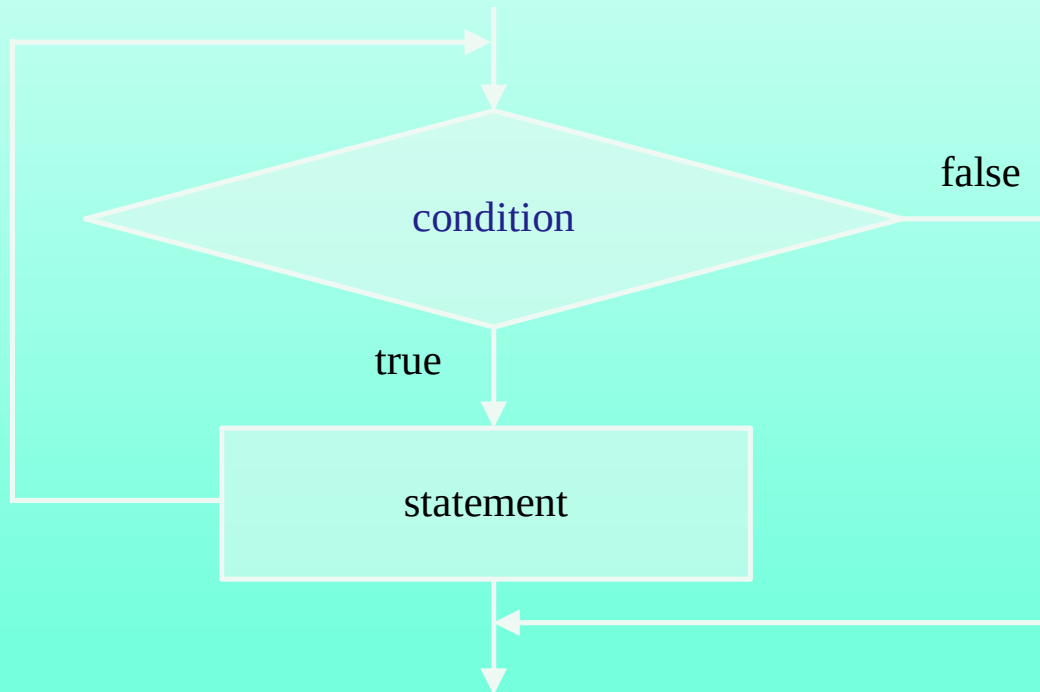
# How To Use While Loop?

- The simplest and most frequently used loop

```
while (condition)
{
 statements;
}
```

- The repeat condition
  - Returns a boolean result of **true** or **false**
  - Also called loop condition

# While Loop – How It Works?



# While Loop – Example

```
int counter = 0;
while (counter < 10)
{
 Console.WriteLine("Number : {0}", counter);
 counter++;
}
```

```
Number : 0
Number : 1
Number : 2
Number : 3
Number : 4
Number : 5
Number : 6
Number : 7
Number : 8
Number : 9
Press any key to continue_
```



`while(...)`

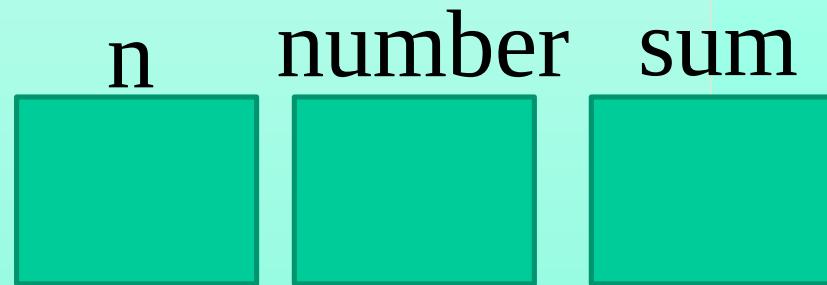
**Examples**

# Sum 1..N – Example

- Calculate and print the sum of the first N natural numbers

**The sum  $1 + 2 + 3 + 4 + 5 = 15$**

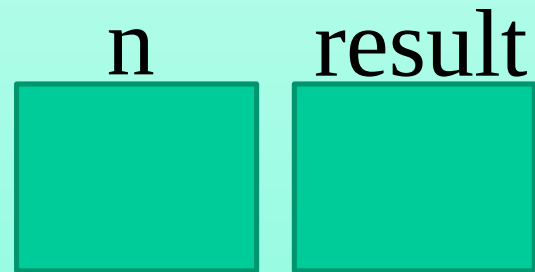
```
Console.Write("n = ");
int n = int.Parse(Console.ReadLine());
int number = 1;
int sum = 1;
Console.Write("The sum 1");
while (number < n)
{
 number++;
 sum += number ;
 Console.Write("+{0}", number);
}
Console.WriteLine(" = {0}", sum);
```



# Using **break** Operator

- **break** operator exits the inner-most loop

```
static void Main()
{
 int n = Convert.ToInt32(Console.ReadLine());
 // Calculate n! = 1 * 2 * ... * n
 int result = 1;
 while (true)
 {
 if(n == 1)
 break;
 result *= n;
 n--;
 }
 Console.WriteLine("n! = " + result);
}
```





```
do { ... }
while (...)
Loop
```



# Using Do-While Loop

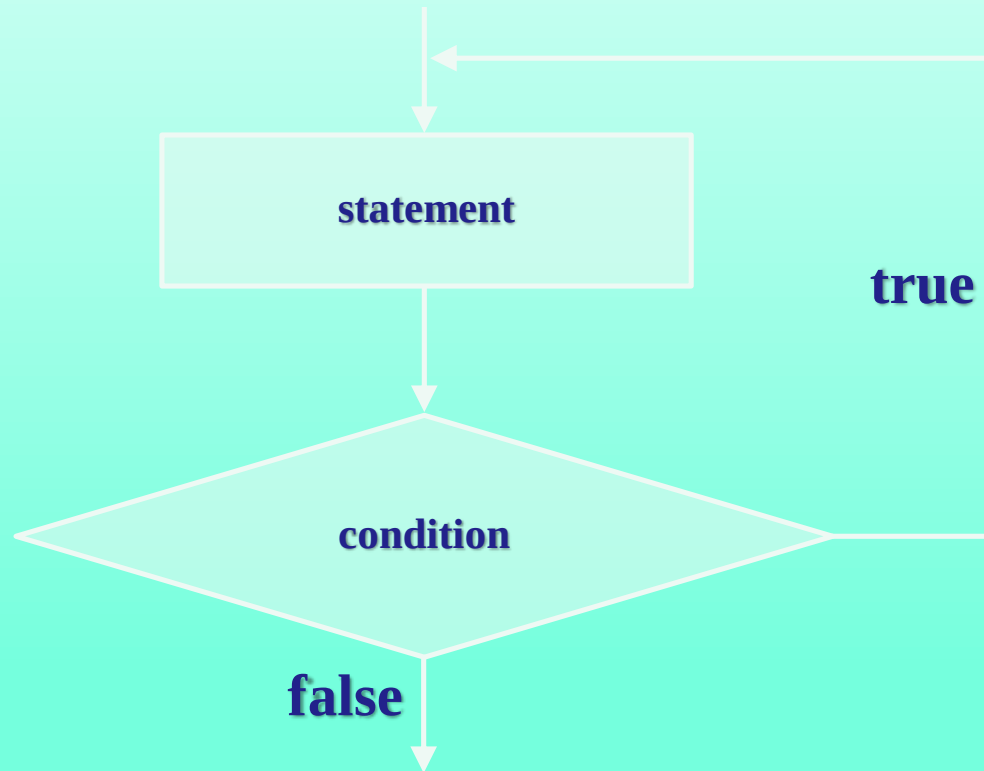


- Another loop structure is:

```
do
{
 statements;
}
while (condition);
```

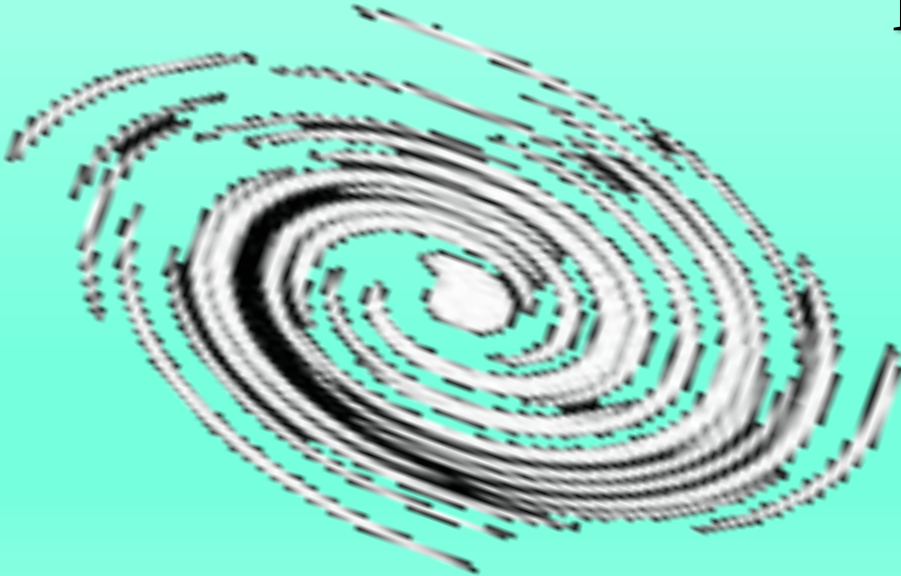
- The block of statements is repeated
  - While the boolean loop condition holds
- The loop is executed at least once

# Do-While Statement



```
do { ... }
while (...)
```

**Examples**



# Factorial – Example

- Calculating N factorial

```
static void Main()
{
 int n = Convert.ToInt32(Console.ReadLine());
 int factorial = 1;

 do
 {
 factorial *= n;
 n--;
 }
 while (n > 0);

 Console.WriteLine("n! = " + factorial);
}
```

# Product[N..M] – Example

- Calculating the product of all numbers in the interval [n..m]:

```
int n = int.Parse(Console.ReadLine());
int m = int.Parse(Console.ReadLine());
int number = n;
decimal product = 1;
do
{
 product *= number;
 number++;
}
while (number <= m);
Console.WriteLine("product[n..m] = " + product);
```

# Compare do while and while

```
int x = 5;
do {
 // D: do something ...

 x++;
} while (x <= 10);
```

```
int x = 5;
while (x <= 10)
{
 // D: do something ...

 x++;
}
```

**Any difference?**

# for Loops





# For Loops

- The typical for loop syntax is:

```
for (initialization; test; update)
{
 statements;
}
```

- Consists of
  - Initialization statement
  - Boolean test expression
  - Update statement
  - Loop body block

# The Initialization Expression

```
for (int number = 0; ...; ...)
{
 // Can use number here
}
// Cannot use number here
```

- Executed once, just before the loop is entered
  - Like it is out of the loop, before it
- Usually used to declare a counter variable

# The Test Expression

```
for (int number = 0; number < 10; ...)
{
 // Can use number here
}
// Cannot use number here
```

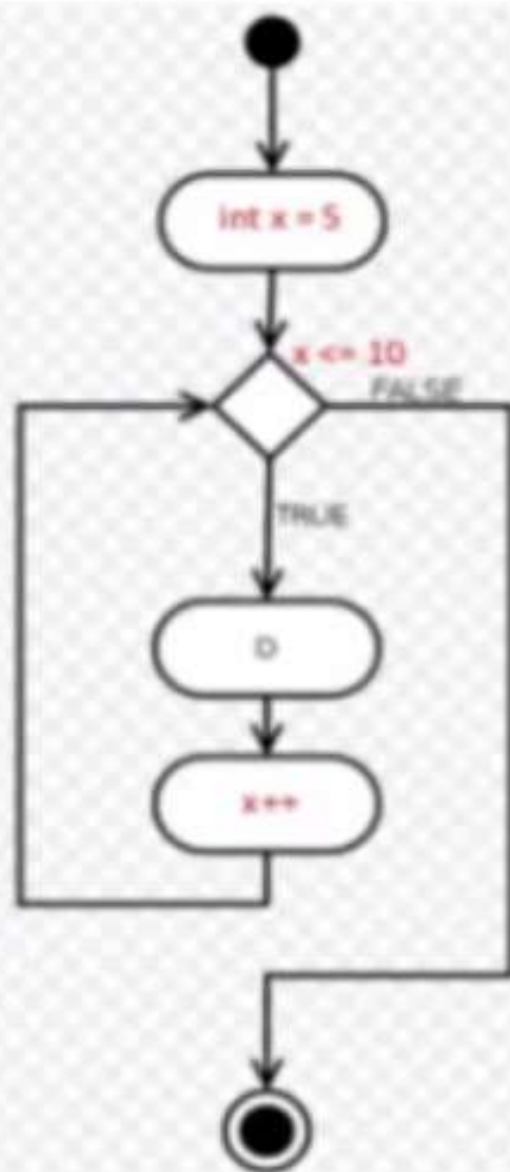
- Evaluated before each iteration of the loop
  - If **true**, the loop body is executed
  - If **false**, the loop body is skipped
- Used as a **loop condition**

# The Update Expression

```
for (int number = 0; number < 10; number++)
{
 // Can use number here
}
// Cannot use number here
```

- Executed at each iteration after the body of the loop is finished
- Usually used to update the counter

# for loop => while loop



```
for (int x = 5; x <= 10; x++)
{
 // D: do something ...
}
```



```
int x = 5;
while (x <= 10)
{
 // D: do something ...

 x++;
}
```

# for Loop

## **Examples**



- A simple for-loop to print the numbers 0...9:

## Simple for Loop – Example

```
for (int number = 0; number < 10; number++)
{
 Console.Write(number + " ");
}
```

- ◆ **A simple for-loop to calculate n!:**

```
int factorial = 1;
for (int i = 1; i <= n; i++)
{
 factorial *= i;
}
```

# Complex for Loop – Example

- ◆ **Complex for-loops could have several counter variables:**

```
for (int i=1, sum=1; i<=128; i=i*2, sum+=i)
{
 Console.WriteLine("i={0}, sum={1}", i, sum);
}
```

- ◆ **Result:**

```
i=1, sum=1
i=2, sum=3
i=4, sum=7
i=8, sum=15
...
```



# N^M – Example

- Calculating **n** to power **m** (denoted as  $n^m$ ):

```
static void Main()
{
 int n = int.Parse(Console.ReadLine());
 int m = int.Parse(Console.ReadLine());
 int result = 1;
 for (int i=0; i<m; i++)
 {
 result *= n;
 }
 Console.WriteLine("n^m = " + result);
}
```

result

n

m

i



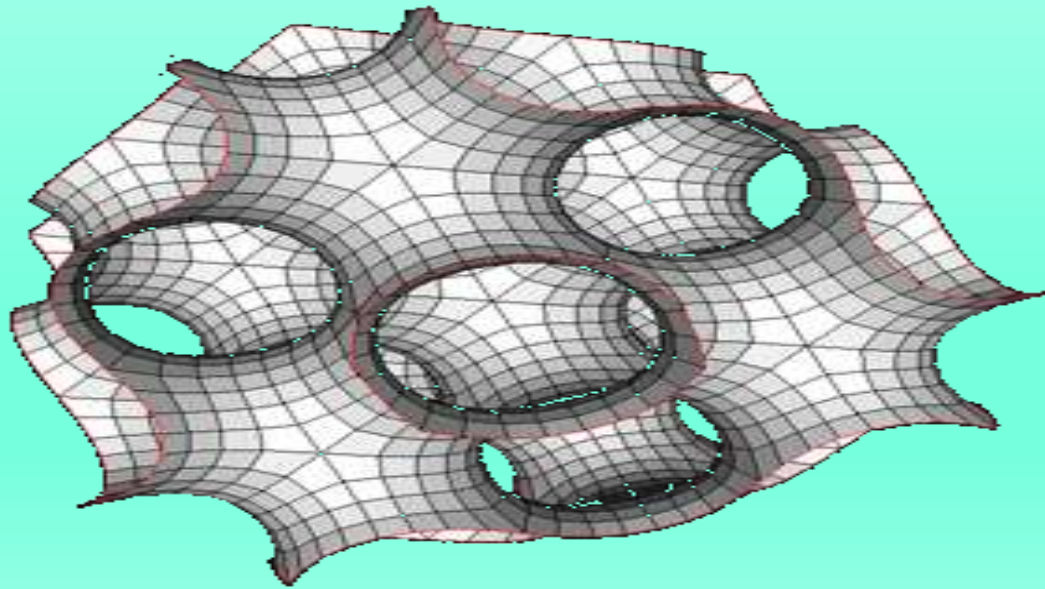
# Using `continue` Operator

- `continue` operator ends the iteration of the inner-most loop
- Example: sum all odd numbers in  $[1, n]$  that are not divisors of 7:

```
int n = int.Parse(Console.ReadLine());
int sum = 0;
for (int i = 1; i <= n; i += 2)
{
 if (i % 7 == 0)
 {
 continue;
 }
 sum += i;
}
Console.WriteLine("sum = {0}", sum);
```

# foreach Loop

**Iteration over a Collection**



# For Loops

- The typical foreach loop syntax is:

```
foreach (Type element in collection)
{
 statements;
}
```

- Iterates over all elements of a collection
  - The element is the loop variable that takes sequentially all collection values
  - The collection can be list, array or other group of elements of the same type

# foreach Loop – Example

- Example of foreach loop:

```
string[] days = new string[] {
 "Monday", "Tuesday", "Wednesday", "Thursday",
 "Friday", "Saturday", "Sunday" };
foreach (string day in days)
{
 Console.WriteLine(day);
}
```

- ◆ The above loop iterates of the array of days
  - ◆ The variable day takes all its values

# Nested Loops

**Using Loops Inside a Loop**



# What Is Nested Loop?

- A composition of loops is called a **nested loop**
  - A loop inside another loop
- Example:

```
for (initialization; test; update)
{
 for (initialization; test; update)
 {
 statements;
 }
 ...
}
```

# Nested Loops

## Examples





# Triangle – Example

- Print the following triangle:

1

1 2

...

1 2 3 ... n

```
int n = int.Parse(Console.ReadLine());
for(int row = 1; row <= n; row++)
{
 for(int column = 1; column <= row; column++)
 {
 Console.Write("{0} ", column);
 }
 Console.WriteLine();
}
```

# Primes[N, M] – Example

- Print all prime numbers in the interval [n, m]:

```
int n = int.Parse(Console.ReadLine());
int m = int.Parse(Console.ReadLine());
for (int number = n; number <= m; number++)
{
 bool prime = true;
 int divider = 2;
 int maxDivider = Math.Sqrt(num);
 while (divider <= maxDivider)
 {
 if (number % divider == 0)
 {
 prime = false;
 break;
 }
 divider++;
 }
 if (prime)
 {
 Console.Write("{0} ", number);
 }
}
```



Programming Fundamentals  
605116

# 6

ISRA University  
Faculty of IT

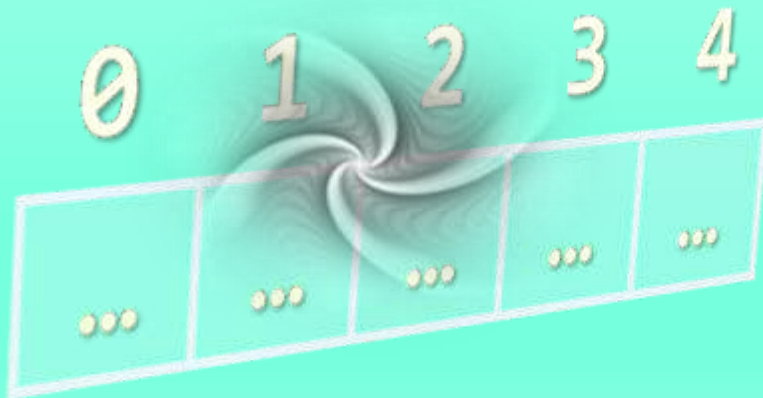
Textbook

BARBARA DOYLE **C# Programming: From Problem Analysis to Program Design 4<sup>th</sup> Edition**



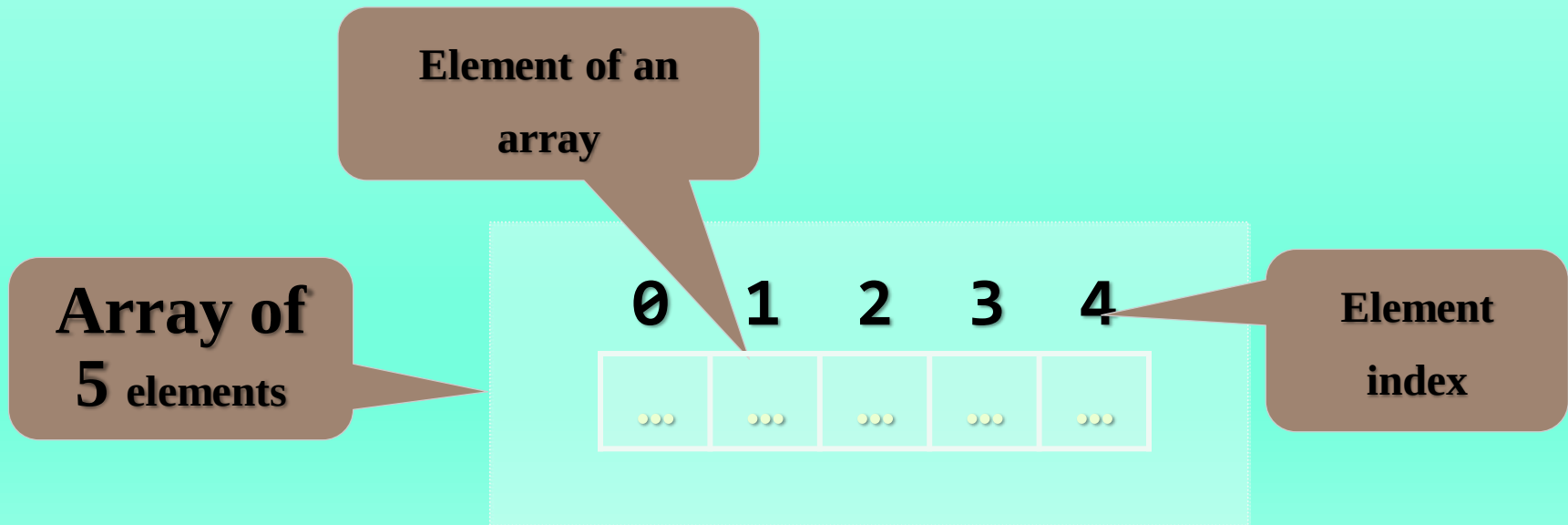
# Arrays

# Declaring and Creating Arrays



# What are Arrays?

- An array is a sequence of elements
  - All elements are of the same type
  - The order of the elements is fixed
  - Has fixed size (`Array.Length`)



# Declaring Arrays

- Declaration defines the type of the elements
- Square brackets [ ] mean "array"
- Examples:

- Declaring array of integers:

```
int[] myIntArray;
```

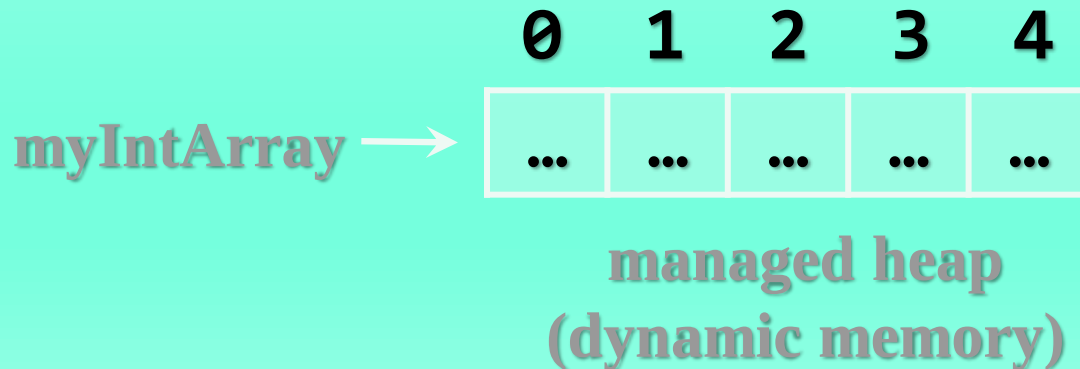
- Declaring array of strings:

```
string[] myStringArray;
```

# Creating Arrays

- Use the operator `new`
  - Specify array length
- Example creating (allocating) array of 5 integers:

```
myIntArray = new int[5];
```

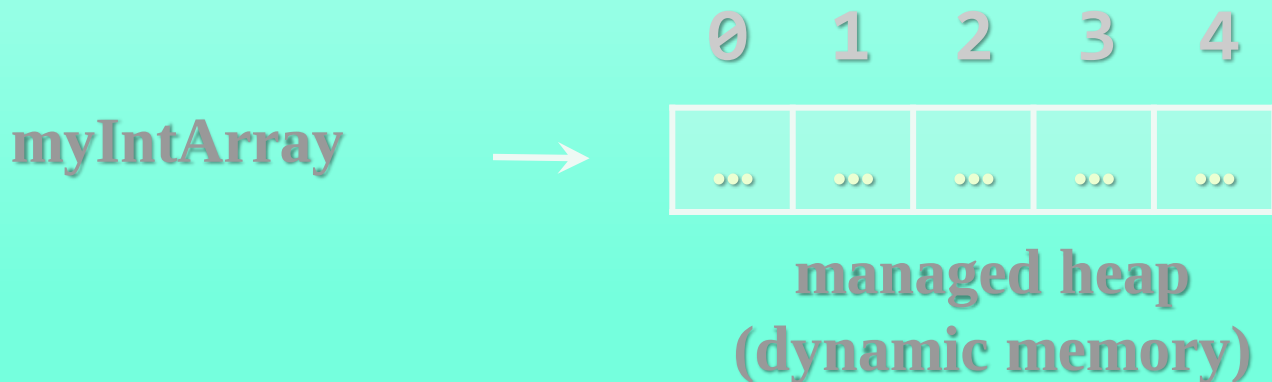




# Creating and Initializing Arrays

- Creating and initializing can be done together:

```
myIntArray = {1, 2, 3, 4, 5};
```



- The new operator is not required when using curly brackets initialization

# Creating Array – Example

- Creating an array that contains the names of the days of the week

```
string[] daysOfWeek =
{
 "Monday",
 "Tuesday",
 "Wednesday",
 "Thursday",
 "Friday",
 "Saturday",
 "Sunday"
};
```

# **Accessing Array Elements**

Read and Modify Elements by Index

# How to Access Array Element?

- Array elements are accessed using the square brackets operator [ ] (indexer)
  - Array indexer takes element's index as parameter
  - The first element has index 0
  - The last element has index Length-1
- Array elements can be retrieved and changed by the [ ] operator

# Reversing an Array – Example

- Reversing the contents of an array

```
int[] array = new int[] {1, 2, 3, 4, 5};
```

```
// Get array size
```

```
int length = array.Length;
```

```
// Declare and create the reversed array
```

```
int[] reversed = new int[length];
```

```
// Initialize the reversed array
```

```
for (int index = 0; index < length; index++)
```

```
{
```

```
 reversed[length-index-1] = array[index];
```

```
}
```

# Arrays: Input and Output

Reading and Printing Arrays on the Console

# Reading Arrays From the Console

- First, read from the console the length of the array

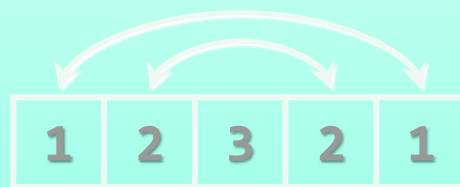
```
int n = int.Parse(Console.ReadLine());
```

- Next, create the array of given size and read its elements in a for loop

```
int[] arr = new int[n];
for (int i=0; i<n; i++)
{
 arr[i] = int.Parse(Console.ReadLine());
}
```

# Symmetry Check – Example

- Read `int` array from the console and check if it is symmetric:



```
bool isSymmetric = true;
for (int i=0; i<(array.Length+1)/2; i++)
{
 if (array[i] != array[n-i-1])
 {
 isSymmetric = false;
 }
}
```



# Printing Arrays on the Console

- Process all elements of the array
- Print each element to the console
- Separate elements with white space or a new line

```
string[] array = {"one", "two", "three"};

// Process all elements of the array
for (int index = 0; index < array.Length; index++)
{
 // Print each element on a separate line
 Console.WriteLine("element[{0}] = {1}",
 index, array[index]);
}
```

# Processing Array Elements Using `for` and `foreach`

# Processing Arrays: ~~for~~ Statement

- Use ~~for~~ loop to process an array when
  - Need to keep track of the index
  - Processing is not strictly sequential from the first to the last element
- In the loop body use the element at the loop index (`array[index]`):

```
for (int index = 0; index < array.Length; index++)
{
 squares[index] = array[index] * array[index];
}
```

# Processing Arrays Using for Loop – Examples

- Printing array of integers in reversed order:

```
Console.WriteLine("Reversed: ");
for (int i = array.Length-1; i >= 0; i--)
{
 Console.Write(array[i] + " ");
}
// Result: 5 4 3 2 1
```

- Initialize all array elements with their corresponding index number:

```
for (int index = 0; index < array.Length-1; index++)
{
 array[index] = index;
}
```

# Processing Arrays: foreach

`foreach (type value in array)`

- How foreach loop works?
  - type – the type of the element
  - value – local name of variable
  - array – processing array
- Used when no indexing is needed
  - All elements are accessed one by one
  - Elements can not be modified (read only)

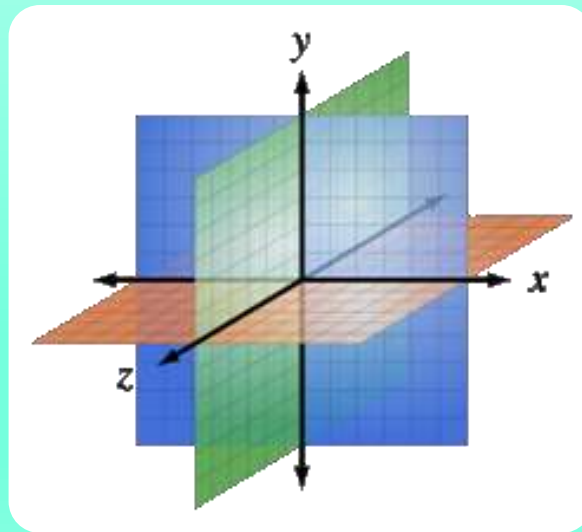
# Processing Arrays Using foreach – Example

- Print all elements of a `string[]` array:

```
string[] capitals =
{
 "Sofia",
 "Washington",
 "London",
 "Paris"
};
foreach (string capital in capitals)
{
 Console.WriteLine(capital);
}
```

# Multidimensional Arrays

Using Array of Arrays, Matrices and Cubes



# What is Multidimensional Array?

- Multidimensional arrays have more than one dimension (2, 3, ...)
  - The most important multidimensional arrays are the 2-dimensional
    - Known as matrices or tables
- Example of matrix of integers with 2 rows and 4 columns:

|   | 0 | 1 | 2  | 3 |
|---|---|---|----|---|
| 0 | 5 | 0 | -2 | 4 |
| 1 | 5 | 6 | 7  | 8 |



# Declaring and Creating Multidimensional Arrays

- Declaring multidimensional arrays:

```
int[,] intMatrix;
float[,] floatMatrix;
string[,,] strCube;
```

- Creating a multidimensional array
  - Use new keyword
  - Must specify the size of each dimension

```
int[,] intMatrix = new int[3, 4];
float[,] floatMatrix = new float[8, 2];
string[,,] stringCube = new string[5, 5, 5];
```

# Initializing Multidimensional Arrays with Values

- Creating and initializing with values multidimensional array:

```
int[,] matrix =
{
 {1, 2, 3, 4}, // row 0 values
 {5, 6, 7, 8}, // row 1 values
}; // The matrix size is 2 x 4 (2 rows, 4 cols)
```

- Matrices are represented by a list of rows
  - Rows consist of list of values
- The first dimension comes first, the second comes next (inside the first)

# Accessing The Elements of Multidimensional Arrays

`nDimensionalArray[index1, ... , indexn]`

- Accessing N-dimensional array element:

```
int[,] array = {{1, 2}, {3, 4}}
int element11 = array[1, 1]; //element11 = 4
```

- Getting element value example:
- Setting element value example:

Number  
of rows

```
int[,] array = new int[3, 4];
for (int row=0; row<array.GetLength(0); row++)
 for (int col=0; col<array.GetLength(1); col++)
 array[row, col] = row + col;
```

Number of  
columns

# Reading Matrix – Example

- Reading a matrix from the console

```
int rows = int.Parse(Console.ReadLine());
int columns = int.Parse(Console.ReadLine());
int[,] matrix = new int[rows, columns];
String inputNumber;
for (int row=0; row<rows; row++)
{
 for (int column=0; column<cols; column++)
 {
 Console.Write("matrix[{0},{1}] = ", row, column);
 inputNumber = Console.ReadLine();
 matrix[row, column] = int.Parse(inputNumber);
 }
}
```

# Printing Matrix – Example

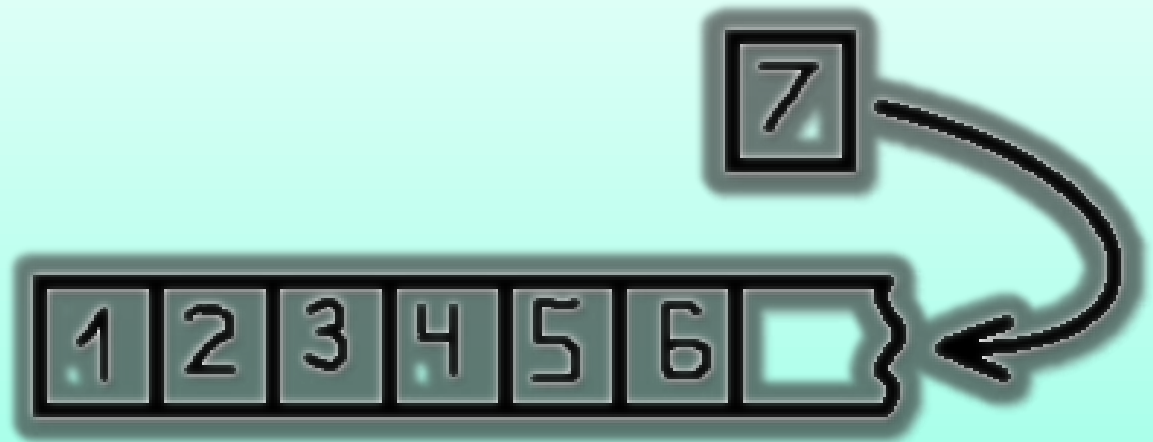
- Printing a matrix on the console:

```
for (int row=0; row<matrix.GetLength(0); row++)
{
 for (int col=0; col<matrix.GetLength(1); col++)
 {
 Console.Write("{0} ", matrix[row, col]);
 }
 Console.WriteLine();
}
```

# Maximal Platform – Example

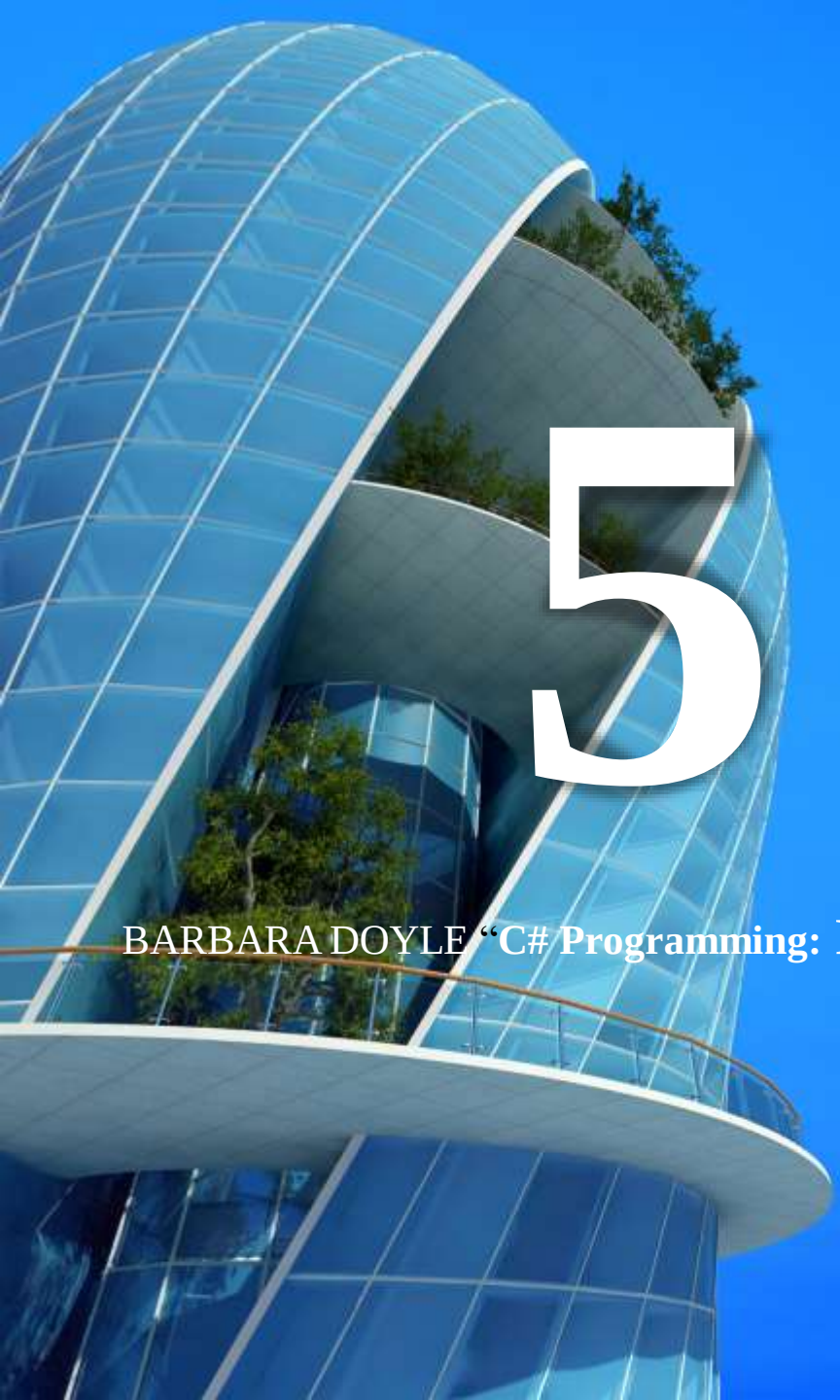
- Finding a 2 x 2 platform in a matrix with a maximal sum of its elements

```
int[,] matrix = {
 {7, 1, 3, 3, 2, 1},
 {1, 3, 9, 8, 5, 6},
 {4, 6, 7, 9, 1, 0}
};
int bestSum = int.MinValue;
for (int row=0; row<matrix.GetLength(0)-1; row++)
 for (int col=0; col<matrix.GetLength(1)-1; col++)
 {
 int sum = matrix[row, col] + matrix[row, col+1]
 + matrix[row+1, col] + matrix[row+1, col+1];
 if (sum > bestSum)
 bestSum = sum;
 }
```



# Dynamic Arrays

List<T>



Programming Fundamentals  
605116

ISRA University  
Faculty of IT

Textbook

BARBARA DOYLE “C# Programming: From Problem Analysis to Program Design 4<sup>th</sup>  
Edition





# C# - Methods

# Methods

- A **method** is a series of statements that carry out a task
- A method can be **invoked** or **called** by other methods
- Method names are always followed by a set of parentheses
- Programmers can create custom methods or call method already defined in the library
- Example:

```
Console.WriteLine("HELLO WORLD");
```

# Method

- To use a method, you need to –  
    Define the method  
    Call the method
- Defining Methods in C#
- When you define a method, you basically declare the elements of its structure. The syntax for defining a method in C# is as follows –

```
<Access Specifier> <Return Type> <Method Name>(Parameter List)
{
Method Body
}
```

# Defining Methods

- Following are the various elements of a method –
  - Access Specifier** – This determines the visibility of a variable or a method from another class.
  - Return type** – A method may return a value. The return type is the data type of the value the method returns. If the method is not returning any values, then the return type is **void**.
  - Method name** – Method name is a unique identifier and it is case sensitive. It cannot be same as any other identifier declared in the class.

# Defining Methods

- Following are the various elements of a method –  
**Parameter list** – Enclosed between parentheses, the parameters are used to pass and receive data from a method. The parameter list refers to the type, order, and number of the parameters of a method. Parameters are optional; that is, a method may contain no parameters.  
**Method body** – This contains the set of instructions needed to complete the required activity.

# Example of Defining Method

```
public int FindMax(int num1, int num2)
{
 /* local variable declaration */
 int result;
 if (num1 > num2)
 result = num1;
 else
 result = num2;
 return result;
}
```

# Calling Methods in C#

```
class Program
{
 static void Main(string[] args)
 {
 /* local variable definition */
 int a = 100;
 int b = 200;
 int ret;

 //calling the FindMax method
 ret = FindMax(a, b);
 Console.WriteLine("Max value is : {0}", ret);
 Console.ReadLine();

 }
 public static int FindMax(int num1, int num2)
 {
 /* local variable declaration */
 int result;

 if (num1 > num2)
 result = num1;
 else
 result = num2;
 return result;
 }
}
```

# Passing Parameters to a Method

- When method with parameters is called, you need to pass the parameters to the method. There are three ways that parameters can be passed to a method —



| Sr.No. | Mechanism & Description                                                                                                                                                                                                                                                                                    |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1      | <b>Value parameters</b> <br>This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument. |
| 2      | <b>Reference parameters</b> <br>This method copies the reference to the memory location of an argument into the formal parameter. This means that changes made to the parameter affect the argument.                      |



# C# - Passing Parameters by Value

- This is the default mechanism for passing parameters to a method. In this mechanism, when a method is called, a new storage location is created for each value parameter.
- The values of the actual parameters are copied into them. Hence, the changes made to the parameter inside the method have no effect on the argument. The following example demonstrates the concept

# Call by value Example

```
static void Main(string[] args)
{ /* local variable definition */
 int a = 100;
 int b = 200;

 Console.WriteLine("Before swap, value of a : {0}", a);
 Console.WriteLine("Before swap, value of b : {0}", b);

 /* calling a function to swap the values */
 swap(a, b);

 Console.WriteLine("After swap, value of a : {0}", a);
 Console.WriteLine("After swap, value of b : {0}", b);

 Console.ReadLine();
}

public static void swap(int x, int y) {
 int temp;

 temp = x; /* save the value of x */
 x = y; /* put y into x */
 y = temp; /* put temp into y */
}
```

# C# - Passing Parameters by Reference

- reference parameter is a **reference to a memory location** of a variable. When you pass parameters by reference, unlike value parameters, a new storage location is not created for these parameters. The reference parameters represent the same memory location as the actual parameters that are supplied to the method.
- You can declare the reference parameters using the **ref** keyword

# Call by reference Example

```
static void Main(string[] args)
{ /* local variable definition */
 int a = 100;
 int b = 200;

 Console.WriteLine("Before swap, value of a : {0}", a);
 Console.WriteLine("Before swap, value of b : {0}", b);

 /* calling a function to swap the values */
 swap(ref a, ref b);

 Console.WriteLine("After swap, value of a : {0}", a);
 Console.WriteLine("After swap, value of b : {0}", b);

 Console.ReadLine();
}

public static void swap(ref int x, ref int y) {
 int temp;

 temp = x; /* save the value of x */
 x = y; /* put y into x */
 y = temp; /* put temp into y */
}
```